THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that **Ms.RAJESWARI V(20524UCSC059),** a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** ▪ CREATE ▪ ALTER ▪ DROP ▪ TRUNCATE		
2	**DML Commands** ▪ INSERT ▪ UPDATE ▪ DELETE		
3	**DQL Commands** ▪ SELECT ▪ WHERE ▪ ORDER BY ▪ GROUP BY ▪ HAVING		
4	**Transactions** ▪ COMMIT ▪ ROLLBACK ▪ SAVEPOINT		
5	**Joins** ▪ Inner Join ▪ Left Join ▪ Right Join ▪ Full Join ▪ Self Join		
6	**SQL Functions** ▪ Aggregate Functions ▪ String Functions ▪ Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: ▪ **Khan Academy** ▪ **Hacker Rank** ▪ **Code Chef**		

EX. NO:01 Creation of a DDL operations using Students Table

DATE: 10.8.2026

Aim

To create a Students table, insert student records, display all records, and retrieve students who scored above 75 marks.

Software Used

Oracle /MySQL/sqlite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects in a database.

DDL COMMANDS:

- Create
- Alter
- Drop
- Truncate

CREATE: Used to create a new table with constraints.

Syntax

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

```
SQL>CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    name TEXT,
```

marks INTEGER

);

Table Created

ALTER: Used to change the structure of a table. Has three operations: ADD, MODIFY, DROP.

ADD: Add command is used to insert a new attribute/column with data type.

Syntax:

ALTER TABLE table_name ADD column_name datatype;

Query:

ALTER TABLE students ADD department VARCHAR(50);

Table Altered

MODIFY: Modify command is used to modify or change the data type and to increase the data size in the table not the attribute/column.

Syntax:

ALTER TABLE table_name MODIFY column_name datatype;

Query:

ALTER TABLE students MODIFY name VARCHAR(100);

ALTER TABLE students ALTER COLUMN name VARCHAR(100);

DROP: Drop command is used to delete the attribute/column in a table.

Syntax:

ALTER TABLE table_name DROP COLUMN column_name;

Query:

ALTER TABLE students DROP COLUMN department;

RENAME:Rename command is used to re-change the table name.

RENAME TABLE students TO learners;

Table renamed

ALTER TABLE students RENAME TO learners;

Column Rename

ALTER TABLE students RENAME COLUMN name TO student_name;

TRUNCATE: Truncate command is used to delete a value in the table.

Syntax: TRUNCATE TABLE tablename;

Query:

TRUNCATE TABLE students;

DROP: Deletes the table and all its data permanently.

Syntax:DROP TABLE tablename;

Query:

DROP TABLE students;

Explanation

Create a table named students.

- Insert student details into the table.
- Display all student records.
- Retrieve students who scored above 75 marks, sorted by marks.

Create Table

```
CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    name TEXT,
    marks INTEGER
);
```

Sample Data

```
INSERT INTO students VALUES (1, 'Arun', 82);
INSERT INTO students VALUES (2, 'Divya', 68);
```

INSERT INTO students VALUES (3, 'Kishore', 91);

INSERT INTO students VALUES (4, 'Meena', 74);

INSERT INTO students VALUES (5, 'Rahul', 55);

INSERT INTO students VALUES (6, 'Sneha', 88);

SQL Query

Query 1: Display all `student` records

SQL> SELECT * FROM students;

Resultant Query

id	name	marks
1	Arun	82
2	Divya	68
3	Kishore	91
4	Meena	74
5	Rahul	55
6	Sneha	88

Query 2: Display students who scored above 75 marks

SELECT *

FROM students

WHERE marks > 75

ORDER BY marks;

Resultant Query

id	name	release_year
4	Arun	82
1	Sneha	88
6	Kishore	91

RESULT:

Thus, the Students table was created successfully, records were inserted, all student details were displayed, and the students who scored above 75 marks were retrieved successfully.

EX. NO: 02 Creation of a DML operations using Employees

DATE:11.8.2026

Aim

To insert employee records, update an employee's details, delete an employee record, and retrieve employees based on conditions using DML commands.

SoftwareUsed

Oracle /MySQL/sqlite3

Description

DML (Data Manipulation Language), it is used to manipulate the data present in the database.

DML COMMANDS:

- Insert
- Update
- Delete

INSERT: Insert command is used to add new records into a table.

Syntax :

INSERT INTO table_name VALUES (value1, value2, value3, ...);

Query :

SQL> INSERT INTO employees VALUES (1, 'Arjun', 'Sales', 25000);

UPDATE: Update command is used to modify existing records in a table.

Syntax:

UPDATE table_name SET column_name = value WHERE condition;

Query:

SQL> UPDATE employees SET salary = 50000 WHERE id = 3;

DELETE: Delete command is used to remove one or more records from a table.

Syntax:

DELETE FROM table_name WHERE condition;

Query:

SQL> DELETE FROM employees WHERE id = 5;

Explanation

• Create a table named employees.

• Insert employee details into the table.

• Update the salary of a particular employee.

• Delete a particular employee record.

• Retrieve employees whose salary is above 30000.

Create Table

```
CREATE TABLE employees (
    id INTEGER PRIMARY KEY,
    name TEXT,
    department TEXT,
    salary INTEGER
);
```

Sample Data

```
INSERT INTO employees VALUES (1, 'Arjun', 'Sales', 25000);
INSERT INTO employees VALUES (2, 'Priya', 'HR', 30000);
INSERT INTO employees VALUES (3, 'Karthik', 'IT', 45000);
INSERT INTO employees VALUES (4, 'Divya', 'Finance', 38000);
INSERT INTO employees VALUES (5, 'Suresh', 'Sales', 27000);
INSERT INTO employees VALUES (6, 'Meena', 'IT', 42000);
```

SQL Query

Query 1: Update the salary of employee with id 3

```
SQL> UPDATE employees SET salary = 50000 WHERE id = 3;
```

Query 2: Delete the employee record with id 5

```
SQL> DELETE FROM employees WHERE id = 5;
```

Query 3: Retrieve employees whose salary is above 30000

```
SQL> SELECT *
FROM employees
WHERE salary > 30000
ORDER BY salary;
```

Table after all operations

ID	NAME	DEPARTMENT	SALARY
1	Arjun	Sales	25000
2	Priya	HR	30000
3	Karthik	IT	500000
4	Divya	Finance	38000
5	Meena	IT	42000

RESULT:

Thus, the Employees table records were inserted successfully, an employee's salary was updated,an employee record was deleted, and employees with salary above 30000 were retrieved successfully.

EX. NO: 03 Implementation of DQL Commands using Products Table

DATE:12.8.2026

Aim

To retrieve and display data from the Products table using DQL commands such as SELECT, WHERE, ORDER BY, GROUP BY and HAVING.

SoftwareUsed

Oracle /MySQL/sqlite3

Description

DQL (Data Query Language), it is used to retrieve/query data from the database.

DQL COMMANDS:

- Select
- Where
- Order by
- Group by
- Having

SELECT : Select command is used to retrieve/display records from a table.

Syntax :

SELECT column1, column2, ... FROM table_name;

Query:

SQL> SELECT * FROM products;

WHERE: Where clause is used to filter records based on a condition.

Syntax:

SELECT * FROM table_name WHERE condition;

Query:

SQL> SELECT * FROM products WHERE price > 500;

ORDER BY: Order by clause is used to sort the result in ascending or descending order.

Syntax:

SELECT * FROM table_name ORDER BY column_name ASC/DESC;

Query:

SQL> SELECT * FROM products ORDER BY price DESC;

GROUP BY: Group by clause is used to group rows that have the same values in specified columns.

Syntax:

SELECT column_name, aggregate_function(column_name) FROM table_name GROUP BY column_name;

Query:

SQL> SELECT category, COUNT(*) FROM products GROUP BY category;

HAVING :Having clause is used to filter groups based on a condition, used along with GROUP BY.

Syntax:

SELECT column_name, aggregate_function(column_name) FROM table_name GROUP BY column_name HAVING condition;

Explanation

• Create a table named products.

• Insert product details into the table.

• Retrieve products whose price is above 500 using WHERE.

• Sort the products by price in descending order using ORDER BY.

• Group products category wise and count them using GROUP BY.

• Filter categories whose average price is above 500 using HAVING.

Create Table

```
CREATE TABLE products (
    id INTEGER PRIMARY KEY,
    name TEXT,
    category TEXT,
```

price INTEGER

);

Sample Data

INSERT INTO products VALUES (1, 'Mobile', 'Electronics', 15000);

INSERT INTO products VALUES (2, 'Shirt', 'Clothing', 800);

INSERT INTO products VALUES (3, 'Laptop', 'Electronics', 45000);

INSERT INTO products VALUES (4, 'Notebook', 'Stationery', 50);

INSERT INTO products VALUES (5, 'Shoes', 'Clothing', 1200);

INSERT INTO products VALUES (6, 'Pen', 'Stationery', 20);

SQL Query

Query 1: Display products whose price is above 500

SQL> SELECT *

FROM products

WHERE price > 500;

Resultant Query

ID	NAME	CATEGORY	PRICE
1	MOBILE	ELECTRONICS	15000
2	SHIRT	CLOTHING	800
3	LAPTOP	ELECTRONICS	45000
4	SHOES	CLOTHING	12000

Query 2: Display products sorted by price in descending order

SQL> SELECT *

FROM products

ORDER BY price DESC;

Resultant Query

ID	NAME	CATEGORY	PRICE
3	Laptop	Electronics	4500
1	Mobile	Electronics	15000
5	Shoes	Clothing	1200
2	Shirt	Clothing	800
4	Notebook	Stationery	5

Query 3: Count the number of products in each category

SQL> SELECT category, COUNT(*)

FROM products

GROUP BY category;

Resultant Query

CATEGORY	COUNT
Clothing	2
Electronics	2
Stationery	2

Query 4: Display categories whose average price is above 500

SQL> SELECT category, AVG(price)

FROM products

GROUP BY category

HAVING AVG(price) > 500;

Resultant Query

NAME	PRICE
MOBILE	15000
SHIRT	800
SHOES	12000

RESULT:

Thus, the DQL commands SELECT, WHERE, ORDER BY, GROUP BY and HAVING were implemented successfully on the Products table and the results were retrieved as required.

EX. NO: 04 Implementation of TCL Commands using Transactions Table

DATE:13.8.2026

Aim

To perform transaction control operations such as COMMIT, ROLLBACK and SAVEPOINT on a Transactions table.

SoftwareUsed

Oracle /MySQL/sqlite3

Description

TCL (Transaction Control Language), it deals with transactions within the database.

TCL COMMANDS:

- Commit
- Rollback
- Savepoint

COMMIT: Commit command is used to permanently save all the changes made in the current transaction.

Syntax:COMMIT;

Query: SQL> COMMIT;

ROLLBACK: Rollback command is used to undo the changes made in the current transaction that have not yet been committed.

Syntax: ROLLBACK;

Query:

SQL> ROLLBACK;

Rollback complete.

SAVEPOINT:

Savepoint command is used to set a point within a transaction to which you can later roll back,without undoing the whole transaction.

Syntax:

SAVEPOINT savepoint_name;

Query:

SQL> SAVEPOINT sp1;

Savepoint created.

Explanation

• Create a table named transactions.

• Insert transaction records into the table.

• Set a savepoint before performing further changes.

• Update a record after the savepoint.

• Roll back to the savepoint to undo only the changes made after it.

• Commit the final changes permanently.

Create Table

```
CREATE TABLE transactions (
        id INTEGER PRIMARY KEY,
        account_no TEXT,
        amount INTEGER,
        type TEXT
);
```

Sample Data

```
INSERT INTO transactions VALUES (1, 'AC101', 5000, 'Deposit');
INSERT INTO transactions VALUES (2, 'AC102', 2000, 'Withdrawal');
INSERT INTO transactions VALUES (3, 'AC103', 7000, 'Deposit');
COMMIT;
```

Commit complete.

SQL Query

Query 1: Display all transaction records

SQL> SELECT * FROM transactions;

Resultant Query

ID	Account_no	Amount	Type
1	AC101	5000	Deposit
2	AC102	2000	Withdrawl
3	AC103	7000	Deposit

Query 2: Create a savepoint before a new transaction

SQL> SAVEPOINT sp1;

Query 3: Insert a new transaction record after the savepoint

SQL> INSERT INTO transactions VALUES (4, 'AC104', 3000, 'Withdrawal');

1 row inserted.

Query 4: Roll back to the savepoint (undoes only the record inserted after sp1)

SQL> ROLLBACK TO sp1;

Rollback complete.

Query 5: Display transaction records after rollback

SQL> SELECT * FROM transactions;

Resultant Query

ID	Account_no	Amount	Type
1	AC101	5000	Deposit
2	AC102	2000	Withdrawl
3	AC103	7000	Deposit

Query 6: Commit the final state permanently

SQL> COMMIT;

Commit complete.

RESULT:

Thus, the TCL commands COMMIT, ROLLBACK and SAVEPOINT were implemented successfully on the Transactions table.

EX. NO: 05 Implementation of Joins using Employees and Departments Table

DATE:14.8.2026

Aim

To retrieve data from multiple tables using different types of joins such as Inner Join, Left Join, Right Join, Full Join and Self Join.

SoftwareUsed

Oracle /MySQL/sqlite3

Description

A Join is used to combine rows from two or more tables based on a related column between them.

JOINS:

- Inner Join
- Left Join
- Right Join
- Full Join
- Self Join

INNER JOIN: Returns only the matching rows present in both tables.

Syntax:

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

Query:

```
SQL> SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

LEFT JOIN :: Returns all rows from the left table and the matching rows from the right table. Unmatched right side shows NULL.

Syntax:

SELECT columns

FROM table1

LEFT JOIN table2

ON table1.column = table2.column;

Query:

SQL> SELECT e.name, d.dept_name

FROM employees e

LEFT JOIN departments d

ON e.dept_id = d.dept_id;

RIGHT JOIN: Returns all rows from the right table and the matching rows from the left table. Unmatched left side shows NULL

Syntax:

SELECT columns

FROM table1

RIGHT JOIN table2

ON table1.column = table2.column;

Query:

SQL> SELECT e.name, d.dept_name

FROM employees e

RIGHT JOIN departments d

ON e.dept_id = d.dept_id;

FULL JOIN: Returns all rows from both tables, with NULLs where there is no match on either side.

Syntax:

SELECT columns

FROM table1

FULL JOIN table2

ON table1.column = table2.column;

Query:

SQL> SELECT e.name, d.dept_name

FROM employees e

FULL JOIN departments d

ON e.dept_id = d.dept_id;

SELF JOIN: A table is joined with itself to compare rows within the same table.

Syntax:

SELECT a.column, b.column

FROM table_name a, table_name b

WHERE condition;

Query:

SQL> SELECT a.name AS employee, b.name AS manager

FROM employees a, employees b

WHERE a.manager_id = b.id;

Explanation

• Create employees and departments tables.

• Insert records into both tables.

• Retrieve matching employee-department details using INNER JOIN.

• Retrieve all employees along with their department (if any) using LEFT JOIN.

• Retrieve all departments along with their employees (if any) using RIGHT JOIN.

• Retrieve all employees and departments, matched or not, using FULL JOIN.

• Retrieve employee-manager relationship using SELF JOIN.

Create Tables

```
CREATE TABLE departments (
        dept_id INTEGER PRIMARY KEY,
        dept_name TEXT
);

CREATE TABLE employees (
```

```
    id INTEGER PRIMARY KEY,
    name TEXT,
    dept_id INTEGER,
    manager_id INTEGER
);
```

Sample Data

```
INSERT INTO departments VALUES (101, 'Sales');
INSERT INTO departments VALUES (102, 'HR');
INSERT INTO departments VALUES (103, 'IT');
INSERT INTO departments VALUES (104, 'Finance');

INSERT INTO employees VALUES (1, 'Arjun', 101, NULL);
INSERT INTO employees VALUES (2, 'Priya', 102, 1);
INSERT INTO employees VALUES (3, 'Karthik', 103, 1);
INSERT INTO employees VALUES (4, 'Divya', NULL, 1);
```

SQL Query

Query 1: Inner Join – employees with matching departments

```
SQL> SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

Resultant Query

NAME	dept_name
Arjun	Sales
Priya	HR
Karthik	IT

Query 2: Left Join – all employees, department if matched

```
SQL> SELECT e.name, d.dept_name
```

FROM employees e

LEFT JOIN departments d

ON e.dept_id = d.dept_id;

Resultant Query

NAME	dept_name
Arjun	Sales
Priya	HR
Karthik	IT
Divya	Null

Query 3: Right Join – all departments, employee if matched

SQL> SELECT e.name, d.dept_name

FROM employees e

RIGHT JOIN departments d

ON e.dept_id = d.dept_id;

Resultant Query

NAME	dept_name
Arjun	Sales
Priya	HR
Karthik	IT
NULL	Finance

Query 4: Full Join – all employees and departments, matched or not

SQL> SELECT e.name, d.dept_name

FROM employees e

FULL JOIN departments d

ON e.dept_id = d.dept_id;

Resultant Query

NAME	dept_name
Arjun	Sales
Priya	HR
Karthik	IT
Divya	NULL
NULL	Finance

Query 5: Self Join – employee and their manager

SQL> SELECT a.name AS employee, b.name AS manager

FROM employees a, employees b

WHERE a.manager_id = b.id;

Resultant Query

Employee	Manager
Divya	Arjun
Priya	Arjun
Karthik	Arjun

RESULT:

Thus, the joins Inner Join, Left Join, Right Join, Full Join and Self Join were implemented successfully using the Employees and Departments tables.

EX. NO: 06 Implementation of SQL Functions using Employees Table

DATE:15.8.2026

Aim

To implement Aggregate Functions, String Functions and Date Functions on the Employees table.

SoftwareUsed

Oracle /MySQL/sqlite3

Description

SQL Functions are built-in functions used to perform calculations, manipulate strings, and handle date values on the data stored in a database.

SQL FUNCTIONS:

- Aggregate Functions
- String Functions
- Date Functions

AGGREGATE FUNCTIONS: Used to perform calculations on a set of values and return a single value. Examples: COUNT, SUM, AVG, MAX, MIN.

Syntax:

SELECT AGGREGATE_FUNCTION(column_name) FROM table_name;

Query:

SQL> SELECT COUNT(*), SUM(salary), AVG(salary), MAX(salary), MIN(salary)

FROM employees;

STRING FUNCTIONS :Used to perform operations on string/text data. Examples: UPPER, LOWER, LENGTH, SUBSTR, CONCAT.

Syntax:

SELECT STRING_FUNCTION(column_name) FROM table_name;

Query:

SQL> SELECT UPPER(name), LOWER(name), LENGTH(name)

FROM employees;

DATEFUNCTIONS: Used to perform operations on date/time data. Examples: CURRENT_DATE, DATE, STRFTIME, DATEDIFF

Syntax:

SELECT DATE_FUNCTION(column_name) FROM table_name;

Query:

SQL> SELECT name, hire_date, CURRENT_DATE

FROM employees;

Explanation

• Create a table named employees with a hire date column.

• Insert employee details into the table.

• Use aggregate functions to find count, sum, average, maximum and minimum of salary.

• Use string functions to convert names to uppercase, lowercase and find their length.

• Use date functions to display the hire date and calculate the number of years worked.

Create Table

```
CREATE TABLE employees (
        id INTEGER PRIMARY KEY,
        name TEXT,
        department TEXT,
        salary INTEGER,
        hire_date DATE
);
```

Sample Data

INSERT INTO employees VALUES (1, 'Arjun', 'Sales', 25000, '2018-06-10');

INSERT INTO employees VALUES (2, 'Priya', 'HR', 30000, '2019-03-15');

INSERT INTO employees VALUES (3, 'Karthik', 'IT', 45000, '2020-01-22');

INSERT INTO employees VALUES (4, 'Divya', 'Finance', 38000, '2021-07-05');

SQL Query

Query 1: Aggregate functions on salary

```
SQL> SELECT COUNT(*) AS total_employees,
     SUM(salary) AS total_salary,
     AVG(salary) AS average_salary,
     MAX(salary) AS highest_salary,
     MIN(salary) AS lowest_salary
FROM employees;
```

Resultant Query

Total_emp	Total_sal	Avg_sal	Hight_sal	Looest_sal
4	138000	34500	45000	25000

Query 2: String functions on employee name

```
SQL> SELECT name,
     UPPER(name) AS upper_name,
     LOWER(name) AS lower_name,
     LENGTH(name) AS name_length
FROM employees;
```

Resultant Query

Name	Upper	Lower	Nme_len
Arjun	ARJUN	arjun	5
priya	PRIYA	priya	5
karthik	KARTHI	karthik	7

Divya	DIVYA	divya	5

Query 3: Date functions on hire date

```
SQL> SELECT name, hire_date,
     (STRFTIME('%Y', 'now') - STRFTIME('%Y', hire_date)) AS years_worked
FROM employees;
```

Resultant Query

Name	**hire_date**	**years_worked**
Arjun	2018-06-10	8
priya	2019-03-15	7
karthik	2020-01-22	6
Divya	2021-07-05	5

RESULT:

Thus, the Aggregate Functions, String Functions and Date Functions were implemented successfully on the Employees table.

EX. NO: 07 Implementation of Subqueries and Nested Queries using Employees Table

DATE:16.9.2026

Aim

To retrieve data using subqueries and nested queries on the Employees table.

SoftwareUsed

Oracle /MySQL/sqlite3

Description

A subquery is a query written inside another query. The inner query runs first and its result is used by the outer query. Nesting one subquery inside another is called a nested query.

Syntax

SELECT column_name FROM table_name

WHERE column_name operator (SELECT column_name FROM table_name WHERE condition);

Explanation

• Create a table named employees and insert records.

• Find employees earning more than the average salary.

• Find the employee with the highest salary.

• Find employees working in the department that has a salary above 40000, using IN.

• Find the second highest salary using a nested query.

Create Table

```
CREATE TABLE employees (
    id INTEGER PRIMARY KEY,
    name TEXT,
    department TEXT,
    salary INTEGER
```

);

Sample Data

INSERT INTO employees VALUES (1, 'Arjun', 'Sales', 25000);

INSERT INTO employees VALUES (2, 'Priya', 'HR', 30000);

INSERT INTO employees VALUES (3, 'Karthik', 'IT', 45000);

INSERT INTO employees VALUES (4, 'Divya', 'Finance', 38000);

INSERT INTO employees VALUES (5, 'Suresh', 'Sales', 27000);

INSERT INTO employees VALUES (6, 'Meena', 'IT', 42000);

SQL Query

Query 1: Employees earning more than the average salary

SQL> SELECT * FROM employees

WHERE salary > (SELECT AVG(salary) FROM employees);

Resultant Query

ID	NAME	DEPARTMENT	SALARY
3	karthik	IT	45000
4	divya	Finance	3800
6	Meena	IT	42000

Query 2: Employee with the highest salary

SQL> SELECT * FROM employees

WHERE salary = (SELECT MAX(salary) FROM employees);

Resultant Query

ID	name	department	salary
3	karthik	IT	420000

Query 3: Employees in departments that have a salary above 40000

SQL> SELECT MAX(salary) AS second_highest

FROM employees

WHERE salary < (SELECT MAX(salary) FROM employees);

ID	NAME	DEPARTMENT	SALARY
3	karthik	IT	45000
4	meena	IT	42000

Query 4: Second highest salary (nested query)

SQL> SELECT MAX(salary) AS second_highest

FROM employees

WHERE salary < (SELECT MAX(salary) FROM employees);

Resultant Query

second_high
42000

RESULT:

Thus, the subqueries and nested queries were implemented successfully on the Employees table.

EX. NO: 08 Implementation of Views, Indexes and Sequences using Employees Table

DATE:17.9.2026

Aim

To create and use views, indexes and sequences on the Employees table.

SoftwareUsed

Oracle /MySQL

Description

VIEW: A view is a virtual table based on the result of a SELECT query. It does not store data itself.

INDEX: An index is used to speed up the searching of records in a table.

SEQUENCE: A sequence generates unique numbers automatically, mostly used for primary key values.

VIEW:

Syntax:

CREATE VIEW view_name AS SELECT columns FROM table_name WHERE condition;

View created.

Display the view:

SQL> SELECT * FROM it_employees;

Resultant Query

ID	NAME	DEPARTMENT	SALARY
3	karthik	IT	45000
6	meena	IT	42000

Drop the view:

SQL> DROP VIEW it_employees;

View dropped.

INDEX:

Syntax:

CREATE INDEX index_name ON table_name (column_name);

Query:

SQL> CREATE INDEX idx_emp_name ON employees (name);

Index created.

Display the indexes (MySQL):

SQL> SHOW INDEX FROM employees;

Drop the index:

SQL> DROP INDEX idx_emp_name ON employees;

Index dropped.

(In Oracle, use DROP INDEX idx_emp_name;)

SEQUENCE(Oracle):

Syntax:

CREATE SEQUENCE sequence_name START WITH value INCREMENT BY value;

Query:

SQL> CREATE SEQUENCE emp_seq START WITH 7 INCREMENT BY 1;

Sequence created.

Use the sequence while inserting:

SQL> INSERT INTO employees VALUES (emp_seq.NEXTVAL, 'Ravi', 'Sales', 28000);

1 row inserted.

SQL> SELECT * FROM employees WHERE name = 'Ravi';

Resultant Query

ID	NAME	DEPARTMENT	SALARY
7	karthik	salaes	28000

Drop the sequence:

SQL> DROP SEQUENCE emp_seq;

Sequence dropped.

MySQLalternativefor sequence: MySQL has no CREATE SEQUENCE, so AUTO_INCREMENT is used instead.

```
CREATE TABLE employees (
        id INT PRIMARY KEY AUTO_INCREMENT,
        name VARCHAR(50),
        department VARCHAR(50),
        salary INT
);
INSERT INTO employees (name, department, salary) VALUES ('Ravi', 'Sales', 28000);
```

Explanation

• Create a view to show only the IT department employees.

• Create an index on the name column for faster searching.

• Create a sequence to generate employee ids automatically.

RESULT:

Thus, the views, indexes and sequences were created and used successfully on the Employees table.

EX. NO: 9 Implementation of Stored Procedures and Functions using Employees Table

DATE:18.9.2026

Aim

To create and execute a stored procedure and a function on the Employees table.

SoftwareUsed

MySQL (Oracle PL/SQL syntax differs slightly)

Description

STORED PROCEDURE: A stored procedure is a group of SQL statements saved in the database that can be executed whenever required. It may or may not return a value.

FUNCTION: A function is a stored program that always returns a single value and can be used inside a SELECT query.

STOREDPROCEDURE:

Syntax:

```
CREATE PROCEDURE procedure_name (parameters)
BEGIN
   statements;
END;
```

Query:

```
SQL> DELIMITER //
CREATE PROCEDURE raise_salary (IN emp_id INT, IN amount INT)
BEGIN
   UPDATE employees SET salary = salary + amount WHERE id = emp_id;
END //
DELIMITER ;
```

Procedure created.

Execute the procedure:

SQL> CALL raise_salary(1, 5000);

Query , 1 row affected.

SQL> SELECT * FROM employees WHERE id = 1;

ID	NAME	DEPARTMENT	SALARY
1	Arjun	salaes	30000

Drop the procedure:

SQL> DROP PROCEDURE raise_salary;

FUNCTION:

Syntax:

```
CREATE FUNCTION function_name (parameters)
RETURNS datatype
DETERMINISTIC
BEGIN
   RETURN value;
END;
```

Query:

```
SQL> DELIMITER //
CREATE FUNCTION annual_salary (monthly INT)
RETURNS INT
DETERMINISTIC
BEGIN
   RETURN monthly * 12;
END //
DELIMITER ;
```

Function created.

Use the function:

SQL> SELECT name, salary, annual_salary(salary) AS yearly_salary

FROM employees WHERE id = 3;

Resultant Query

NAME	SALARY	YEAR_SALARY
Karthik	45000	540000

Drop the function:

SQL> DROP FUNCTION annual_salary;

Explanation

• Create a procedure that increases an employee's salary by a given amount.

• Execute the procedure using the CALL statement.

• Create a function that calculates the annual salary from the monthly salary.

• Use the function inside a SELECT query

RESULT:

Thus, the stored procedure and function were created and executed successfully on the Employees table.

EX. NO: 10 Creation of Triggers using the Students Table

DATE:19.9.2026

Aim

To create triggers on the Students table that automatically validate data and record every insert, update, and delete operation in a log table.

Software Used

Oracle / MySQL / SQLite3

Description

A **trigger** is a special stored program that executes automatically when a specified event (INSERT, UPDATE, or DELETE) occurs on a table. Triggers are used for auditing, validating data, and enforcing business rules.

Types of Triggers:

- **BEFORE trigger:** executes before the operation (used for validation).
- **AFTER trigger:** executes after the operation (used for logging and auditing).
- **NEW / OLD:** NEW holds the new row values (INSERT, UPDATE) and OLD holds the previous row values (UPDATE, DELETE).

Syntax (SQLite):

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE} ON table_name
[WHEN condition]
BEGIN
   statements;
END;
```

Syntax (MySQL):

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE} ON table_name
FOR EACH ROW
BEGIN
   statements;
END;
```

Dropping a trigger:

```
DROP TRIGGER trigger_name;
```

Explanation:

Create a log table to store the changes made to the students table.

Create a trigger to validate marks before insertion.

Create triggers to log INSERT, UPDATE, and DELETE operations.

Perform the operations and verify that the triggers fired.

Create Log Table

```
CREATE TABLE students_log (
   log_id      INTEGER PRIMARY KEY AUTOINCREMENT,
   student_id  INTEGER,
   action      TEXT,
   old_marks   INTEGER,
   new_marks   INTEGER,
   action_date TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Trigger 1: BEFORE INSERT (validate marks)

```
CREATE TRIGGER trg_check_marks
BEFORE INSERT ON students
WHEN NEW.marks < 0 OR NEW.marks > 100
BEGIN
   SELECT RAISE(ABORT, 'Marks must be between 0 and 100');
END;
```

MySQL version:

```
DELIMITER //
CREATE TRIGGER trg_check_marks
BEFORE INSERT ON students
FOR EACH ROW
BEGIN
   IF NEW.marks < 0 OR NEW.marks > 100 THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Marks must be between 0 and 100';
   END IF;
END//
DELIMITER ;
```

Trigger 2: AFTER INSERT (log new record)

```
CREATE TRIGGER trg_after_insert
AFTER INSERT ON students
BEGIN
   INSERT INTO students_log (student_id, action, new_marks)
   VALUES (NEW.id, 'INSERT', NEW.marks);
END;
```

Trigger 3: AFTER UPDATE (log old and new marks)

```
CREATE TRIGGER trg_after_update
AFTER UPDATE OF marks ON students
BEGIN
   INSERT INTO students_log (student_id, action, old_marks, new_marks)
   VALUES (OLD.id, 'UPDATE', OLD.marks, NEW.marks);
END;
```

Testing the Triggers

Test 1: Valid insert

```
INSERT INTO students VALUES (7, 'Priya', 79);
```

Test 2: Invalid insert (trigger blocks it)

```
INSERT INTO students VALUES (8, 'Test', 150);
```

Test 3: Update

```
UPDATE students SET marks = 60 WHERE id = 5;
```

Test 4: Delete

```
DELETE FROM students WHERE id = 2;
```

Display the log:

```
SELECT * FROM students_log;
```

Resultant Query

log_ID	STD_ID	Old mark	New mark
1	7	null	79
2	5	55	60
3	2	68	null

RESULT

Thus, the triggers were created successfully on the Students table. The BEFORE INSERT trigger rejected invalid marks, and the AFTER INSERT, UPDATE, and DELETE triggers automatically recorded each change in the students_log table.

EX. NO: 11 Implementation of Cursors

DATE:20.9.2026

Aim

To process the rows of the Employees table one at a time using implicit and explicit cursors in PL/SQL.

SoftwareUsed

Oracle (SQL*Plus / Live SQL)

Description

A cursor is a pointer to the memory area where Oracle stores the result of a query. An implicit cursor is created automatically for DML statements. An explicit cursor is declared by the programmer to fetch multiple rows one by one.

Cursor steps: DECLARE → OPEN → FETCH → CLOSE

Syntax

CURSOR cursor_name IS SELECT statement;

OPEN cursor_name;

FETCH cursor_name INTO variables;

CLOSE cursor_name;

Explanation

- Use the employees table from Experiment 07.
- Display all employees using an explicit cursor.
- Update salaries and count the affected rows using an implicit cursor.

Program 1: Explicit Cursor

```
SET SERVEROUTPUT ON;

DECLARE

   CURSOR emp_cur IS SELECT id, name, salary FROM employees;
```

```
    v_idemployees.id%TYPE;
    v_name   employees.name%TYPE;
    v_salary employees.salary%TYPE;
BEGIN
    OPEN emp_cur;
    LOOP
        FETCH emp_cur INTO v_id, v_name, v_salary;
        EXIT WHEN emp_cur%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_id || ' ' || v_name || ' ' || v_salary);
    END LOOP;
    CLOSE emp_cur;
END;
/
```

Output

```
1 Arjun 25000
2 Priya 30000
3 Karthik 45000
4 Divya 38000
5 Suresh 27000
6 Meena 42000
PL/SQL procedure successfully completed.
```

Program 2: Implicit Cursor

```
BEGIN
    UPDATE employees SET salary = salary + 1000 WHERE department = 'Sales';
```

```
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows updated');
END;
/
```

Output:

2 rows updated

RESULT:

Thus, the implicit and explicit cursors were implemented successfully on the Employees table.

EX. NO: 12 PL/SQL Exception Handling

DATE:21.9.2026

Aim

To handle runtime errors in PL/SQL using predefined and user-defined exceptions.

SoftwareUsed

Oracle (SQL*Plus / Live SQL)

Description

An exception is an error that occurs while a program is running. PL/SQL handles it in the EXCEPTION block so that the program does not terminate abruptly.

Syntax

```
BEGIN
   -- statements
EXCEPTION
   WHEN exception_name THEN
      -- handling statements
END;
```

Explanation

- NO_DATA_FOUND: raised when a SELECT INTO returns no rows.
- ZERO_DIVIDE: raised when a number is divided by zero.
- User-defined exception: declared by the programmer and raised with RAISE.

Program 1: Predefined Exceptions

```
SET SERVEROUTPUT ON;
DECLARE
   v_name employees.name%TYPE;
   v_result NUMBER;
```

```
BEGIN
   SELECT name INTO v_name FROM employees WHERE id = 10;
   DBMS_OUTPUT.PUT_LINE(v_name);
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('No employee found with ID 10');
END;
/
BEGIN
   DBMS_OUTPUT.PUT_LINE(100 / 0);
EXCEPTION
   WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');
END;
/
```

OUTPUT:

No employee found with ID 10

Cannot divide by zero

Program 2: User-Defined Exception

```
DECLARE
   low_salary EXCEPTION;
   v_salary employees.salary%TYPE;
BEGIN
   SELECT salary INTO v_salary FROM employees WHERE id = 1;
```

```
    IF v_salary < 30000 THEN
        RAISE low_salary;
    END IF;
    DBMS_OUTPUT.PUT_LINE('Salary is acceptable');
EXCEPTION
    WHEN low_salary THEN
        DBMS_OUTPUT.PUT_LINE('Salary is below the minimum limit');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Some other error occurred');
END;
/
```

OUTPUT:

Salary is below the minimum limit

RESULT:

Thus, the predefined and user-defined exceptions were handled successfully in PL/SQL.

EX. NO: 13 Database Normalization (1NF, 2NF, 3NF, BCNF)

DATE:22.9.2-026

Aim

To normalize a database table up to BCNF to remove redundancy and anomalies.

SoftwareUsed

Oracle / MySQL / sqlite3

Description

Normalization is the process of organizing data into tables to reduce redundancy and avoid insertion, update and deletion anomalies**.**

Normal Form Rule

1NF	Every column holds atomic (single) values; no repeating groups
2NF	In 1NF, and no partial dependency on part of a composite key
3NF	In 2NF, and no transitive dependency between non-key columns
BCNF	In 3NF, and every determinant is a candidate key

Unnormalized Table: STUDENT_COURSE

RegNo	SName	Course name	faculty	facultyDept
101	Arun	DBMS,OS	Ravi,Kumar	CSE,CSE
102	Bala	DBMS	Ravi	CSE

Step 1: 1NF (remove multi-valued attributes)

```
CREATE TABLE student_course (
   regno INTEGER, sname TEXT,
   courseid TEXT, coursename TEXT,
```

faculty TEXT, facultydept TEXT,

PRIMARY KEY (regno, courseid)

);

INSERT INTO student_course VALUES (101,'Arun','C1','DBMS','Ravi','CSE');

INSERT INTO student_course VALUES (101,'Arun','C2','OS','Kumar','CSE');

INSERT INTO student_course VALUES (102,'Bala','C1','DBMS','Ravi','CSE');

Step2:2NF(removepartialdependency)

sname depends only on regno, and coursename and faculty depend only on courseid, so we split the table.

CREATE TABLE student (regno INTEGER PRIMARY KEY, sname TEXT);

CREATE TABLE course (courseid TEXT PRIMARY KEY, coursename TEXT,faculty TEXT,facultydept TEXT);

CREATE TABLE enrolls (regno INTEGER, courseid TEXT, PRIMARY KEY (regno, courseid));

Step3:3NF(removetransitivedependency)

courseid → faculty → facultydept, so facultydept depends on faculty and not directly on the key

CREATE TABLE faculty (faculty TEXT PRIMARY KEY, facultydept TEXT);

CREATE TABLE course (courseid TEXT PRIMARY KEY, coursename TEXT, faculty TEXT);.

Step4:BCNF

Every determinant is now a candidate key:

- regno → sname
- courseid → coursename, faculty
- faculty → facultydept
- (regno, courseid) is the key of enrolls

So all tables satisfy BCNF.

Final Tables

STUDENT (regno PK, sname)

FACULTY (faculty PK, facultydept)

COURSE (courseid PK, coursename, faculty FK)

ENROLLS (regno FK, courseid FK) PK(regno, courseid)

RESULT:

Thus, the table was normalized to 1NF, 2NF, 3NF and BCNF successfully, removing redundancy and anomalies.

KHAN ACEDEMY BOOK LIST DATABASE

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating. Now, add three of your favorite books into the table.

Program:

```
CREATE TABLE books(id INTEGER PRIMARY KEY,  name TEXT, rating INTEGER);

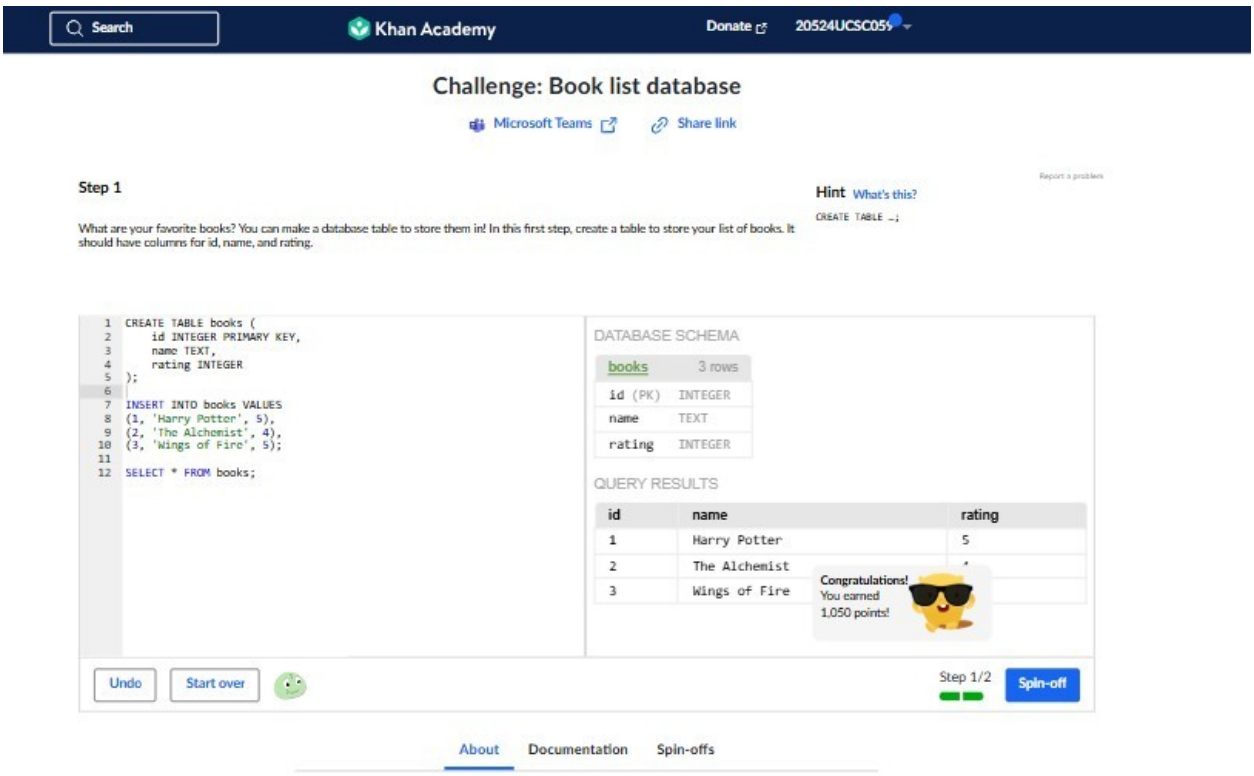
CREATE TABLE books(id INTEGER PRIMARY KEY,  name TEXT, rating INTEGER);

INSERT INTO books VALUES(1,'English',9/10);

INSERT INTO books VALUES(2,'Maths',10/10);

INSERT INTO books VALUES(3,'Tamil',9/10);

SELECT*FROM books;
```



BOX OFFICE HITS DATABASE

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies. Now, add a second query after the first, that retrieves only the movies that were released in the year 2000 or later, not before. Sort the results so that the earlier movies are listed first. You should have 2 SELECT statements after this step.

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year
INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009);

INSERT INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);

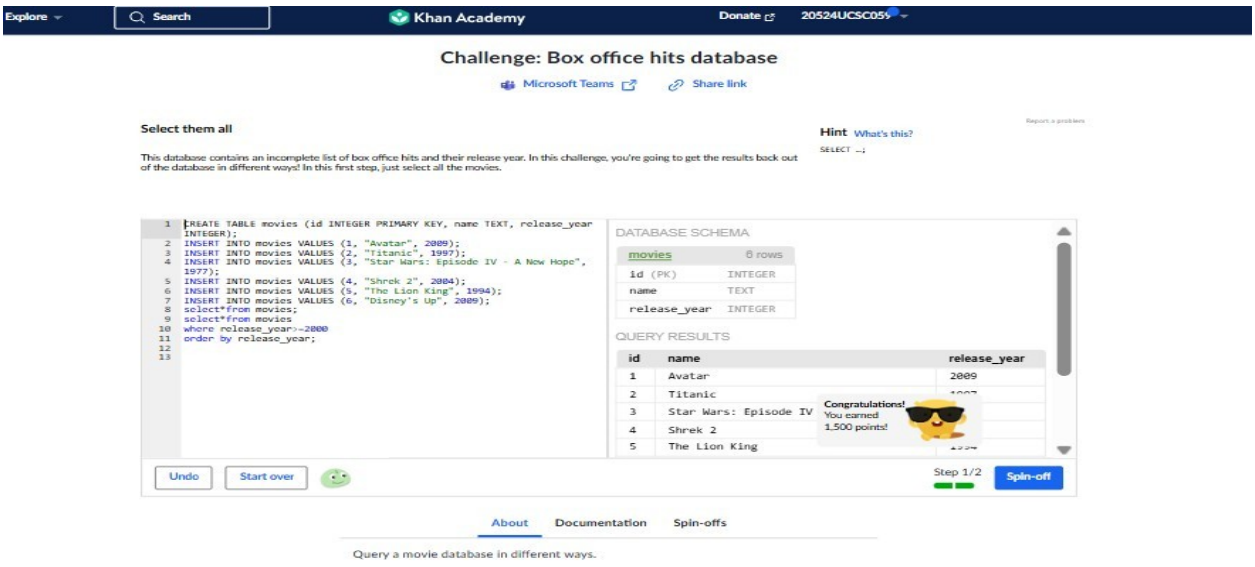
INSERT INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994);

INSERT INTO movies VALUES (6, "Disney's Up", 2009);

SELECT*FROM movies;

SELECT*FROM movies where release_year > 2000 order by release_year;
```



TODO LISTDATABASE STATS

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take. Select the SUM of minutes it will take to do all of the items on your TODO list. You should only have one SELECT statement.

Program:

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

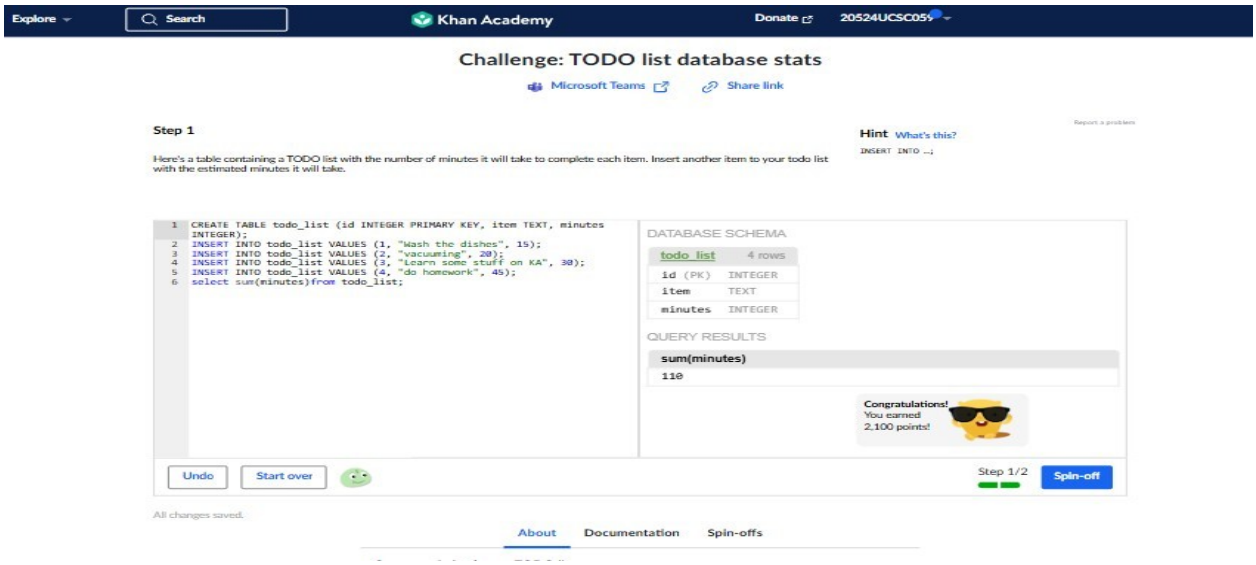
INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "Dry your clothes", 40);

SELECT SUM(minutes) FROM todo_list;
```



PROJECT : DESIGN A STORE DATABASE

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

Program:

```
CREATE TABLE sneakers (     id
INTEGER     PRIMARY     KEY,
brand  TEXT,        model  TEXT,
size  INTEGER,      color  TEXT,
price REAL
);
INSERT INTO sneakers VALUES (1, "Nike", "Air Max", 9, "Black", 120);
INSERT INTO sneakers VALUES (2, "Adidas", "Ultraboost", 10, "White", 180);
INSERT INTO sneakers VALUES (3, "Puma", "RS-X", 8, "Blue", 110);
INSERT INTO sneakers VALUES (4, "New Balance", "574", 9, "Gray", 95);
INSERT INTO sneakers VALUES (5, "Converse", "Chuck Taylor", 10, "Red", 60);
INSERT INTO sneakers VALUES (6, "Vans", "Old Skool", 8, "Black", 70);
INSERT INTO sneakers VALUES (7, "Nike", "Jordan 1", 11, "Red", 200);
INSERT INTO sneakers VALUES (8, "Adidas", "NMD", 9, "Black", 140);
INSERT INTO sneakers VALUES (9, "Puma", "Suede Classic", 10, "Green", 75);
INSERT INTO sneakers VALUES (10, "New Balance", "990", 11, "Gray", 185);
INSERT INTO sneakers VALUES (11, "Converse", "Run Star Hike", 8, "White", 115);
INSERT INTO sneakers VALUES (12, "Vans", "Sk8-Hi", 9, "Blue", 80);
INSERT INTO sneakers VALUES (13, "Nike", "Air Force 1", 10, "White", 130);
INSERT INTO sneakers VALUES (14, "Adidas", "Superstar", 8, "Black", 100);
INSERT INTO sneakers VALUES (15, "Puma", "Future Rider", 9, "Yellow", 90);
```

```
SELECT *
FROM sneakers
ORDER BY price DESC;
SELECT
    AVG(price) AS average_price,
    MAX(price) AS highest_price,
    MIN(price) AS lowest_price
FROM sneakers;
```

Output

DATABASE SCHEMA

sneakers 15 rows

id (PK) INTEGER
brand TEXT
model TEXT
size INTEGER
color TEXT
price REAL

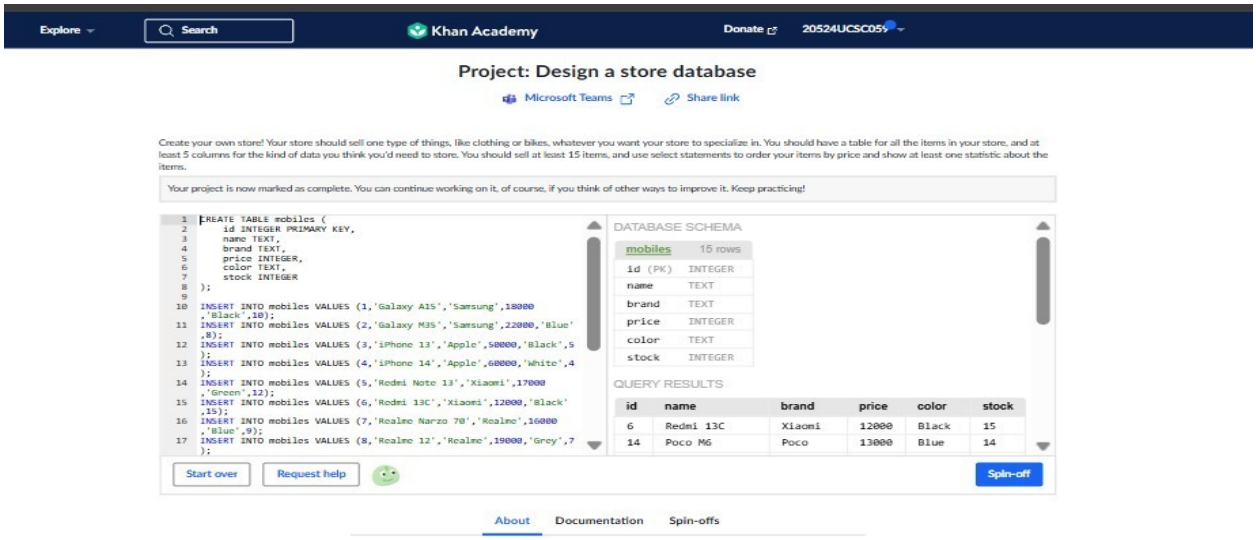
QUERY RESULTS

id	brand	model	size	color	price
7	Nike	Jordan 1	11	Red	200
10	New Balance	990	11	Gray	185

| id | brand | model | size | color | price |

2	Adidas	Ultraboost	10	White	180
8	Adidas	NMD	9	Black	140
13	Nike	Air Force 1	10	White	130
1	Nike	Air Max	9	Black	120
11	Converse	Run Star Hike	8	White	115
3	Puma	RS-X	8	Blue	110
14	Adidas	Superstar	8	Black	100
4	New Balance	574	9	Gray	95
15	Puma	Future Rider	9	Yellow	90
12	Vans	Sk8-Hi	9	Blue	80
9	Puma	Suede Classic	10	Green	75
6	Vans	Old Skool	8	Black	70
5	Converse	Chuck Taylor	10	Red	60

average_price	highest_price	lowest_price
116.66666666666667	200	60


Explore
Search
Khan Academy
Donate
20524UCSC059
Project: Design a store database
Microsoft Teams
Share link
Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.
Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!
CREATE TABLE mobiles (
id INTEGER PRIMARY KEY,
name TEXT,
brand TEXT,
price INTEGER,
color TEXT,
stock INTEGER
);
INSERT INTO mobiles VALUES (1,'Galaxy A15','Samsung',18000,'Black',10);
INSERT INTO mobiles VALUES (2,'Galaxy M35','Samsung',22000,'Blue',8);
INSERT INTO mobiles VALUES (3,'iPhone 13','Apple',50000,'Black',5);
INSERT INTO mobiles VALUES (4,'iPhone 14','Apple',60000,'White',4);
INSERT INTO mobiles VALUES (5,'Redmi Note 13','Xiaomi',17000,'Green',12);
INSERT INTO mobiles VALUES (6,'Redmi 13C','Xiaomi',12000,'Black',15);
INSERT INTO mobiles VALUES (7,'Realme Narzo 70','Realme',16000,'Blue',9);
INSERT INTO mobiles VALUES (8,'Realme 12','Realme',19000,'Grey',7);
DATABASE SCHEMA
mobiles 15 rows
id (PK) INTEGER
name TEXT
brand TEXT
price INTEGER
color TEXT
stock INTEGER
QUERY RESULTS
id name brand price color stock
6 Redmi 13C Xiaomi 12000 Black 15
14 Poco M6 Poco 13000 Blue 14
Start over
Request help
Spin-off
About
Documentation
Spin-offs

KARAOKE SONG SELECTOR

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles. Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECT that uses OR to show the titles of the songs that have an 'epic' mood or a release date after 1990. People get picky at the end of the night. Add another SELECT that uses AND to show the titles of songs that are 'epic', and released after 1990, and less than 4 minutes long.

Program:

```
CREATE TABLE songs (     id
INTEGER PRIMARY KEY,
    title TEXT,      artist
TEXT,     mood TEXT,
duration      INTEGER,
released INTEGER);
INSERT INTO songs (title, artist, mood, duration, released)
   VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
   VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
   VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
   VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
   VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
   VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
```

INSERT INTO songs (title, artist, mood, duration, released)

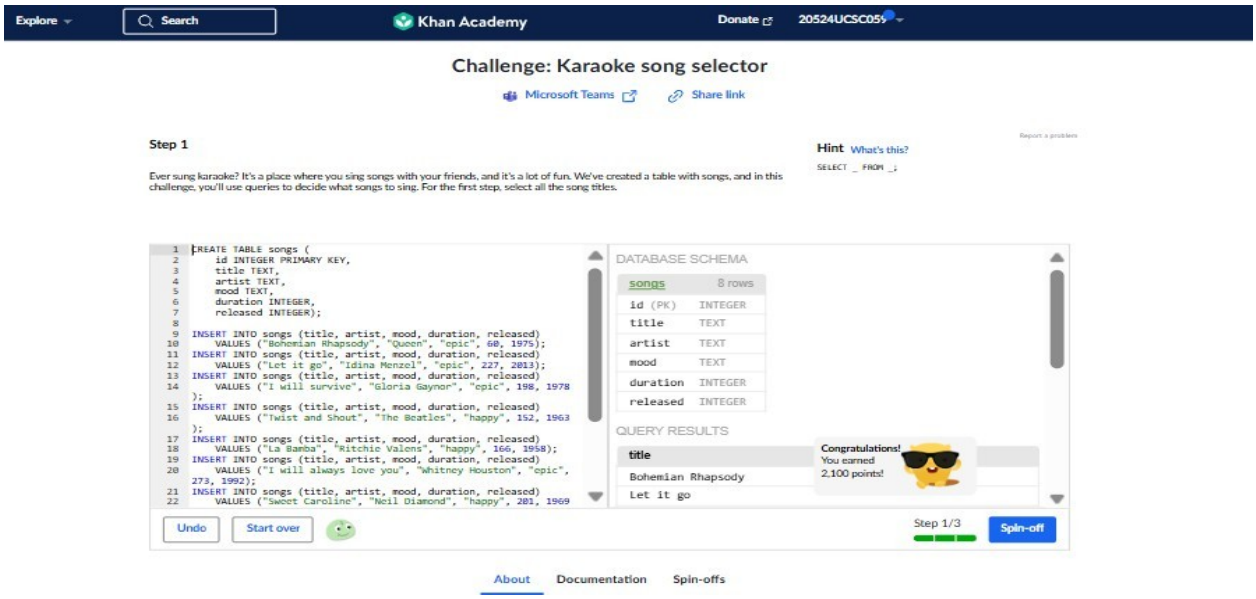
VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);

INSERT INTO songs (title, artist, mood, duration, released)

VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011); select title from songs;

select title from songs where mood = 'epic' or released > 1990;

select title from songs where mood = 'epic' and released > 1990 and duration < 240;



PLAYLIST MAKER

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the `title` of all the songs by the `artist` named 'Queen'. Now you'll make a 'Pop' playlist. In preparation, select the `name` of all of the artists from the 'Pop' genre. To finish creating the 'Pop' playlist, add another query that will select the `title` of all the songs from the 'Pop' artists. It should use `IN` on a nested subquery that's based on your previous query.

Program:

```
CREATE TABLE artists (     id INTEGER PRIMARY

KEY AUTOINCREMENT,

       name  TEXT,

country     TEXT,

genre TEXT);

INSERT INTO artists (name, country, genre)

   VALUES ("Taylor Swift", "US", "Pop");

INSERT INTO artists (name, country, genre)

VALUES ("Led Zeppelin", "US", "Hard rock");

INSERT INTO artists (name, country, genre)

   VALUES ("ABBA", "Sweden", "Disco");

INSERT INTO artists (name, country, genre)

   VALUES ("Queen", "UK", "Rock");

INSERT INTO artists (name, country, genre)
```

```
    VALUES ("Celine Dion", "Canada", "Pop");
INSERT INTO artists (name, country, genre)
VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Guns N' Roses", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
VALUES ("Gloria Estefan", "US", "Pop");
INSERT INTO artists (name, country, genre)
        VALUES  ("Bob  Marley",  "Jamaica",  "Reggae");
CREATE TABLE songs (       id INTEGER PRIMARY
KEY AUTOINCREMENT,
    artist TEXT,
title TEXT);
INSERT INTO songs (artist, title)
```

```
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
VALUES ("Rihanna", "Stay"); INSERT
INTO songs (artist, title)

    VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
```

```
INSERT INTO songs (artist, title)

VALUES ("Guns N' Roses", "Don't cry");

select title from songs where artist = 'Queen';

select name from artists where genre = 'Pop';

SELECT title

FROM songs

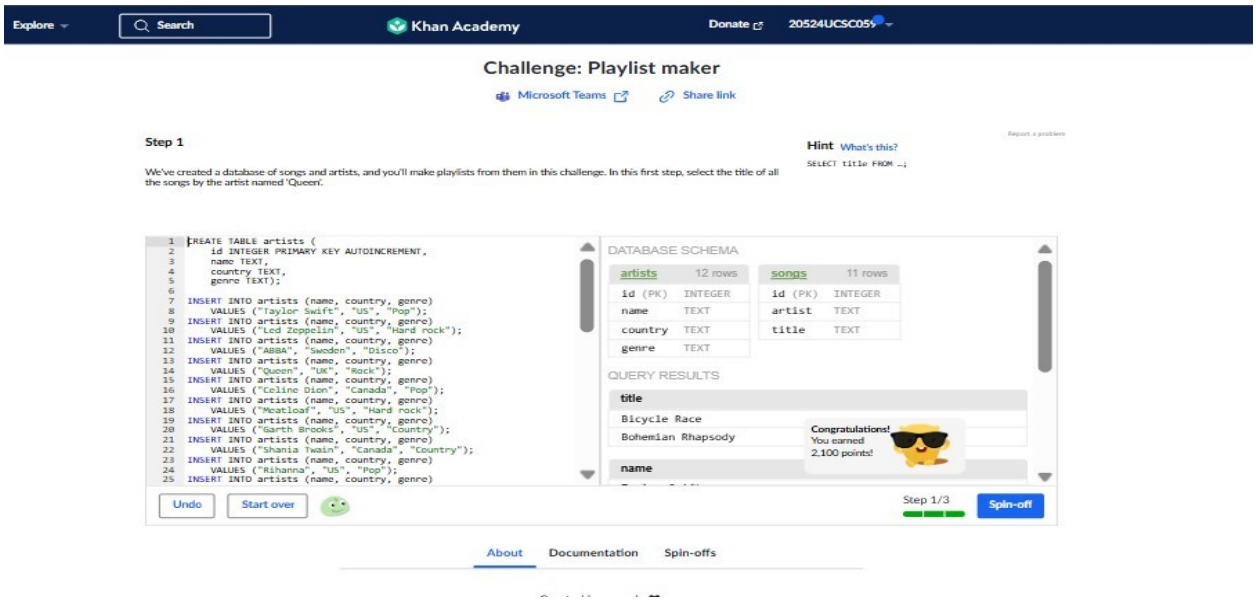
WHERE artist IN (

    SELECT name

    FROM artists

    WHERE genre = 'Pop'

);
```



THE WORDIEST AUTHOR

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using `GROUP BY` and `HAVING`. Your results table should include the 'author' and their total word count as a 'total_words' column. Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

Program:

```
CREATE TABLE books (     id INTEGER PRIMARY

KEY AUTOINCREMENT,

   author TEXT,

title TEXT,     words

INTEGER);

INSERT INTO books (author, title, words)

   VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944); INSERT

INTO books (author, title, words)

   VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);

INSERT INTO books (author, title, words)

   VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);

INSERT INTO books (author, title, words)

   VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);

INSERT INTO books (author, title, words)

   VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);

INSERT INTO books (author, title, words)
```

```
    VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
SELECT author,
      SUM(words) AS total_words
FROM books
```

```
GROUP BY author

HAVING SUM(words) > 1000000;

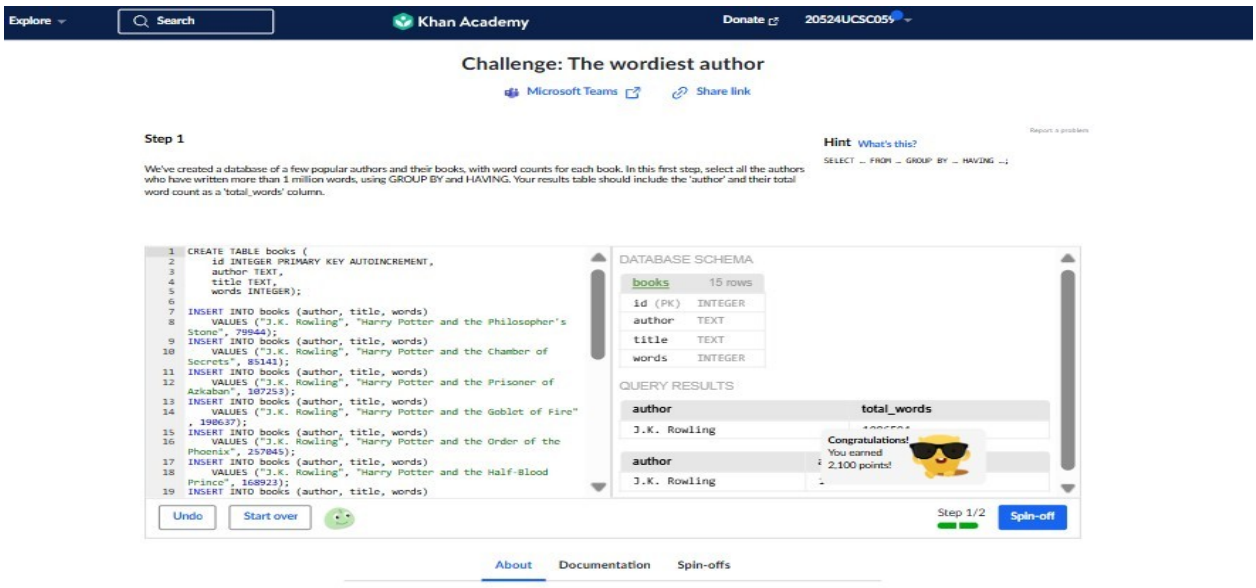
SELECT author,

    AVG(words) AS avg_words

FROM books

GROUP BY author

HAVING AVG(words) > 150000;
```



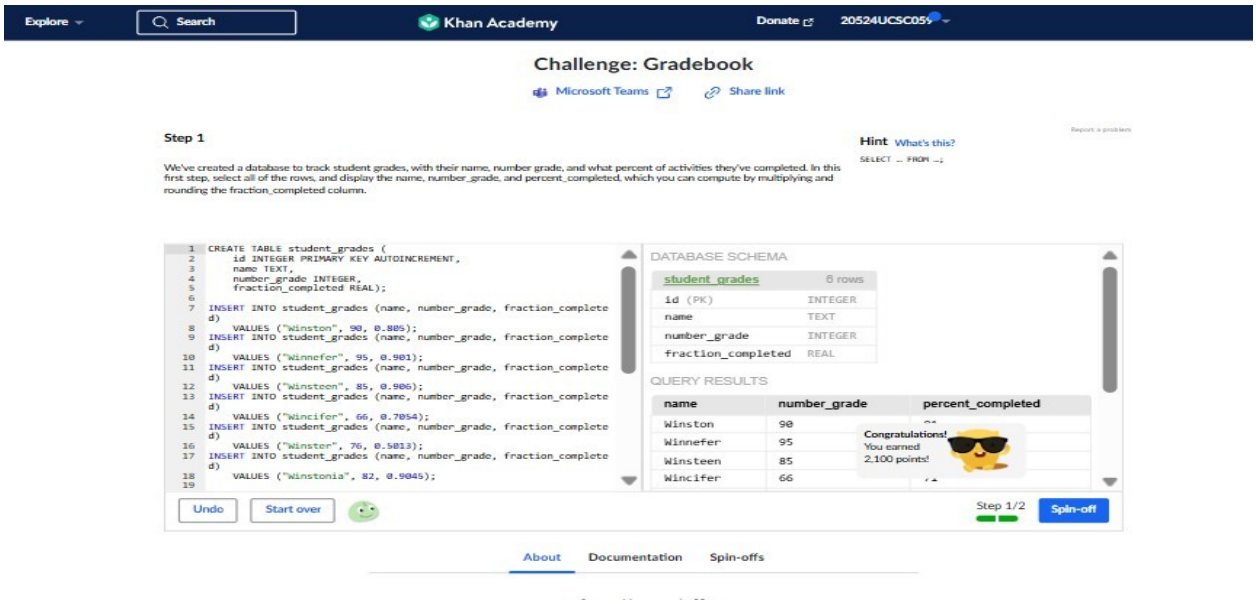
GRADEBOOK

We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display the name, number_grade, and percent_completed, which you cancompute by multiplying and rounding the fraction_completed column. Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.

Program:

```
CREATE TABLE student_grades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,     number_grade INTEGER,
fraction_completed REAL);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winstonia", 82, 0.9045); SELECT
name,
      number_grade,
```

```
    ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;
SELECT COUNT(*),
  CASE
    WHEN number_grade > 90 THEN 'A'
    WHEN number_grade > 80 THEN 'B'
    WHEN number_grade > 70 THEN 'C'
    ELSE 'F'
  END AS "letter_grade"
FROM student_grades
GROUP BY
  CASE
    WHEN number_grade > 90 THEN 'A'
    WHEN number_grade > 80 THEN 'B'
    WHEN number_grade > 70 THEN 'C'
    ELSE 'F'
  END;
```



DATA DIG

What are average, max, and min values in the data?

What about those numbers per category in the data (using HAVING)? What ways are there to group the data values that don't exist yet (using CASE)? What interesting ways are there to filter the data (using AND/OR)?

Program:

```
CREATE TABLE pokemon
(     id INTEGER PRIMARY
KEY,     name TEXT,     type
TEXT,     hp INTEGER,     attack
INTEGER
);
INSERT INTO pokemon VALUES (1, "Pikachu", "Electric", 35, 55);
INSERT INTO pokemon VALUES (2, "Charizard", "Fire", 78, 84);
INSERT INTO pokemon VALUES (3, "Blastoise", "Water", 79, 83);
INSERT INTO pokemon VALUES (4, "Venusaur", "Grass", 80, 82);
INSERT INTO pokemon VALUES (5, "Jolteon", "Electric", 65, 65);
INSERT INTO pokemon VALUES (6, "Arcanine", "Fire", 90, 110);
INSERT INTO pokemon VALUES (7, "Gyarados", "Water", 95, 125);
INSERT INTO pokemon VALUES (8, "Sceptile", "Grass", 70, 85);
SELECT
   AVG(attack) AS avg_attack,
   MAX(attack) AS max_attack,
   MIN(attack) AS min_attack
FROM pokemon; SELECT
   type,
   AVG(attack) AS avg_attack
FROM pokemon
```

```
GROUP BY type; SELECT
    type,
    AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type
HAVING AVG(attack) > 80; SELECT
    name,
attack,
    CASE
        WHEN attack >= 100 THEN "Very Strong"
        WHEN attack >= 80 THEN "Strong"
        ELSE "Average"
    END AS strength
FROM pokemon;
SELECT *
FROM pokemon
WHERE (type = "Fire" OR type = "Water")
AND attack > 80;
```

Output

DATABASE SCHEMA

pokemon 8 rows
id (PK) INTEGER
name TEXT
type TEXT

hpINTEGER
attackINTEGER

QUERY RESULTS

avg_attack	max_attack	min_attack
86.125	125	55

type	avg_attack
Electric	60
Fire	97
Grass	83.5

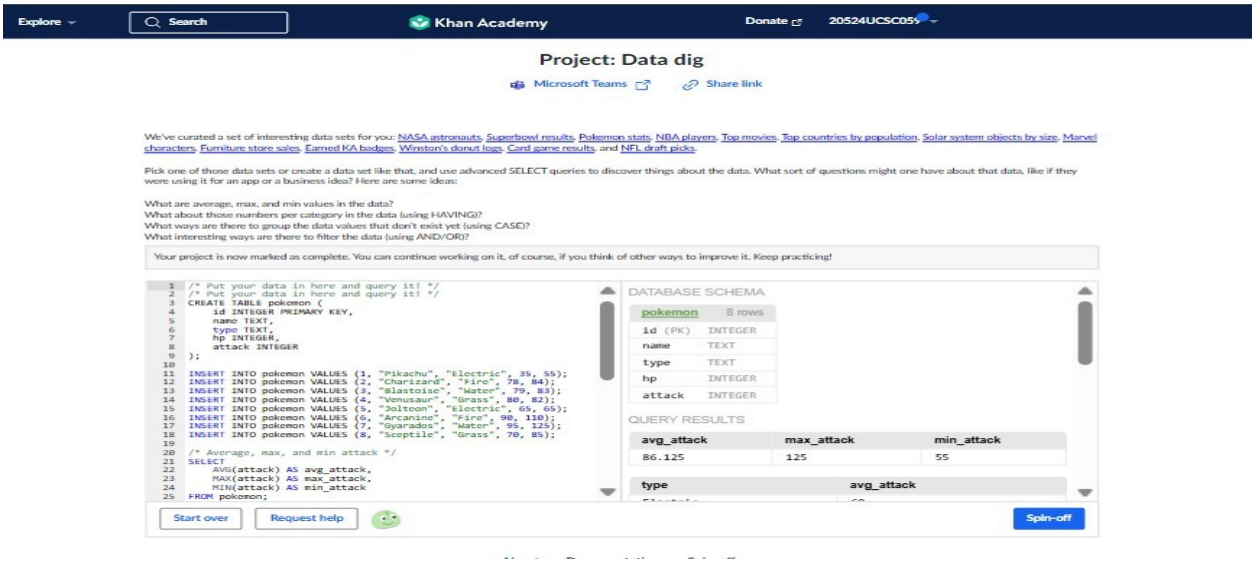
type	avg_attack
Water	104

type	avg_attack
Fire	97
Grass	83.5
Water	104

name	attack	strength
Pikachu	55	Average

Charizard	84	Strong
Blastoise	83	Strong
Venusaur	82	Strong
Jolteon	65	Average
Arcanine	110	Very Strong
Gyarados	125	Very Strong
Sceptile	85	Strong

id	name	type	hp	attack
2	Charizard	Fire	78	84
3	Blastoise	Water	79	83
6	Arcanine	Fire	90	110
7	Gyarados	Water	95	125


Explore
Search
Khan Academy
Donate
20524UCSC059
Project: Data dig
Microsoft Teams
Share link
We've curated a set of interesting data sets for you: NASA astronauts, Superbowl results, Pokemon stats, NBA players, Top movies, Top countries by population, Solar system objects by size, Marvel characters, Furniture store sales, Earned KA badges, Winston's donut logs, Card game results, and NFL draft picks.
Pick one of those data sets or create a data set like that, and use advanced SELECT queries to discover things about the data. What sort of questions might one have about that data, like if they were using it for an app or a business idea? Here are some ideas:
What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?
Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!
/* Put your data in here and query it! */
/* Put your data in here and query it! */
CREATE TABLE pokemon (
id INTEGER PRIMARY KEY,
name TEXT,
type TEXT,
hp INTEGER,
attack INTEGER
);
INSERT INTO pokemon VALUES (1, "Pikachu", "Electric", 35, 55);
INSERT INTO pokemon VALUES (2, "Charizard", "Fire", 78, 84);
INSERT INTO pokemon VALUES (3, "Blastoise", "Water", 79, 83);
INSERT INTO pokemon VALUES (4, "Venusaur", "Grass", 80, 82);
INSERT INTO pokemon VALUES (5, "Jolteon", "Electric", 65, 65);
INSERT INTO pokemon VALUES (6, "Arcanine", "Fire", 90, 110);
INSERT INTO pokemon VALUES (7, "Gyarados", "Water", 95, 125);
INSERT INTO pokemon VALUES (8, "Sceptile", "Grass", 70, 85);
/* Average, max, and min attack */
SELECT
AVG(attack) AS avg_attack,
MAX(attack) AS max_attack,
MIN(attack) AS min_attack
FROM pokemon;
DATABASE SCHEMA
pokemon 8 rows
id (PK) INTEGER
name TEXT
type TEXT
hp INTEGER
attack INTEGER
QUERY RESULTS
avg_attack max_attack min_attack
86.125 125 55
type avg_attack
Start over
Request help
Spin-off

BOBBY'S HOBBIES

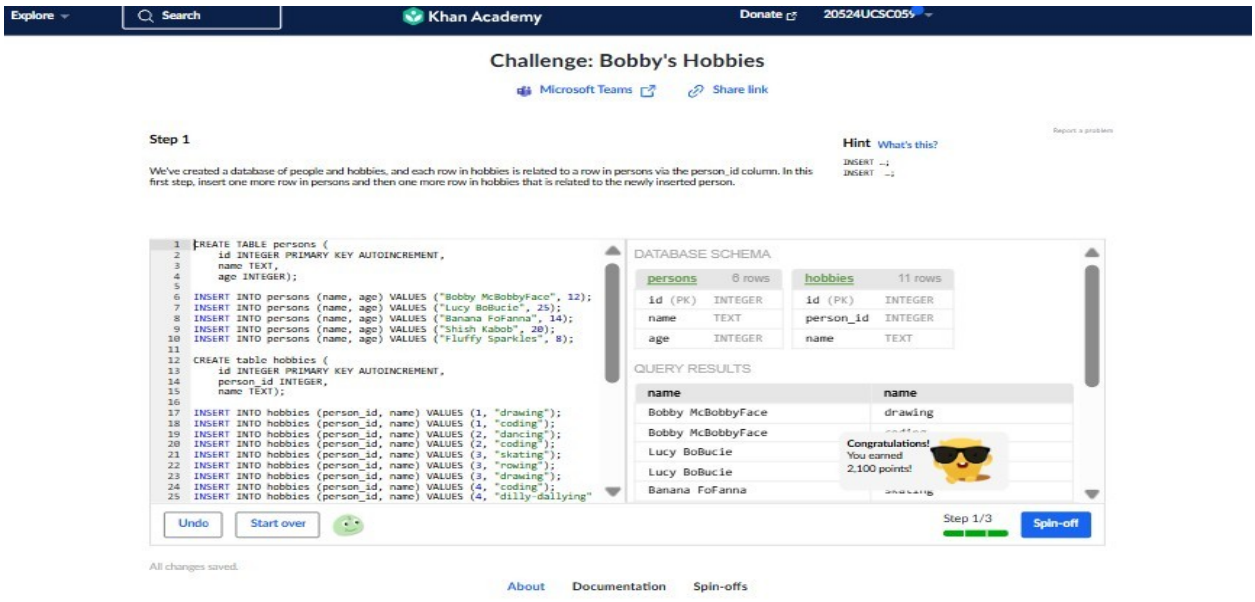
We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person. Now, select the 2 tables with a join so that you can see each person's name next to their hobby. Now, add an additional query that shows only the name and hobbies of 'Bobby McBobbyFace', using JOIN combined with WHERE.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,     age INTEGER);

INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8); CREATE
table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
person_id INTEGER,     name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
```

```
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age) VALUES
("Sam Star", 18);
INSERT INTO hobbies (person_id, name)
VALUES (6, "gaming");
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT persons.name,
    hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id
WHERE persons.name = "Bobby McBobbyFace";
```



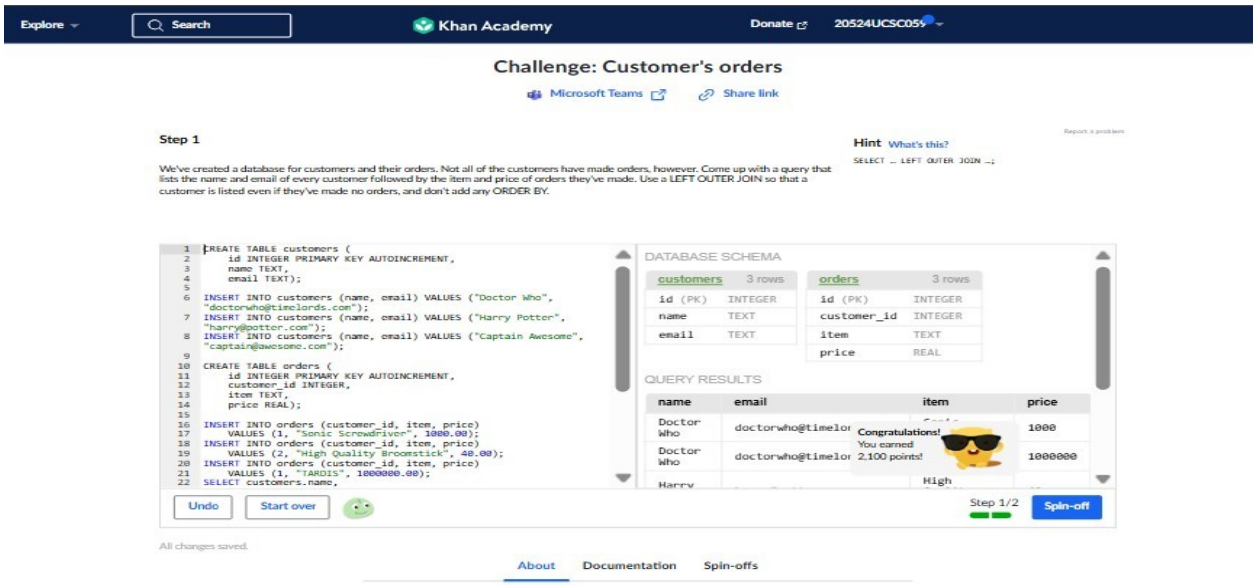
CUSTOMER'S ORDERS

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY. Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.

Program:

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT,     email TEXT);
INSERT INTO customers (name, email) VALUES ("Doctor Who",
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com"); CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
customer_id INTEGER,     item TEXT,     price REAL);
INSERT INTO orders (customer_id, item, price)
   VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
   VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
   VALUES (1, "TARDIS", 1000000.00); SELECT
customers.name,
      customers.email,
orders.item,       orders.price
FROM customers
LEFT OUTER JOIN orders
```

```
ON customers.id = orders.customer_id; SELECT
customers.name,
    customers.email,
    SUM(orders.price) AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_spent DESC;
```



SEQUELS IN SQL

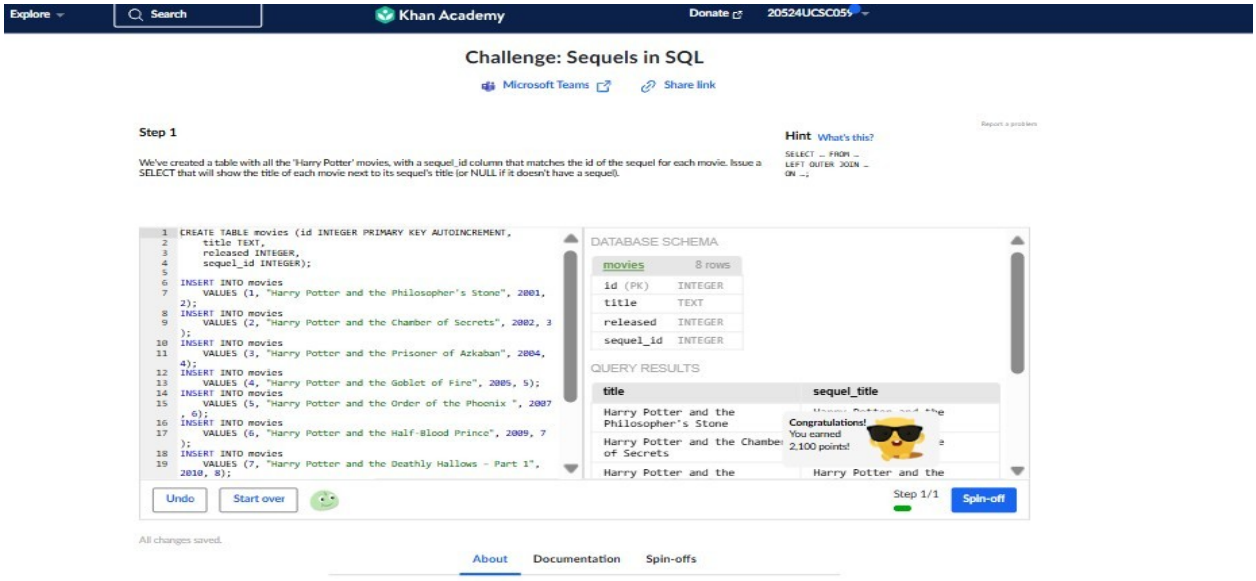
We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
title TEXT,     released INTEGER,     sequel_id INTEGER);
INSERT INTO movies
   VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
   VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
   VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
   VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
   VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
   VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
   VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
   VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
SELECT m1.title,
      m2.title AS sequel_title
FROM movies AS m1
```

LEFT OUTER JOIN movies AS m2

ON m1.sequel_id = m2.id;



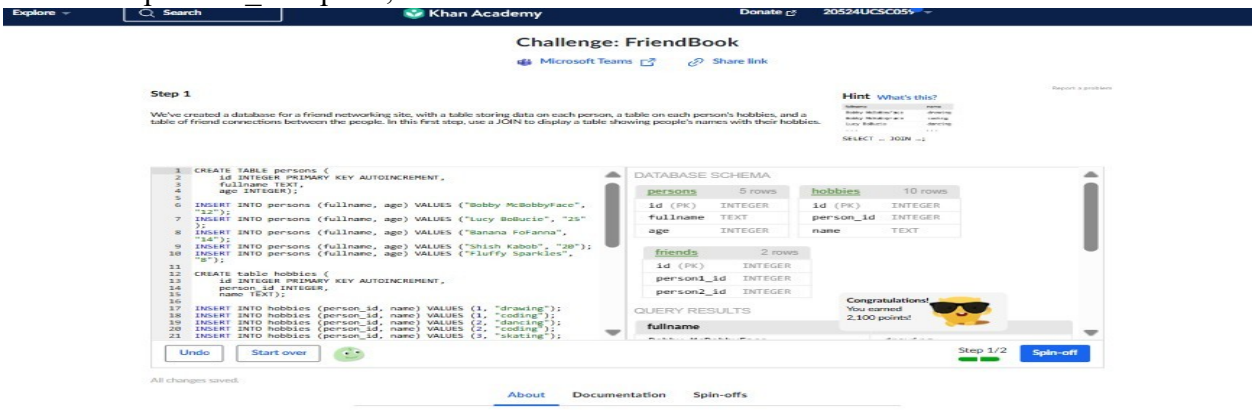
FRIENDBOOK

We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies. Now, use another SELECT with a JOIN to show the names of each pair of friends, based on the data in the friends table.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
fullname TEXT,     age INTEGER);
INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8"); CREATE
table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
person_id INTEGER,     name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
```

```
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing"); CREATE
table friends (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
person1_id INTEGER,     person2_id INTEGER);
INSERT INTO friends (person1_id, person2_id)
    VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id)
    VALUES (2, 3); SELECT
persons.fullname,
hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT  p1.fullname  AS  person1,
p2.fullname AS person2
FROM friends
JOIN persons AS p1
ON friends.person1_id = p1.id
JOIN persons AS p2
ON friends.person2_id = p2.id;
```



FAMOUS PEOPLE

Movie stars: What movies are they in? Are they married to each other?

Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?

Program:

```
CREATE TABLE authors (      id
INTEGER     PRIMARY     KEY,
name TEXT,     country TEXT
);
INSERT INTO authors VALUES (1, "J.K. Rowling", "United Kingdom");
INSERT INTO authors VALUES (2, "George Orwell", "United Kingdom");
INSERT INTO authors VALUES (3, "Mark Twain", "United States");
INSERT INTO authors VALUES (4, "Jane Austen", "United Kingdom");
CREATE TABLE books (     id INTEGER PRIMARY KEY,
   title TEXT,
author_id INTEGER
);
INSERT INTO books VALUES (1, "Harry Potter and the Sorcerer's Stone", 1);
INSERT INTO books VALUES (2, "Harry Potter and the Chamber of Secrets", 1);
INSERT INTO books VALUES (3, "1984", 2);
INSERT INTO books VALUES (4, "Animal Farm", 2);
INSERT INTO books VALUES (5, "The Adventures of Tom Sawyer", 3);
INSERT INTO books VALUES (6, "Pride and Prejudice", 4);
CREATE TABLE author_friends (     id INTEGER
PRIMARY KEY,     author1_id INTEGER,     author2_id
INTEGER
);
```

```
INSERT INTO author_friends VALUES (1, 1, 2);
INSERT INTO author_friends VALUES (2, 3, 4);
SELECT authors.name AS author,        books.title
AS book
FROM authors
JOIN books
ON authors.id = books.author_id; SELECT
name,
      country
FROM     authors;     SELECT
a1.name        AS        author1,
a2.name  AS  author2  FROM
author_friends
JOIN authors AS a1
ON author_friends.author1_id = a1.id
JOIN authors AS a2
ON author_friends.author2_id = a2.id;
```

Output:

DATABASE SCHEMA

authors	4 rows
id (PK)	INTEGER
name	TEXT
country	TEXT

books	6 rows
id (PK)	INTEGER
title	TEXT
author_id	INTEGER

author_friends	2 rows
id (PK)	INTEGER
author1_id	INTEGER
author2_id	INTEGER

QUERY RESULTS

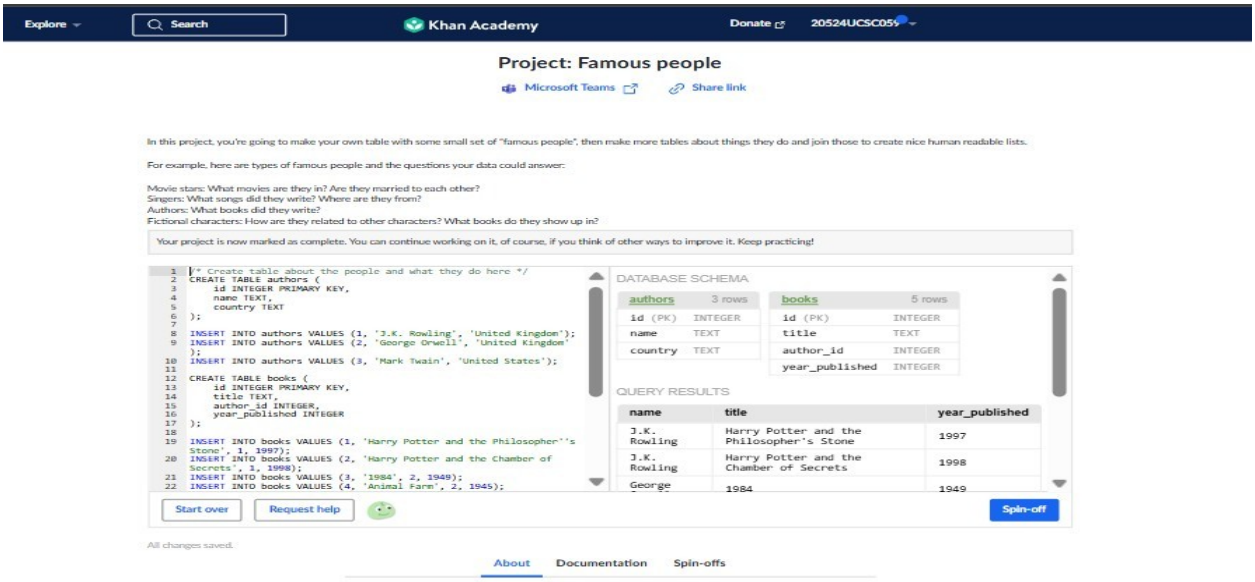
author	book
J.K. Rowling	Harry Potter and the Sorcerer's Stone
J.K. Rowling	Harry Potter and the Chamber of Secrets

George Orwell	1984
George Orwell	Animal Farm
Mark Twain	The Adventures of Tom Sawyer

author	book
Jane Austen	Pride and Prejudice

name	country
J.K. Rowling	United Kingdom
George Orwell	United Kingdom
Mark Twain	United States
Jane Austen	United Kingdom

author1	author2
J.K. Rowling	George Orwell
Mark Twain	Jane Austen


Explore
Search
Khan Academy
Donate
20524UCSC059
Project: Famous people
Microsoft Teams
Share link
In this project, you're going to make your own table with some small set of "famous people", then make more tables about things they do and join those to create nice human readable lists.
For example, here are types of famous people and the questions your data could answer:
Movie stars: What movies are they in? Are they married to each other?
Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?
Your project is now marked as complete. You can continue working on it, of course, if you think of other ways to improve it. Keep practicing!
/* Create table about the people and what they do here */
CREATE TABLE authors (
id INTEGER PRIMARY KEY,
name TEXT,
country TEXT
);
INSERT INTO authors VALUES (1, 'J.K. Rowling', 'United Kingdom');
INSERT INTO authors VALUES (2, 'George Orwell', 'United Kingdom'
);
INSERT INTO authors VALUES (3, 'Mark Twain', 'United States');
CREATE TABLE books (
id INTEGER PRIMARY KEY,
title TEXT,
author_id INTEGER,
year_published INTEGER
);
INSERT INTO books VALUES (1, 'Harry Potter and the Philosopher''s Stone', 1, 1997);
INSERT INTO books VALUES (2, 'Harry Potter and the Chamber of Secrets', 1, 1998);
INSERT INTO books VALUES (3, '1984', 2, 1949);
INSERT INTO books VALUES (4, 'Animal Farm', 2, 1945);
DATABASE SCHEMA
authors 3 rows
id (PK) INTEGER
name TEXT
country TEXT
books 5 rows
id (PK) INTEGER
title TEXT
author_id INTEGER
year_published INTEGER
QUERY RESULTS
name
title
year_published
J.K. Rowling
Harry Potter and the Philosopher's Stone
1997
J.K. Rowling
Harry Potter and the Chamber of Secrets
1998
George
1984
1949
Start over
Request help
Spin-off
All changes saved
About
Documentation
Spin-offs

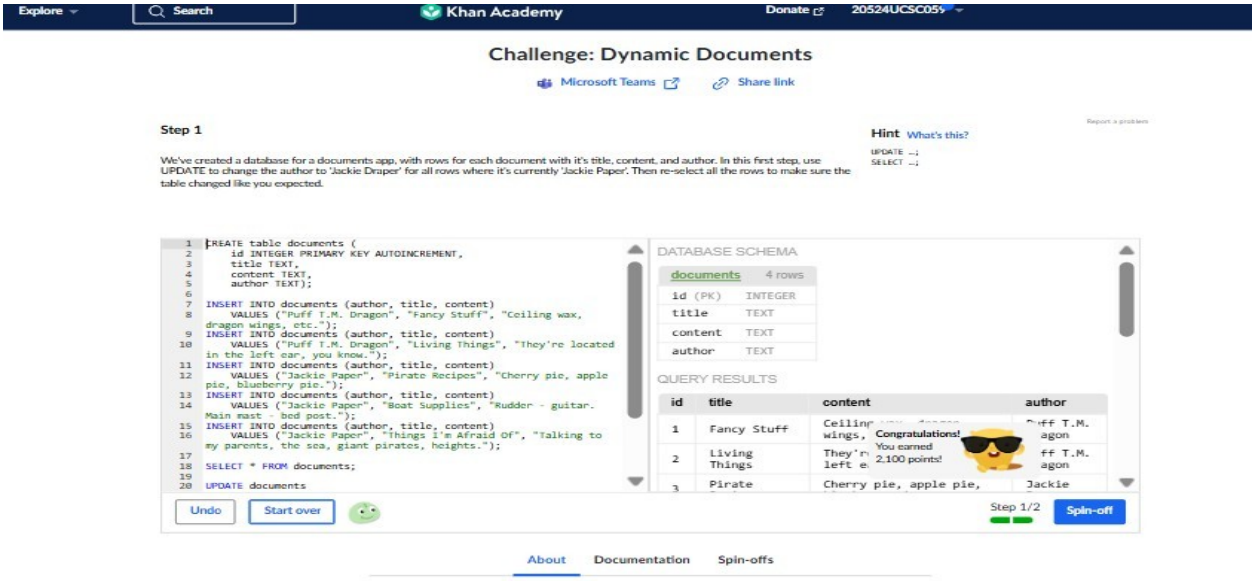
DYNAMIC DOCUMENTS

We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected. Now you'll delete a row, being very careful not to delete all the rows. Only delete the row where the title is 'Things I'm Afraid Of'. Then re-select all the rows to make sure the table changed like you expected.

Program:

```
CREATE table documents (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
title TEXT,     content TEXT,     author TEXT);

INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates, heights.");
SELECT * FROM documents;
UPDATE documents
SET author = "Jackie Draper"
WHERE author = "Jackie Paper";
SELECT * FROM documents;
DELETE FROM documents
WHERE title = "Things I'm Afraid Of";
SELECT * FROM documents;
```


Explore
Search
Khan Academy
Donate
20524UCSC055
Challenge: Dynamic Documents
Microsoft Teams
Share link
Step 1
We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected.
Hint What's this?
UPDATE ...;
SELECT ...;
CREATE table documents (
id INTEGER PRIMARY KEY AUTOINCREMENT,
title TEXT,
content TEXT,
author TEXT);
INSERT INTO documents (author, title, content)
VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");
INSERT INTO documents (author, title, content)
VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant pirates, heights.");
SELECT * FROM documents;
UPDATE documents
DATABASE SCHEMA
documents 4 rows
id (PK) INTEGER
title TEXT
content TEXT
author TEXT
QUERY RESULTS
id
title
content
author
1
Fancy Stuff
2
Living Things
3
Pirate
Cherry pie, apple pie,
Jackie
Congratulations!
You earned 2,100 points!
Undo
Start over
Step 1/2
Spin-off
About
Documentation
Spin-offs

CLOTHING ALTERATIONS

We've created a database of clothes, and decided we need a price column. Use ALTER to add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now. Now assign each item a price, using UPDATE - item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another SELECT of all the rows to check that it worked as expected. Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final SELECT of all the rows to check it worked.

Program:

```
CREATE TABLE clothes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
type TEXT,     design TEXT);
INSERT INTO clothes (type, design)
   VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
   VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
   VALUES ("blazer", "black sequin");
ALTER TABLE clothes
ADD COLUMN price REAL;
SELECT * FROM clothes;
UPDATE clothes
SET price = 10
WHERE id = 1;
UPDATE clothes
SET price = 20
WHERE id = 2;
UPDATE clothes
```

SET price = 30

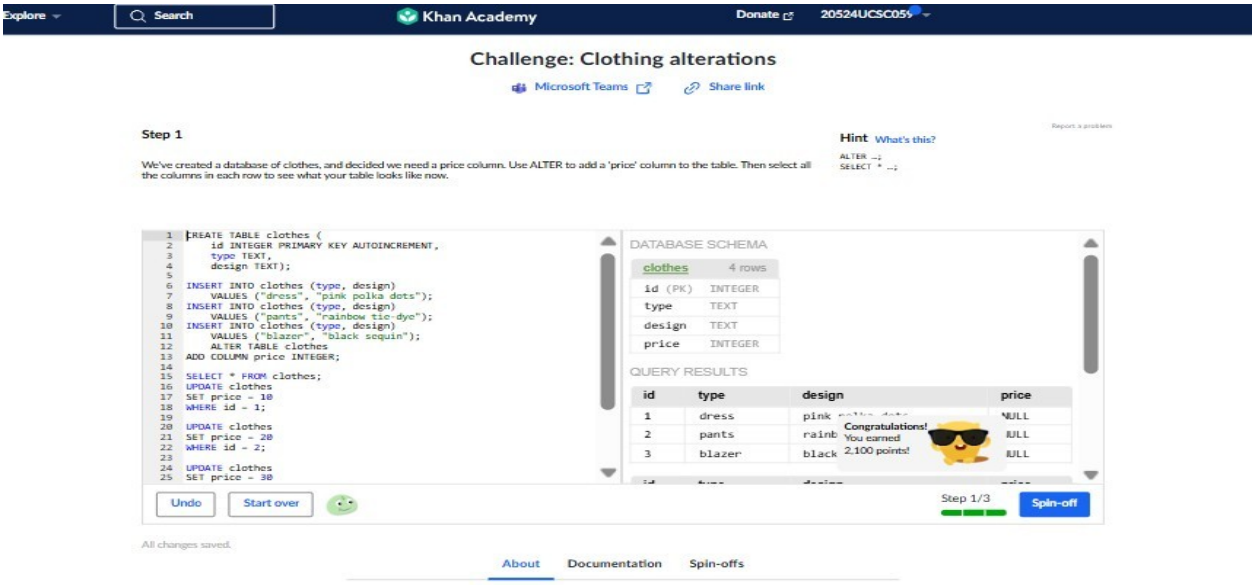
WHERE id = 3;

SELECT * FROM clothes;

INSERT INTO clothes (type, design, price)

VALUES ("shirt", "blue stripes", 15);

SELECT * FROM clothes;



PROJECT: APP IMPERSONATO

CREATE a table to store the data.
INSERT a few example rows in the table.
Use an UPDATE to emulate what happens when you edit data in the app.
Use a DELETE to emulate what happens when you delete data in the app.

Program:

```
CREATE TABLE posts (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   username TEXT,
content TEXT,     likes
INTEGER
);
INSERT INTO posts (username, content, likes)
VALUES ("alex", "Just finished my homework!", 12);
INSERT INTO posts (username, content, likes)
VALUES ("sam", "Playing my favorite game tonight.", 25);
INSERT INTO posts (username, content, likes)
VALUES ("taylor", "Had a great day at the beach.", 18);
SELECT * FROM posts;
UPDATE posts
SET content = "Just finished my homework and studied SQL!"
WHERE id = 1;
SELECT * FROM posts;
DELETE FROM posts
WHERE id = 2;
SELECT * FROM posts;
```

Output:

DATABASE SCHEMA

posts2 rows
id (PK)INTEGER
usernameTEXT
contentTEXT

likesINTEGER

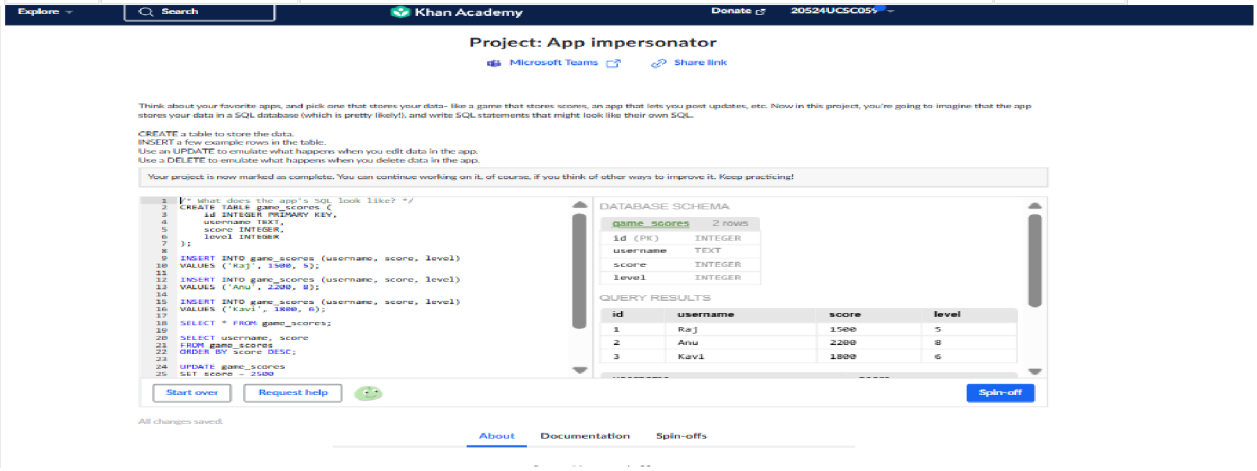
QUERY RESULTS

id	username	content	likes
1	alex	Just finished my homework!	12

id	username	content	likes
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
3	taylor	Had a great day at the beach.	18



HACKER RANK

REVISING THE SELECT QUERY I

Query all columns for all American cities in the **CITY** table with populations larger than 100000.

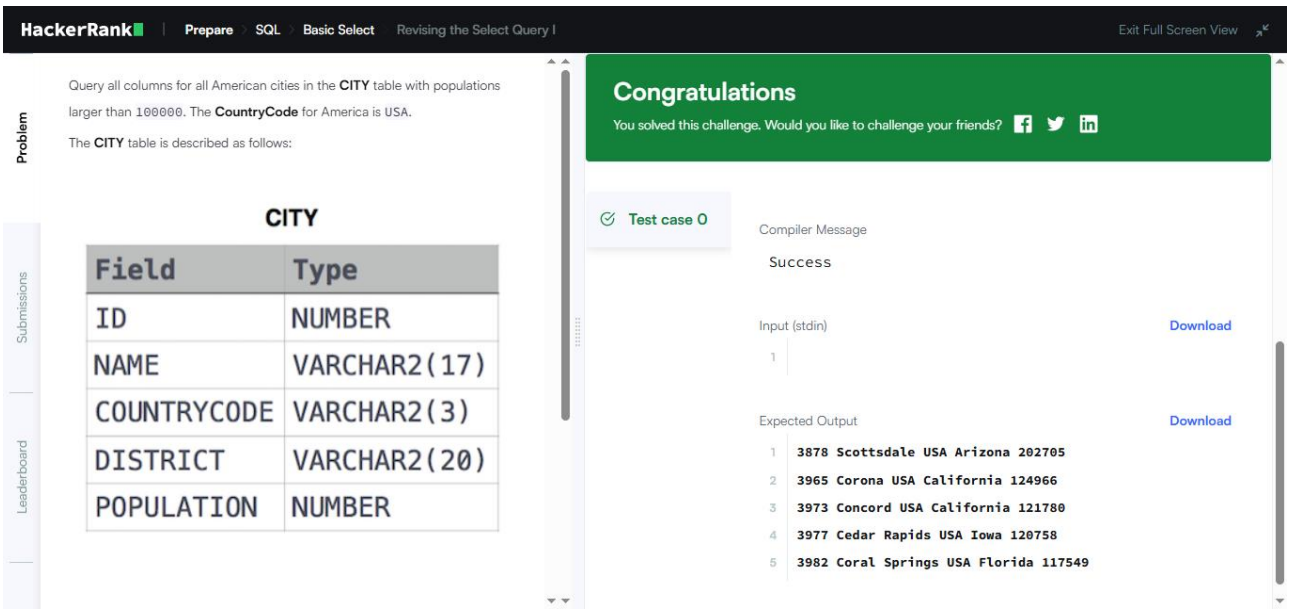
The **CountryCode** for America is USA.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
select * from city where countrycode = 'usa' and population > 100000;
```



REVISING THE SELECT QUERY II

Query the **NAME** field for all American cities in the **CITY** table with populations larger than 120000.

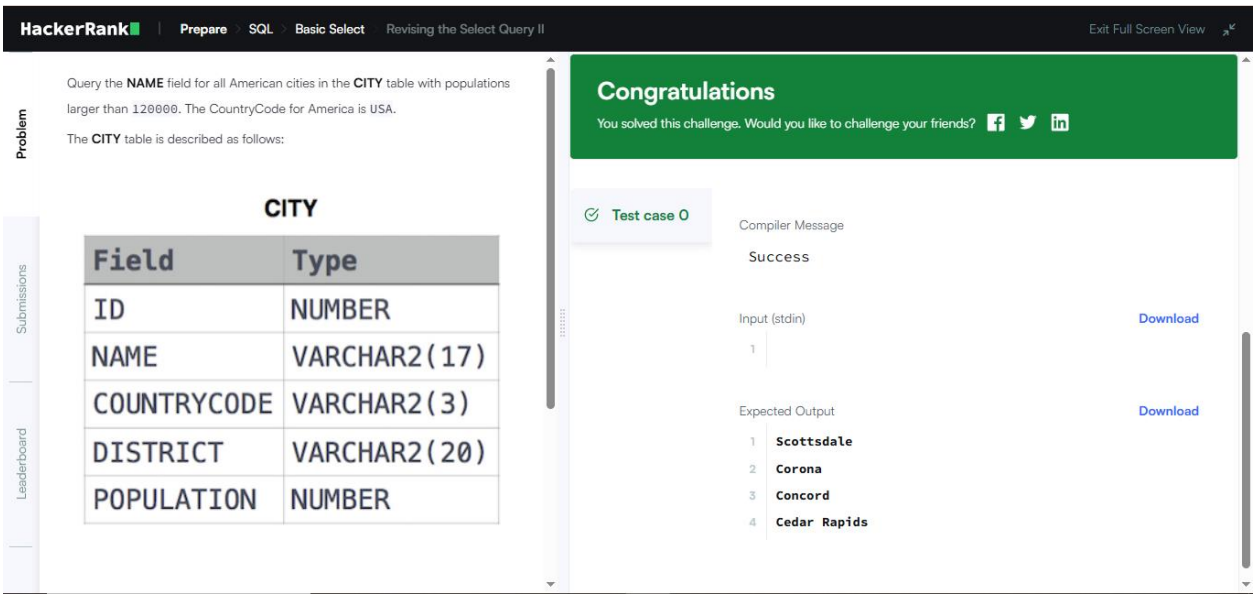
The *CountryCode* for America is USA.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select name from city where countrycode = 'usa' and population > 120000;



SELECT ALL

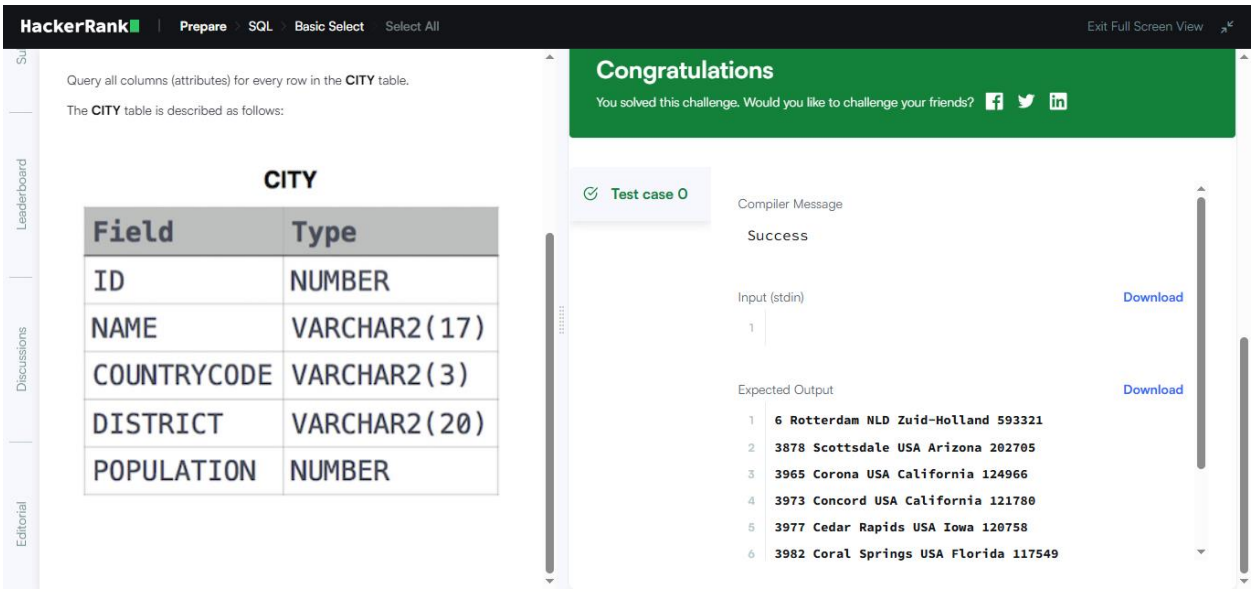
Query all columns (attributes) for every row in the **CITY** table.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city;



SELECT BY ID

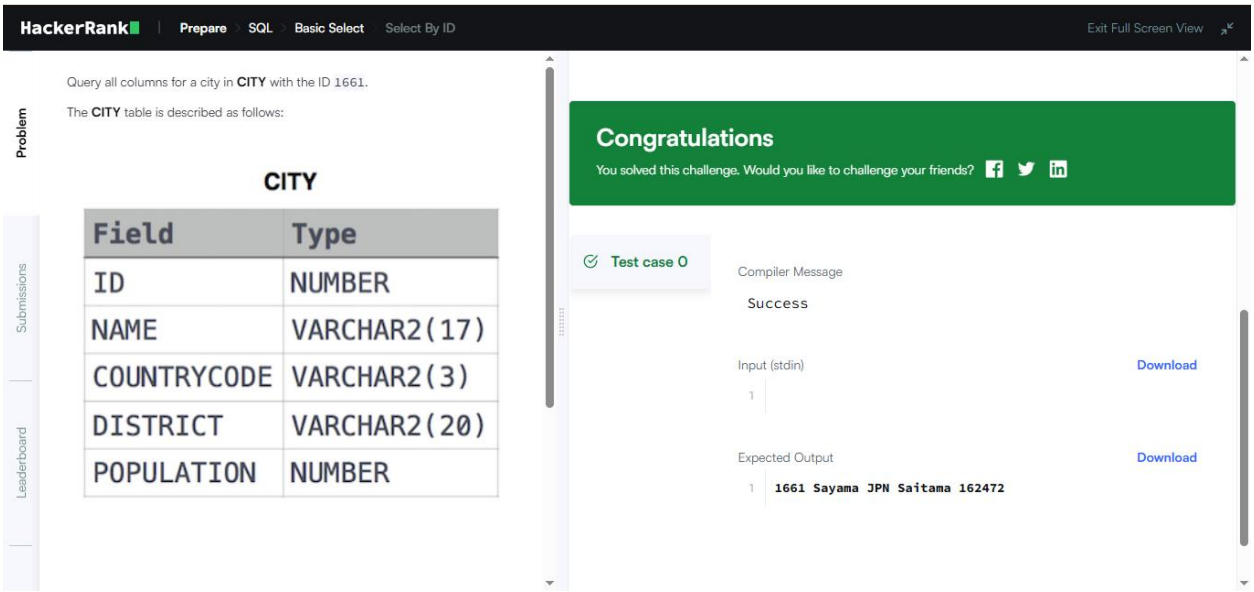
Query all columns for a city in **CITY** with the *ID* 1661.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city where id = 1661;



JAPANESE CITIES' ATTRIBUTES

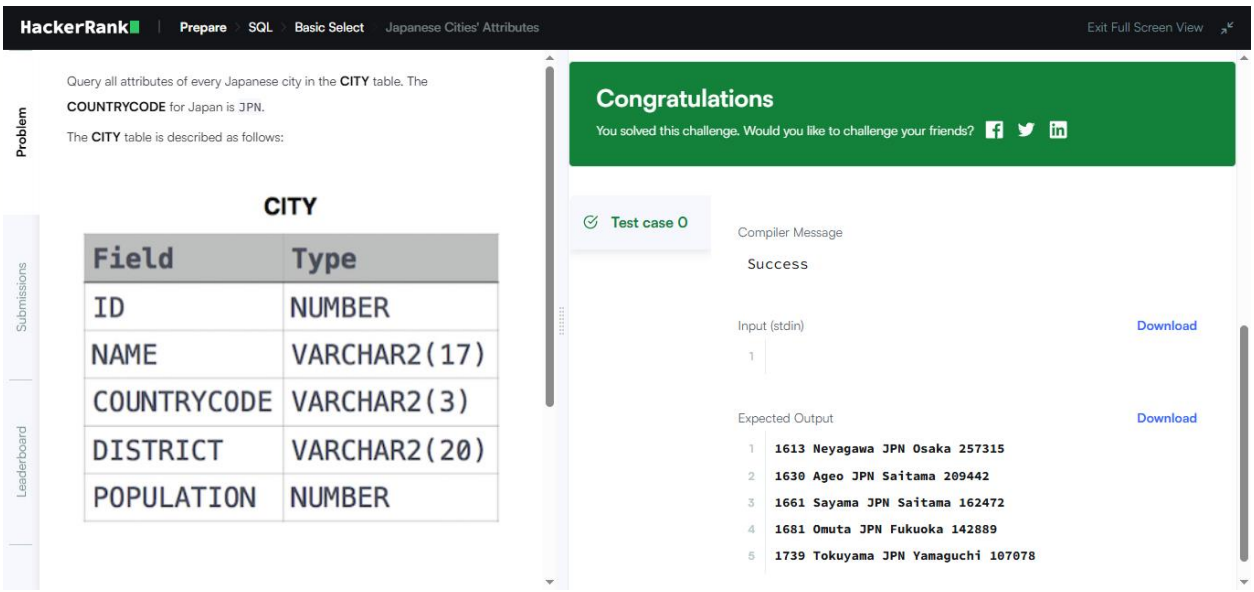
Query all attributes of every Japanese city in the **CITY** table. The **COUNTRYCODE** for Japan is JPN.

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city where countrycode = 'jpn';



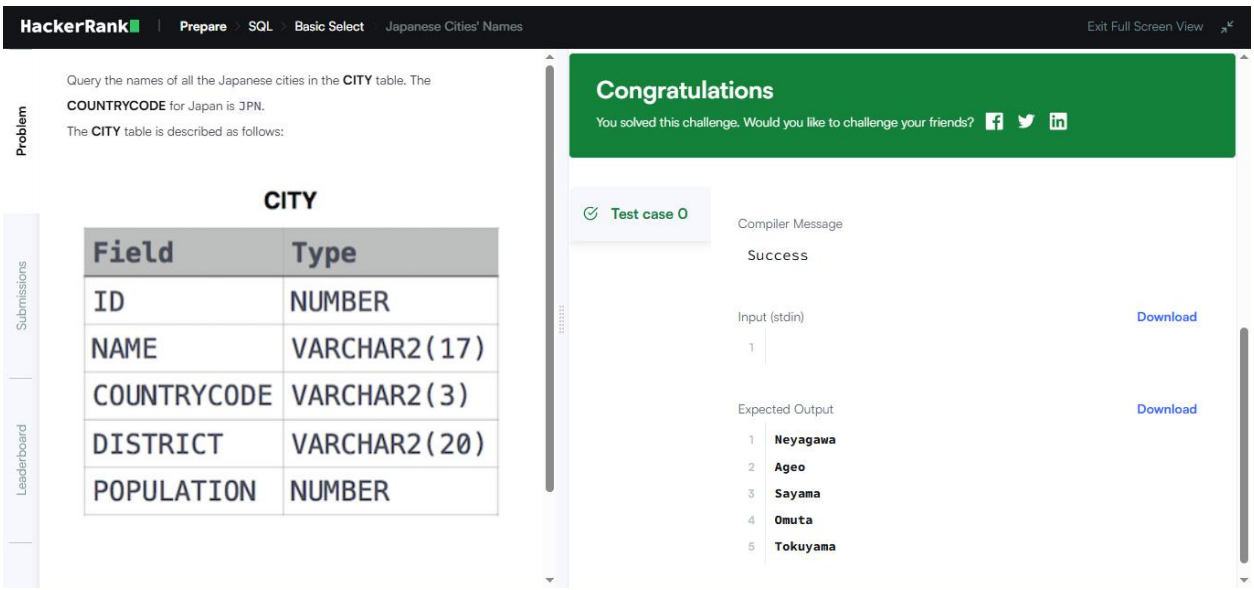
JAPANESE CITIES' NAMES

Query the names of all the Japanese cities in the **CITY** table. The **COUNTRYCODE** for Japan is JPN.
The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
select name from city where countrycode = 'jpn';
```



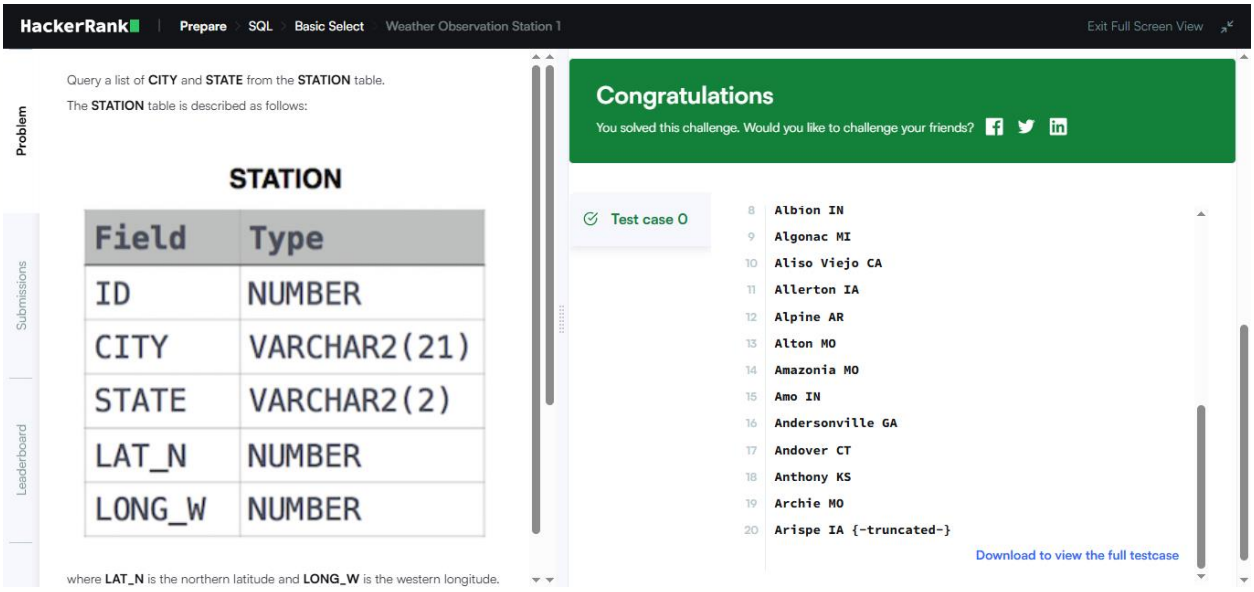
WEATHER OBSERVATION STATION 1

Query a list of **CITY** and **STATE** from the **STATION** table.
The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select city, state from station;



WEATHER OBSERVATION STATION 3

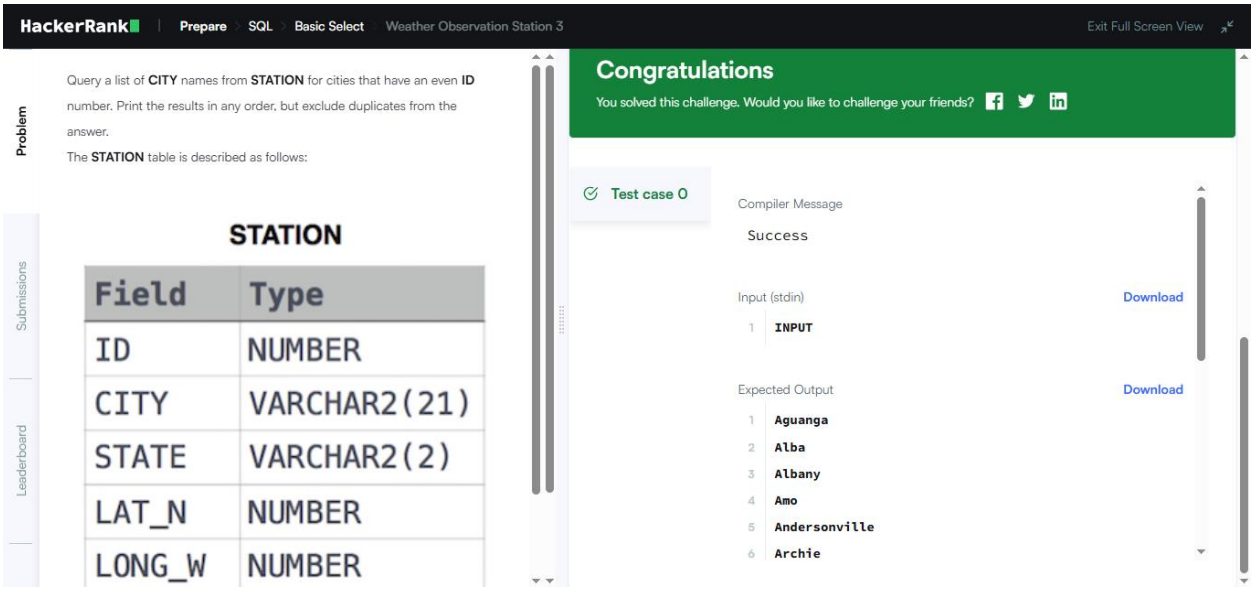
Query a list of **CITY** names from **STATION** for cities that have an even **ID** number. Print the results in any order, but exclude duplicates from the answer.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where id % 2 = 0;



WEATHER OBSERVATION STATION 4

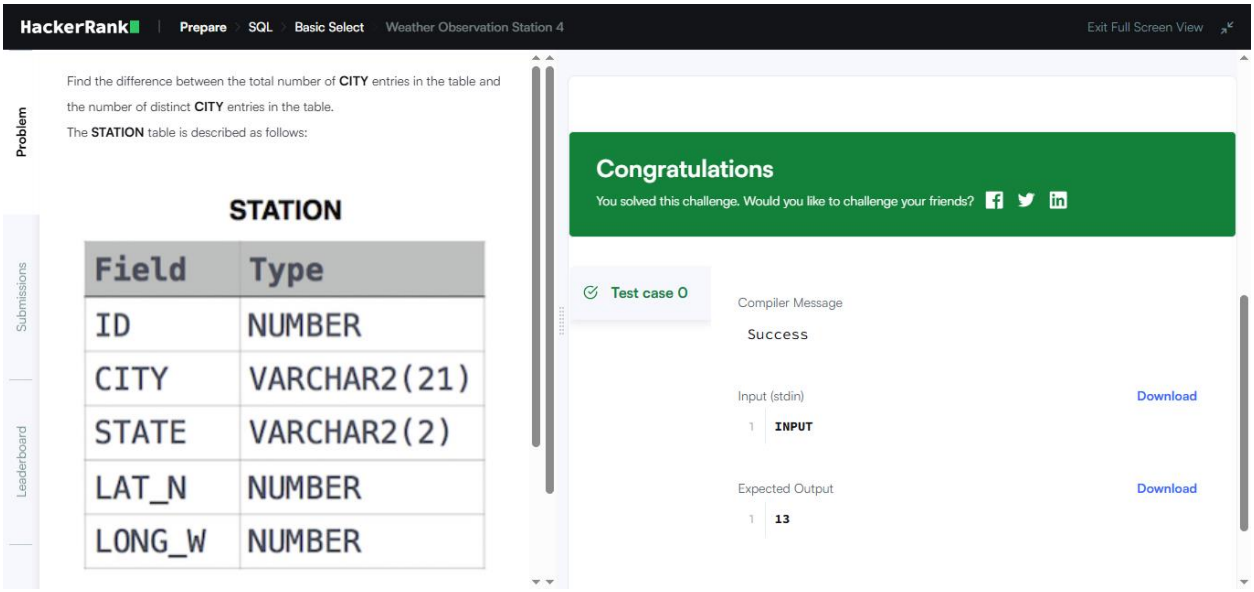
Find the difference between the total number of **CITY** entries in the table and the number of distinct **CITY** entries in the table.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select count(city) - count(distinct city)
from station;
```



WEATHER OBSERVATION STATION 5

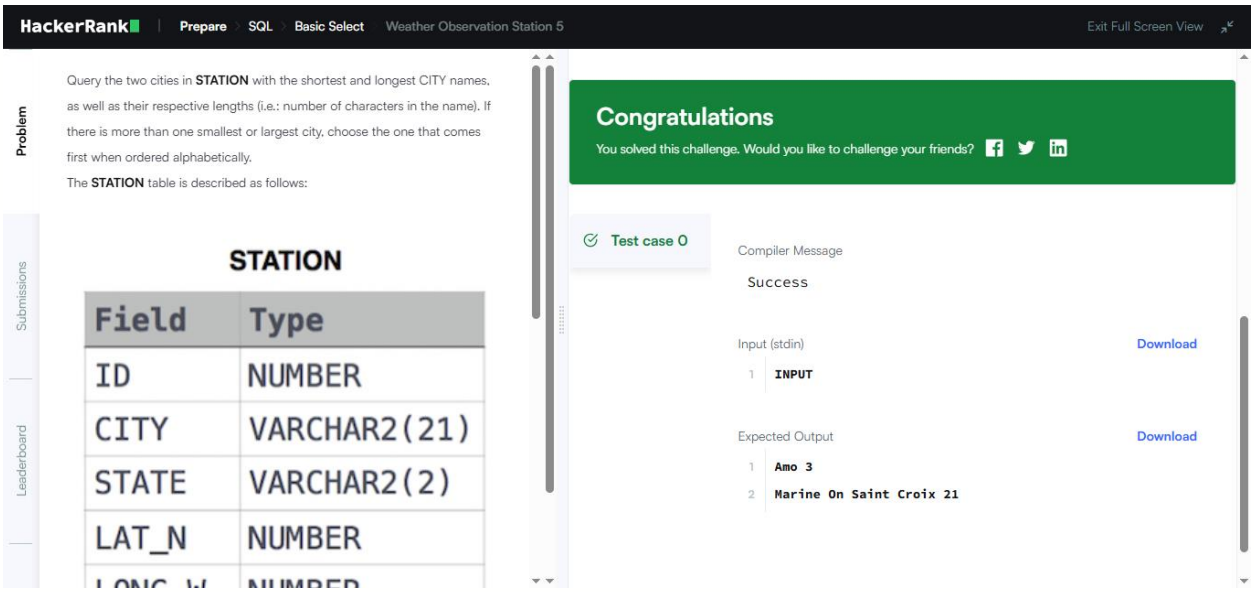
Query the two cities in **STATION** with the shortest and longest *CITY* names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select city, length(city)from station order by length(city) asc ,city asc limit 1;
select city, length(city)from stationorder by length(city) desc, city asc limit 1;



WEATHER OBSERVATION STATION 6

Query the list of *CITY* names starting with vowels (i.e., a, e, i, o, or u) from **STATION**. Your result *cannot* contain duplicates.

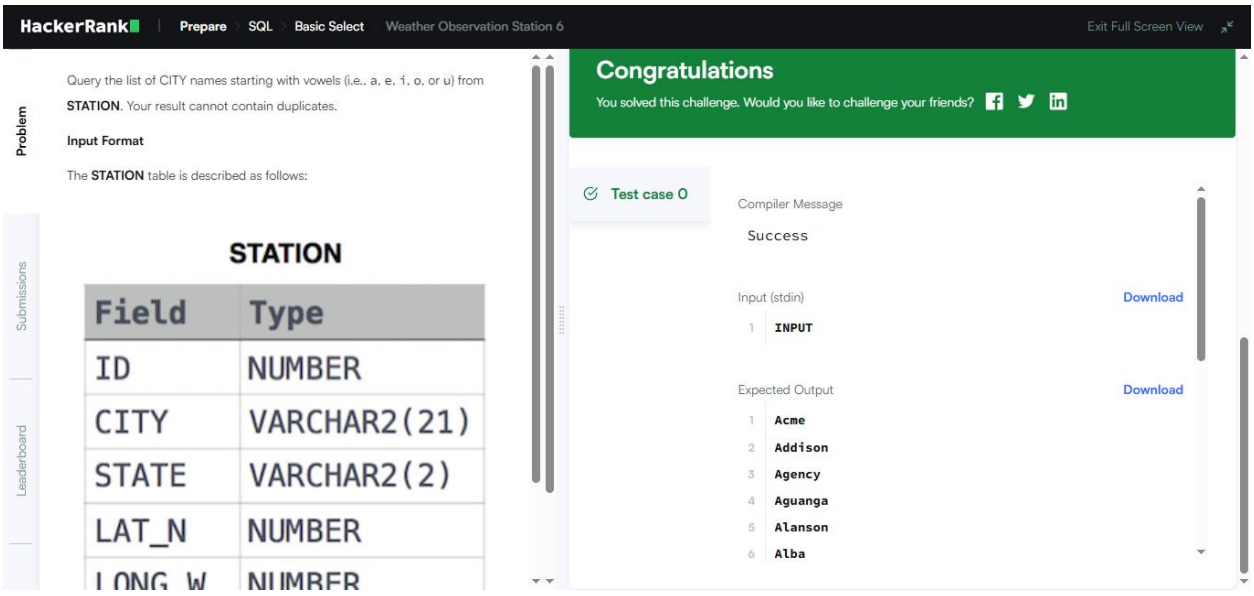
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '^[aeiouaeiou]';



WEATHER OBSERVATION STATION 7

Query the list of *CITY* names ending with vowels (a, e, i, o, u) from **STATION**. Your result *cannot* contain duplicates.

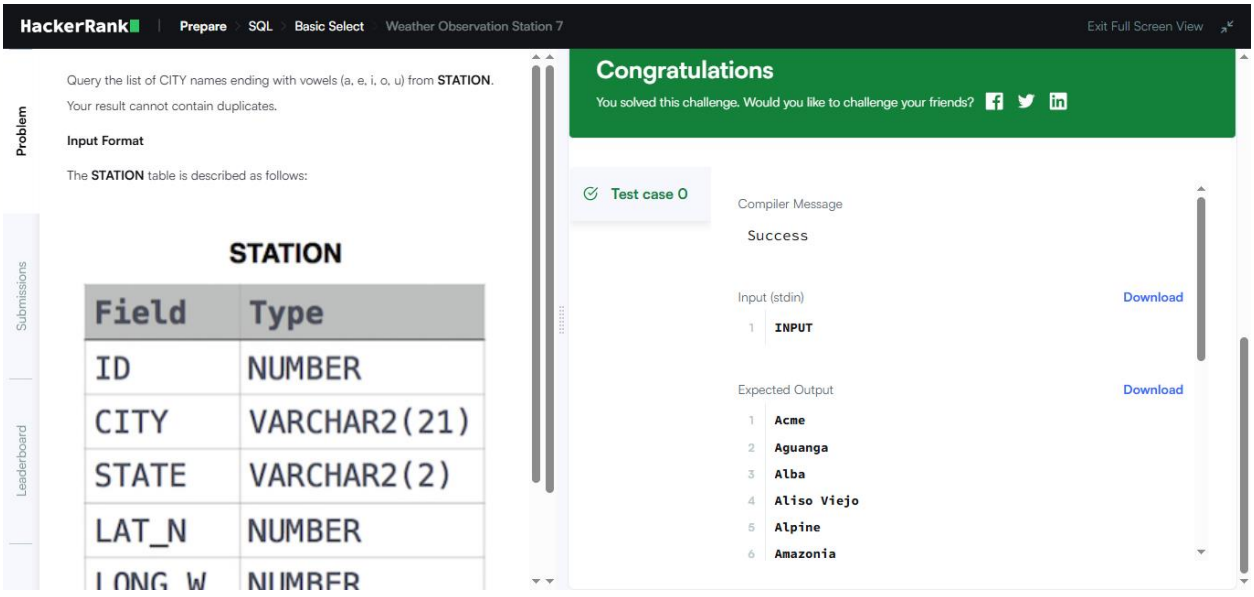
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '[aeiouaeiou]$';



WEATHER OBSERVATION STATION 8

Query the list of *CITY* names from **STATION** which have vowels (i.e., *a*, *e*, *i*, *o*, and *u*) as both their first *and* last characters. Your result cannot contain duplicates.

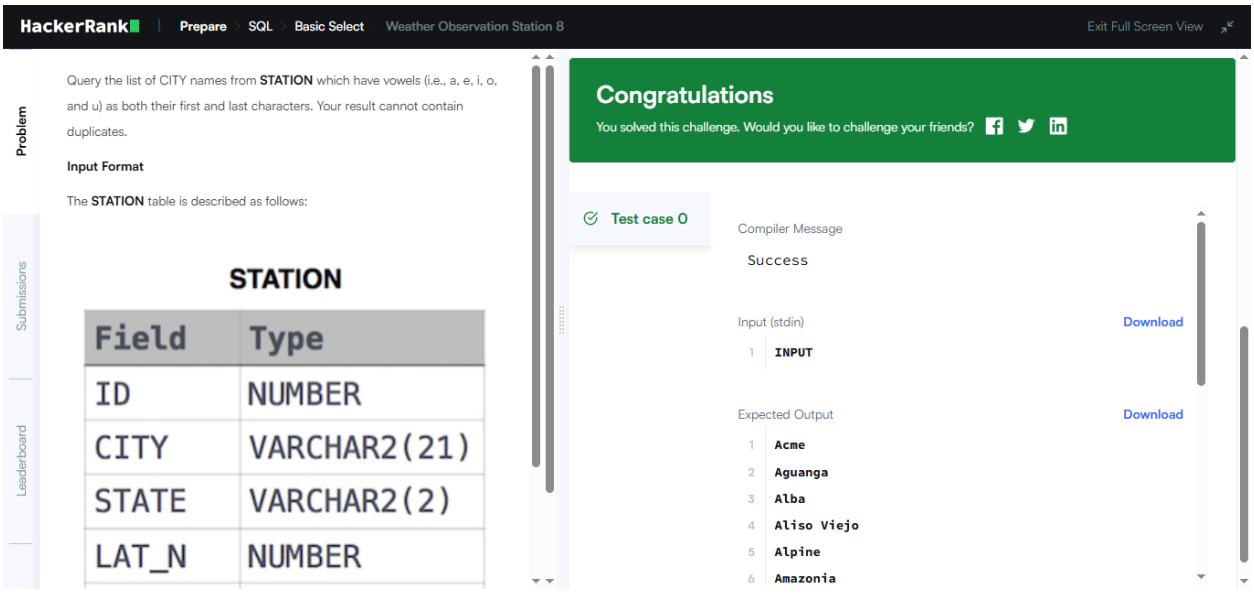
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '^[aeiou].*[aeiou]$';



WEATHER OBSERVATION STATION 9

Query the list of *CITY* names from **STATION** that *do not start* with vowels. Your result cannot contain duplicates.

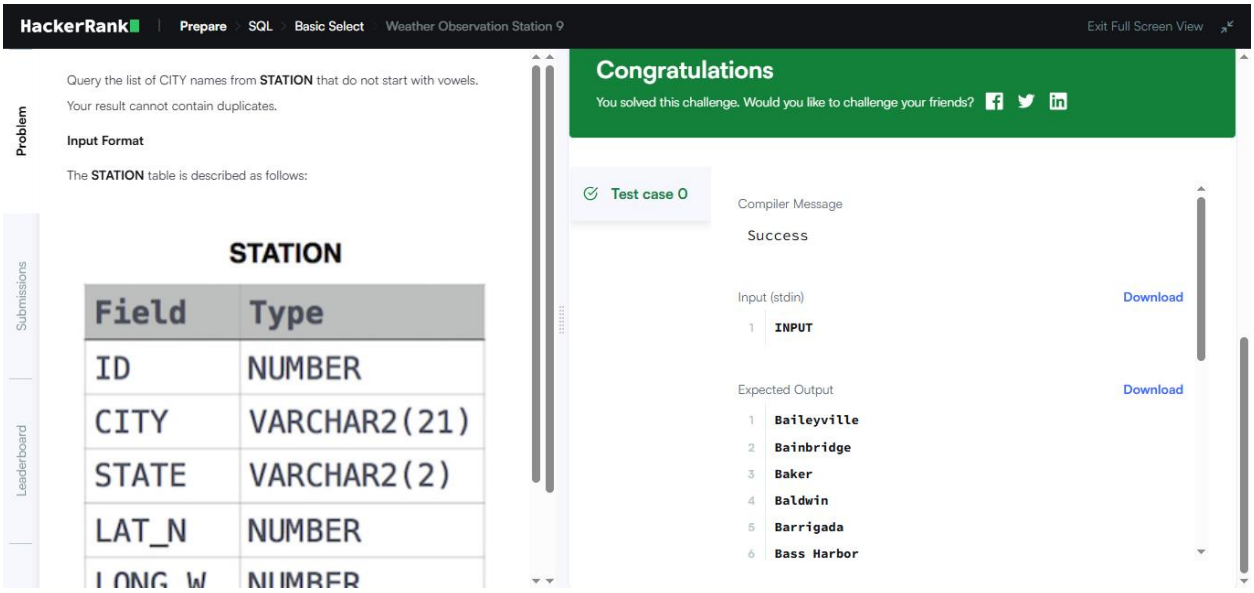
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '^[aeiou].*[aeiou]$';



WEATHER OBSERVATION STATION 10

Query the list of *CITY* names from **STATION** that *do not end* with vowels. Your result cannot contain duplicates.

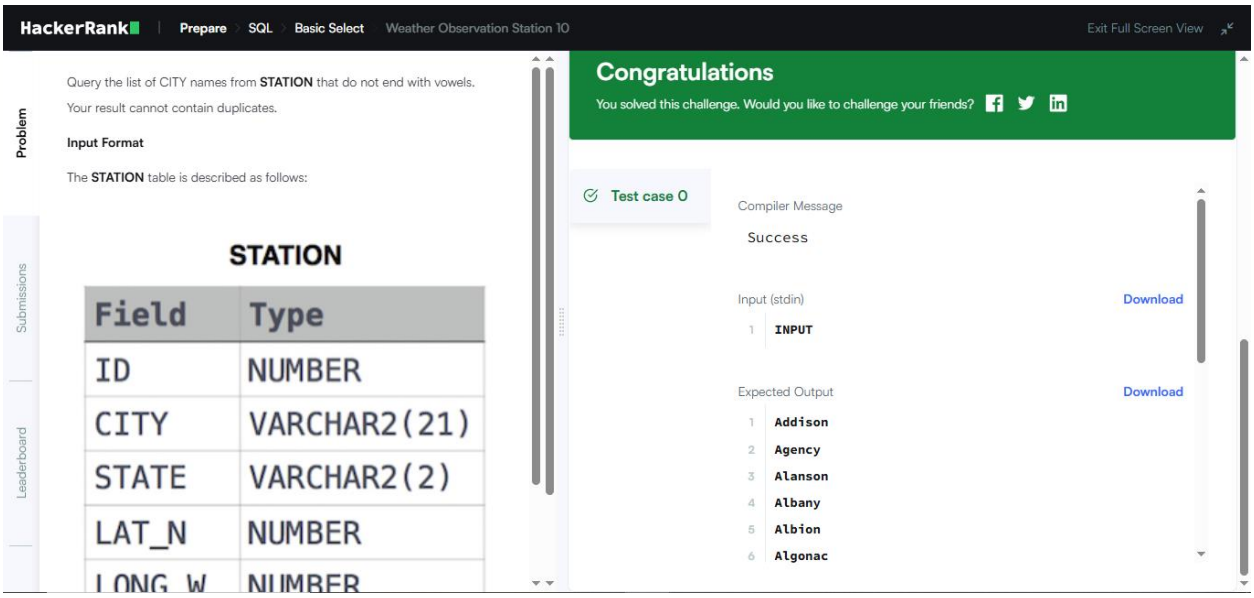
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '[^aeiouaeiou]$';



WEATHER OBSERVATION STATION 11

Query the list of *CITY* names from **STATION** that either do not start with vowels or do not end with vowels.

Your result cannot contain duplicates.

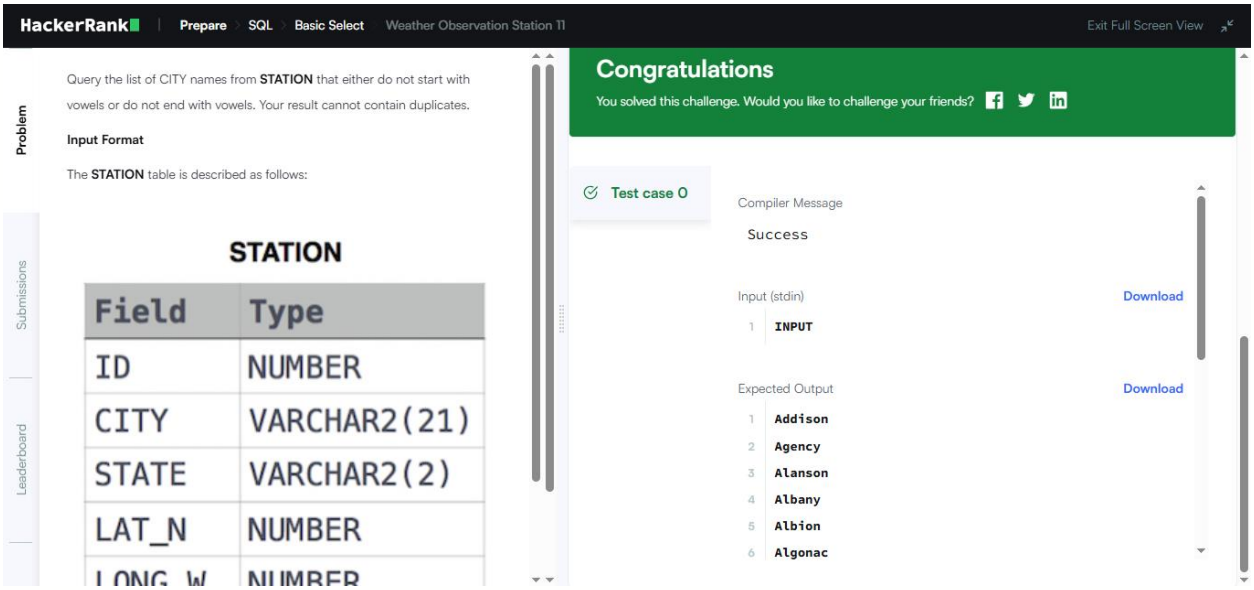
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where left(city, 1) not in ('a','e','i','o','u','a','e','i','o','u') or right(city, 1) not in ('a','e','i','o','u','a','e','i','o','u');



WEATHER OBSERVATION STATION 12

Query the list of *CITY* names from **STATION** that *do not start* with vowels and *do not end* with vowels. Your result cannot contain duplicates.

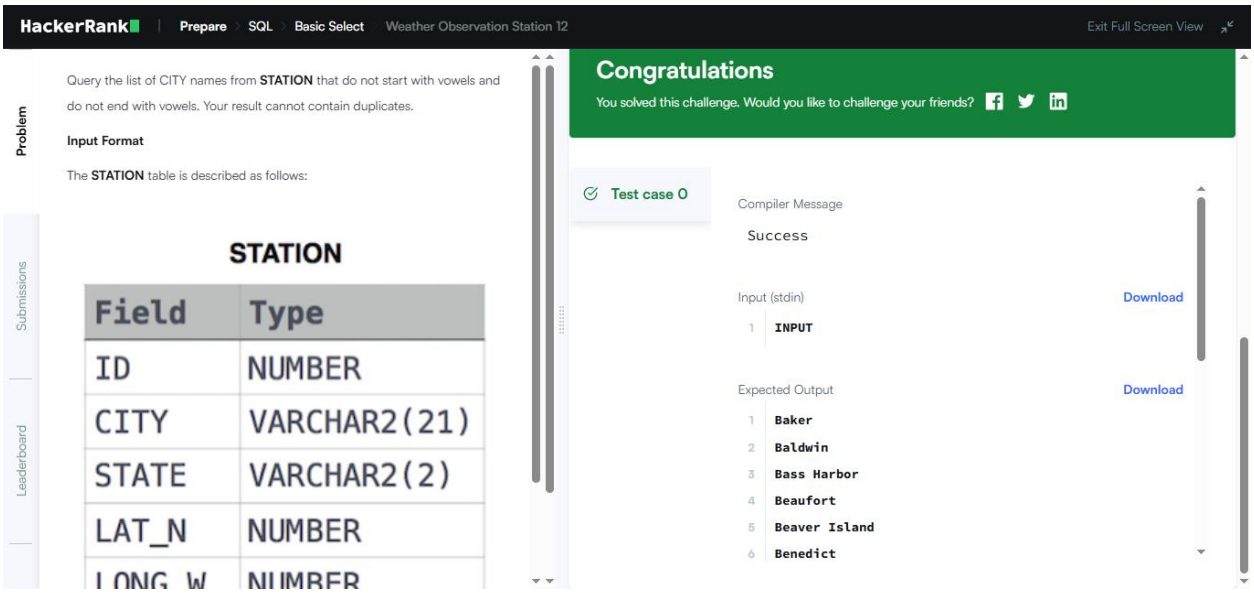
Input Format

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city not regexp '^[aeiouaeiou]' and city not regexp '[aeiouaeiou]$';



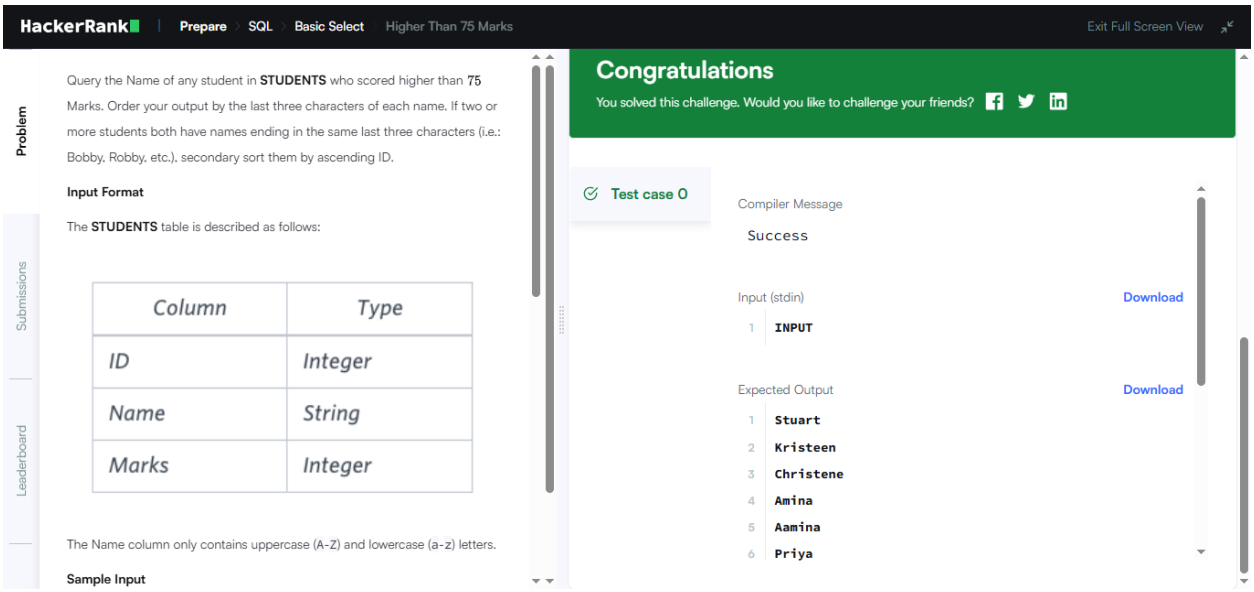
HIGHER THAN 75 MARKS

Query the *Name* of any student in **STUDENTS** who scored higher than *Marks*. Order your output by the *last three characters* of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending *ID*.

Input Format

Column	Type
ID	Integer
Name	String
Marks	Integer

select name from students where marks > 75 order by right(name, 3), id asc;



EMPLOYEE NAMES

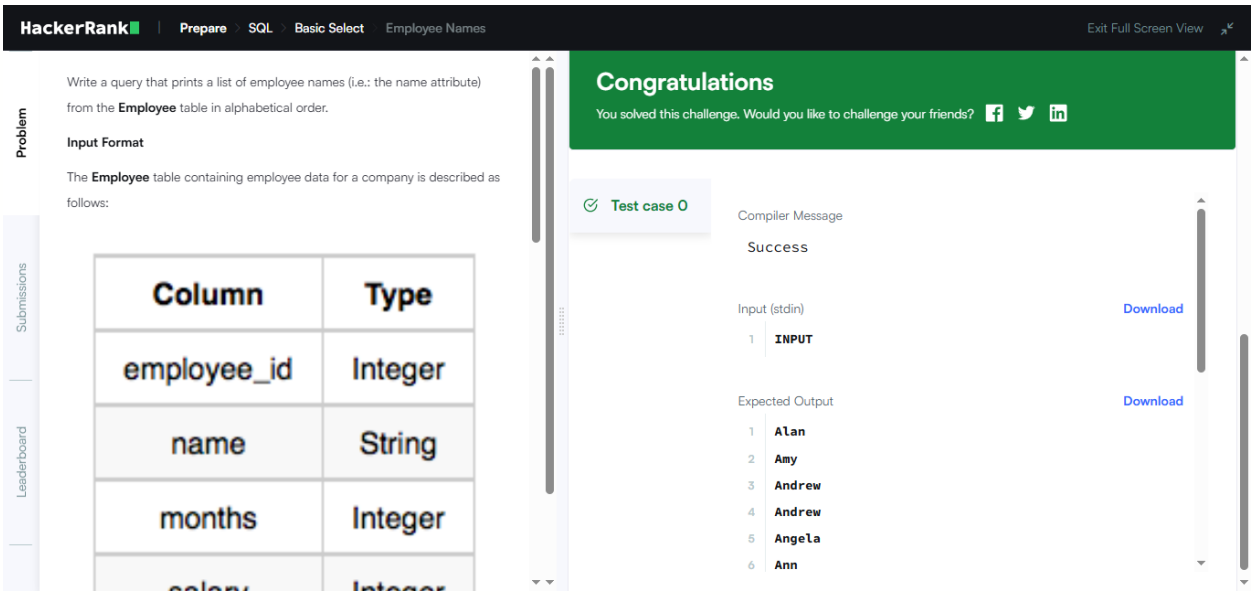
Write a query that prints a list of employee names (i.e.: the *name* attribute) from the **Employee** table in alphabetical order.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

select name from employee order by name;



EMPLOYEE SALARIES

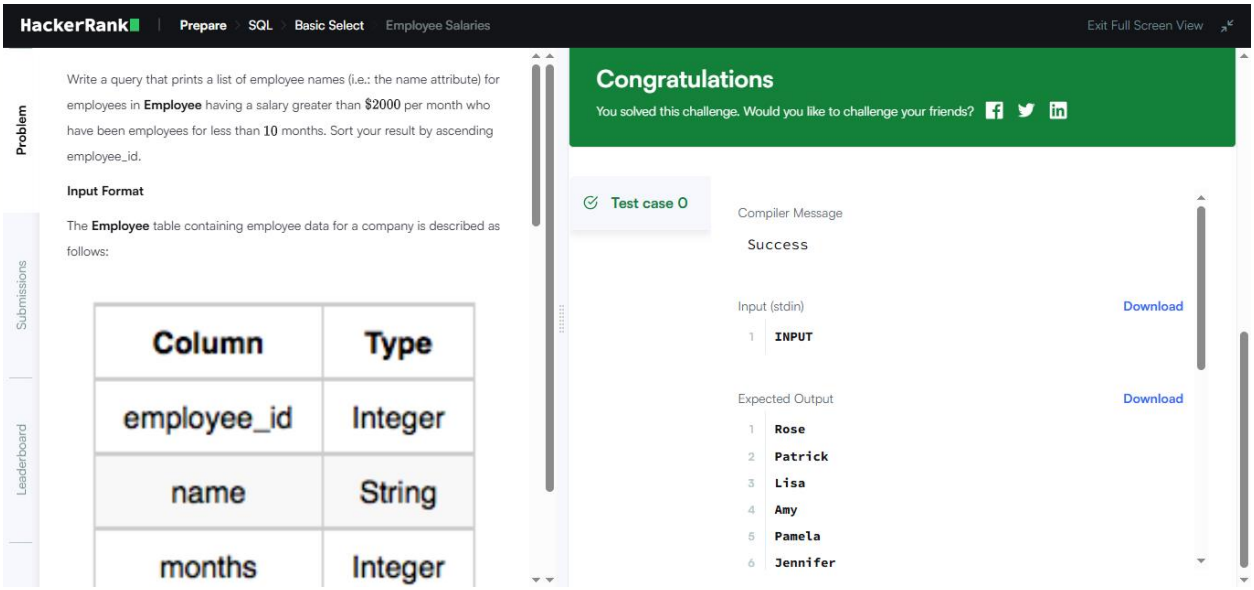
Write a query that prints a list of employee names (i.e.: the *name* attribute) for employees in **Employee** having a salary greater than per month who have been employees for less than months. Sort your result by ascending *employee_id*.

Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

select name from employee where salary > 2000 and months < 10 order by employee_id asc;



TYPE OF TRIANGLE

Write a query identifying the *type* of each record in the **TRIANGLES** table using its three side lengths. Output one of the following statements for each record in the table:

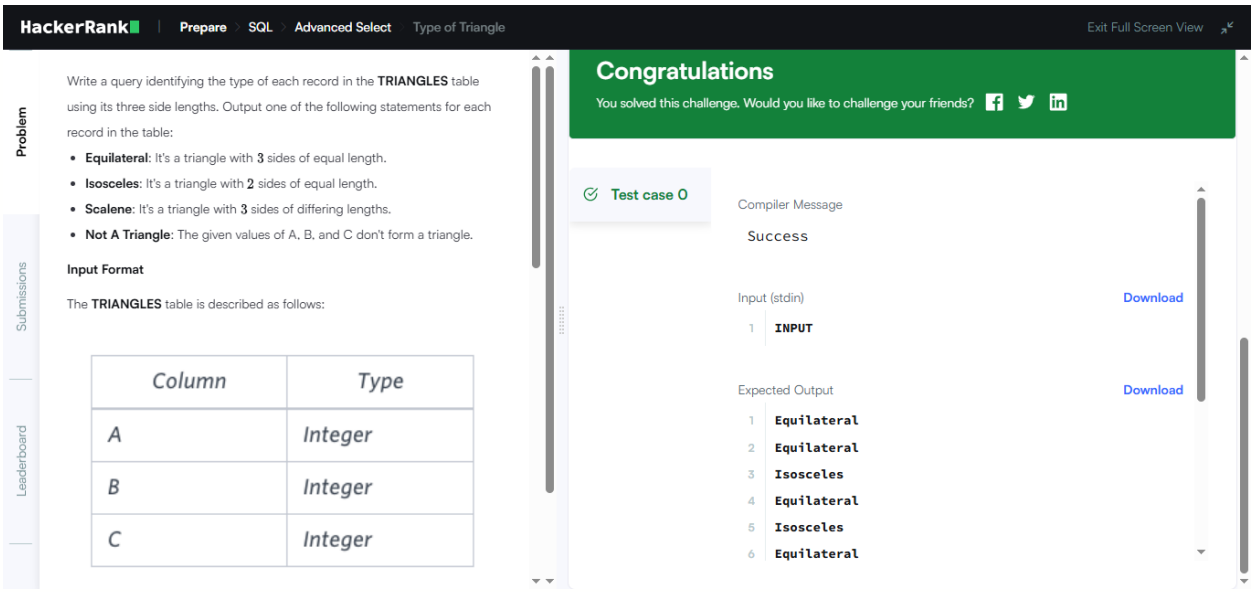
- **Equilateral**: It's a triangle with sides of equal length.
- **Isosceles**: It's a triangle with sides of equal length.
- **Scalene**: It's a triangle with sides of differing lengths.
- **Not A Triangle**: The given values of *A*, *B*, and *C* don't form a triangle.

Input Format

The **TRIANGLES** table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

```
select
   case
      when a + b <= c or a + c <= b or b + c <= a then 'not a triangle'
      when a = b and b = c then 'equilateral'
      when a = b or b = c or a = c then 'isosceles'
      else 'scalene'
   end
from triangles;
```



REVISING AGGREGATIONS - THE COUNT FUNCTION

Query a *count* of the number of cities in **CITY** having a *Population* larger than .

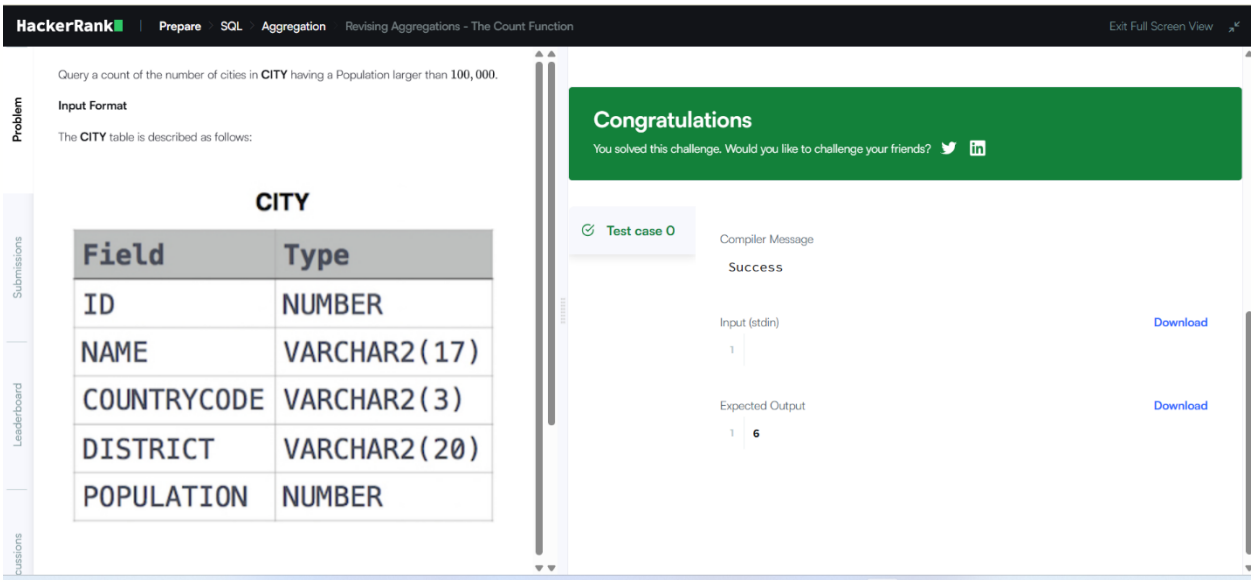
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

SELECT COUNT(*) FROM CITY WHERE POPULATION > 100000;



REVISING AGGREGATIONS - THE SUM FUNCTION

Query the total population of all cities in **CITY** where *District* is **California**.

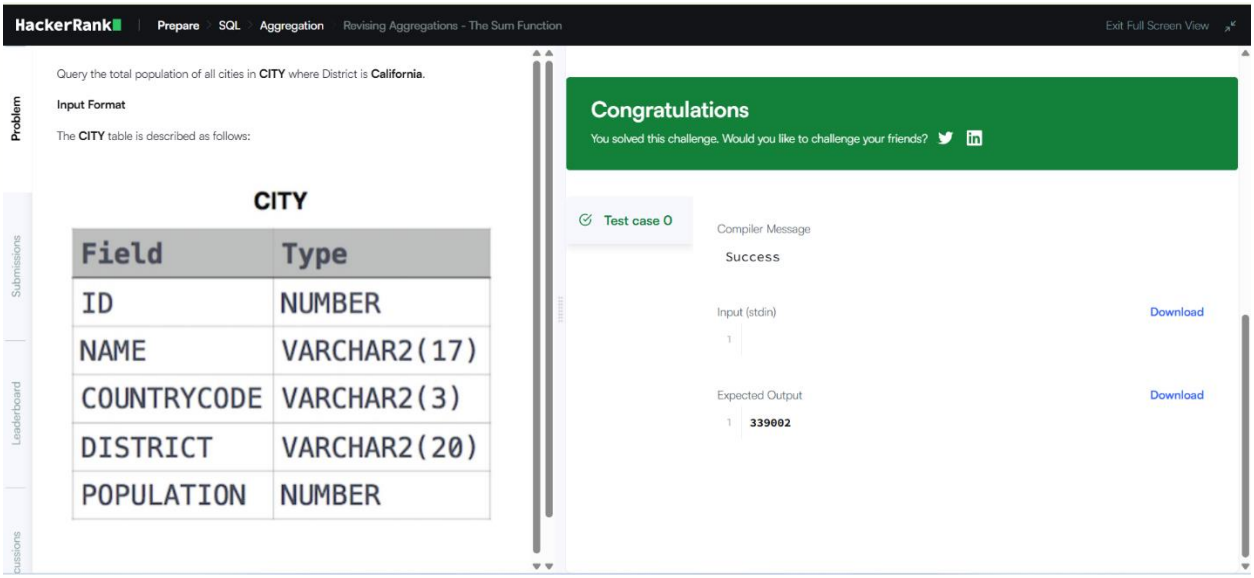
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT SUM(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```



REVISING AGGREGATIONS – AVERAGES

Query the average population of all cities in **CITY** where *District* is **California**.

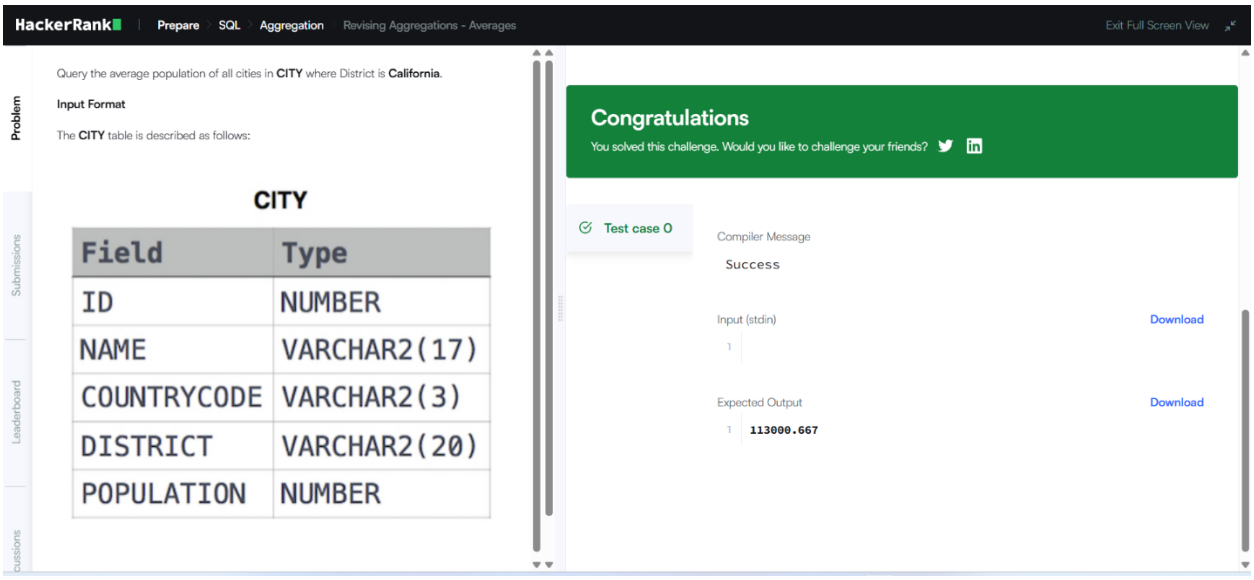
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT AVG(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```



AVERAGE POPULATION

Query the average population for all cities in **CITY**, rounded *down* to the nearest integer.

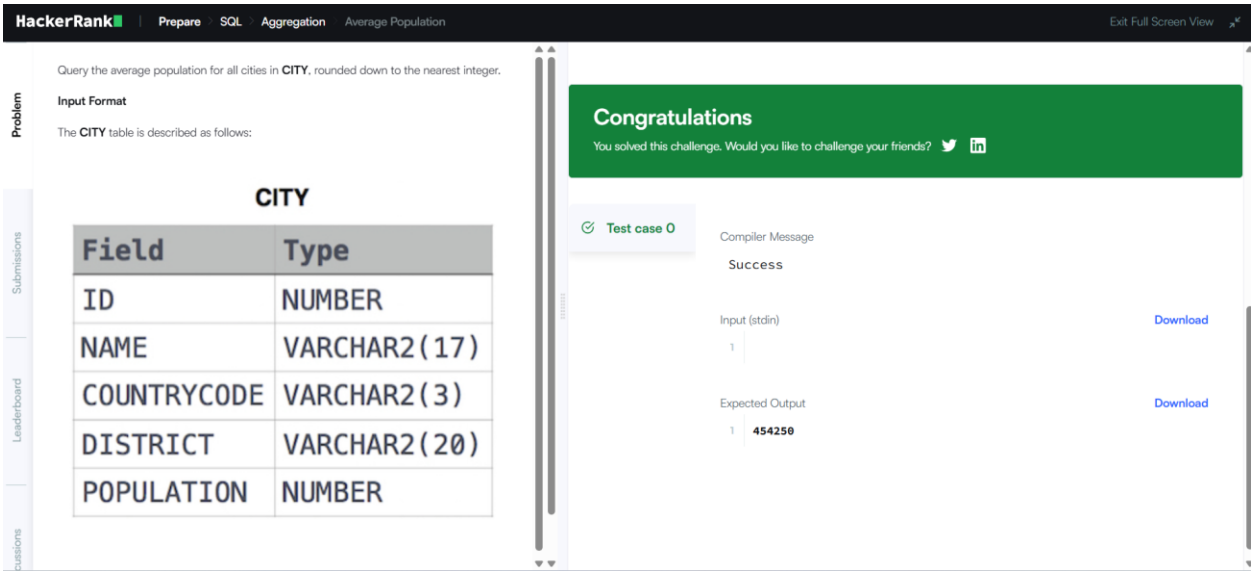
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT  FLOOR (AVG(POPULATION)) FROM CITY;
```



JAPAN POPULATION

Query the sum of the populations for all Japanese cities in **CITY**. The *COUNTRYCODE* for Japan is **JPN**.

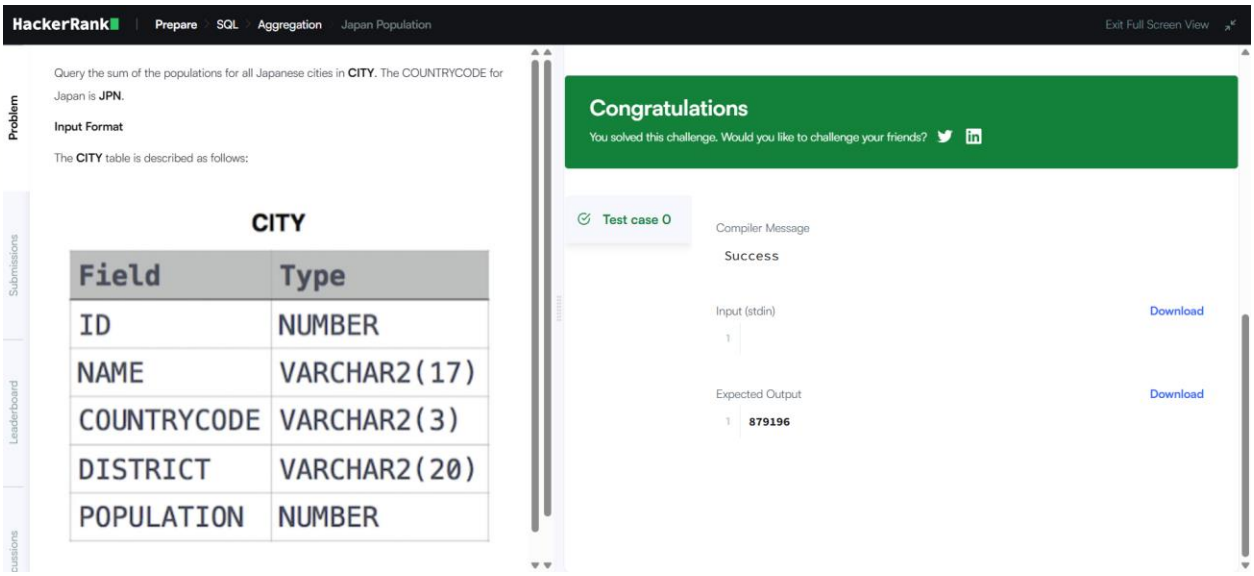
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT SUM(POPULATION) FROM CITY WHERE COUNTRYCODE = 'JPN';
```



POPULATION DENSITY DIFFERENCE

Query the difference between the maximum and minimum populations in **CITY**.

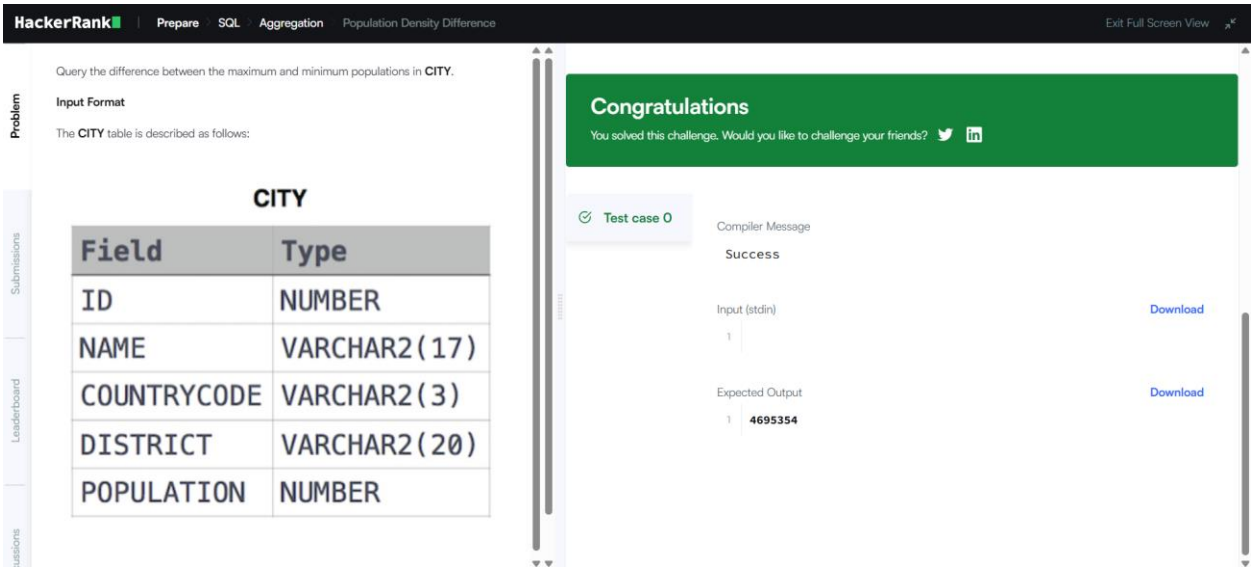
Input Format

The **CITY** table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT MAX(POPULATION) - MIN(POPULATION) FROM CITY;
```



THE BLUNDER

Samantha was tasked with calculating the average monthly salaries for all employees in the **EMPLOYEES** table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

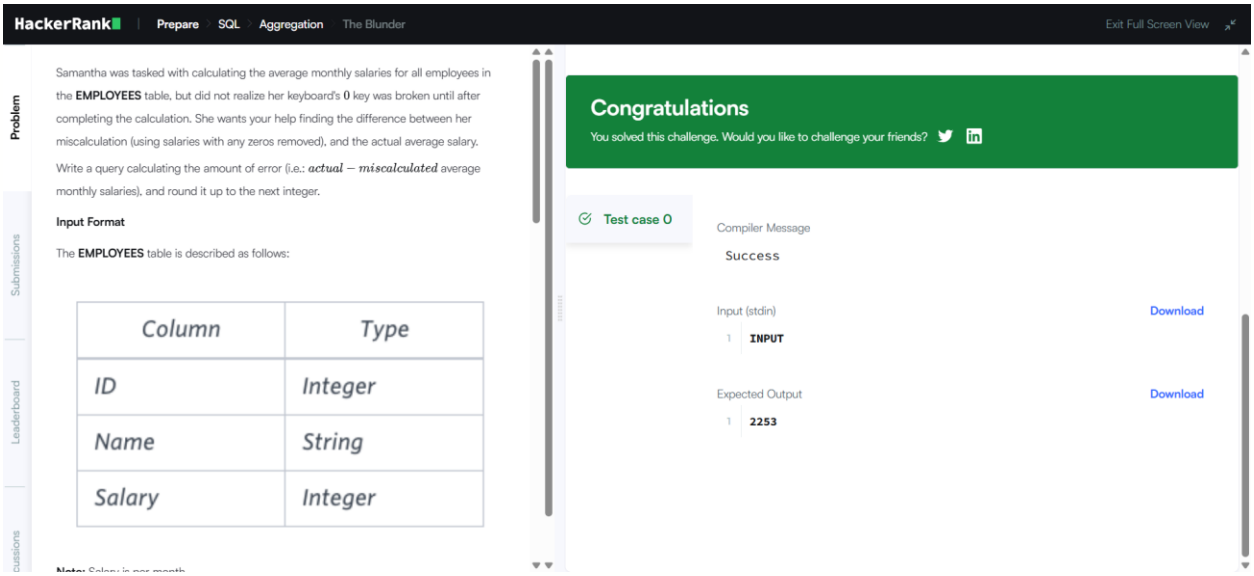
Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

The **EMPLOYEES** table is described as follows:

Column	*Type*
ID	*Integer*
Name	*String*
Salary	*Integer*

SELECT CEIL(AVG(Salary) - AVG(CAST(REPLACE(Salary, '0', '') AS UNSIGNED)))FROM EMPLOYEES;



TOP EARNERS

We define an employee's *total earnings* to be their monthly worked, and the *maximum total earnings* to be the maximum total earnings for any employee in the **Employee** table. Write a query to find the *maximum total earnings* for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

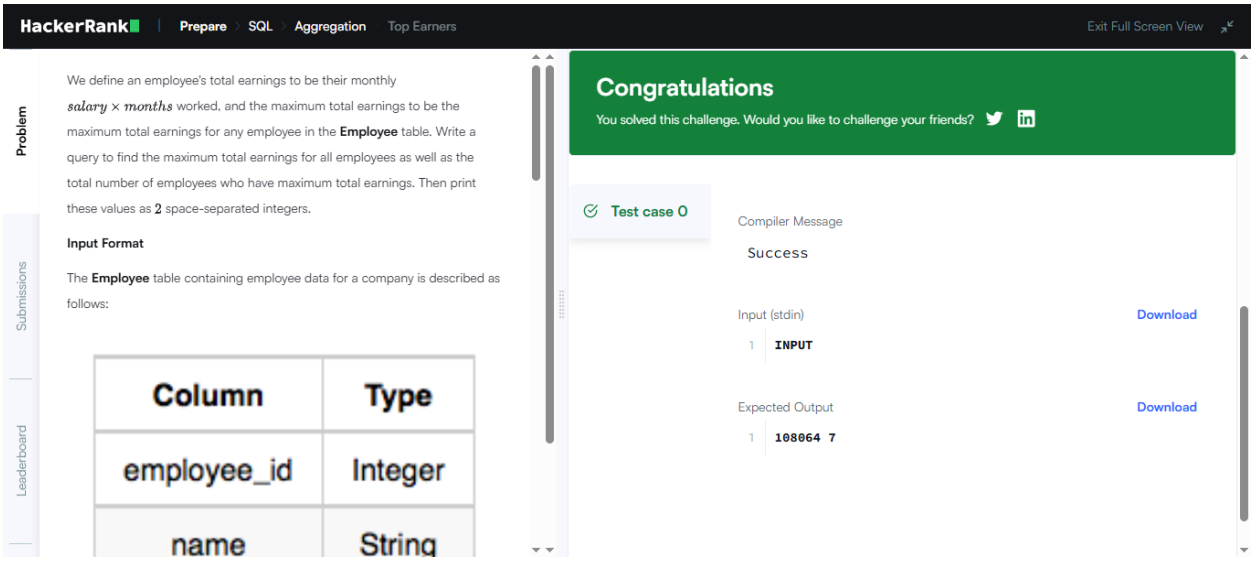
Input Format

The **Employee** table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where *employee_id* is an employee's ID number, *name* is their name, *months* is the total number of months they've been working for the company, and *salary* is the their monthly salary.

```
SELECT
    (salary * months) AS earnings,
    COUNT(*) AS total_count
FROM EMPLOYEE
GROUP BY earnings
ORDER BY earnings DESC
LIMIT 1;
```

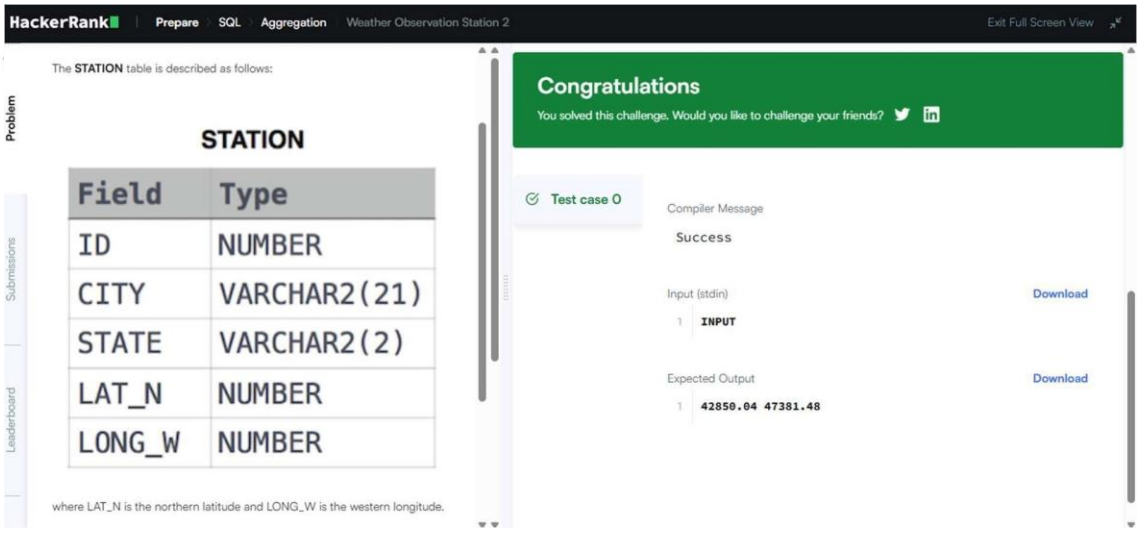


WEATHER OBSERVATION STATION 2

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER



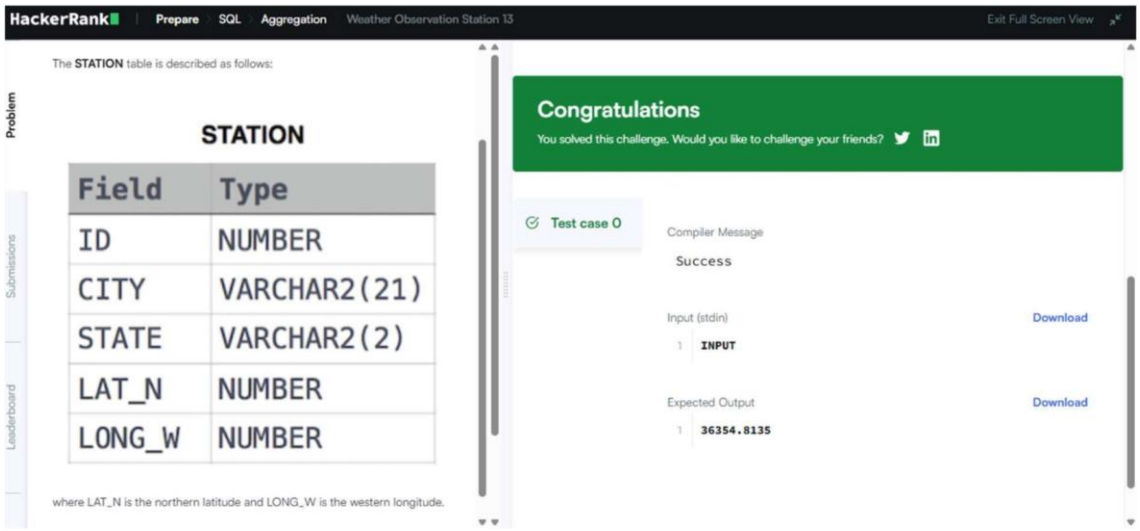
WEATHER OBSERVATION STATION 13

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



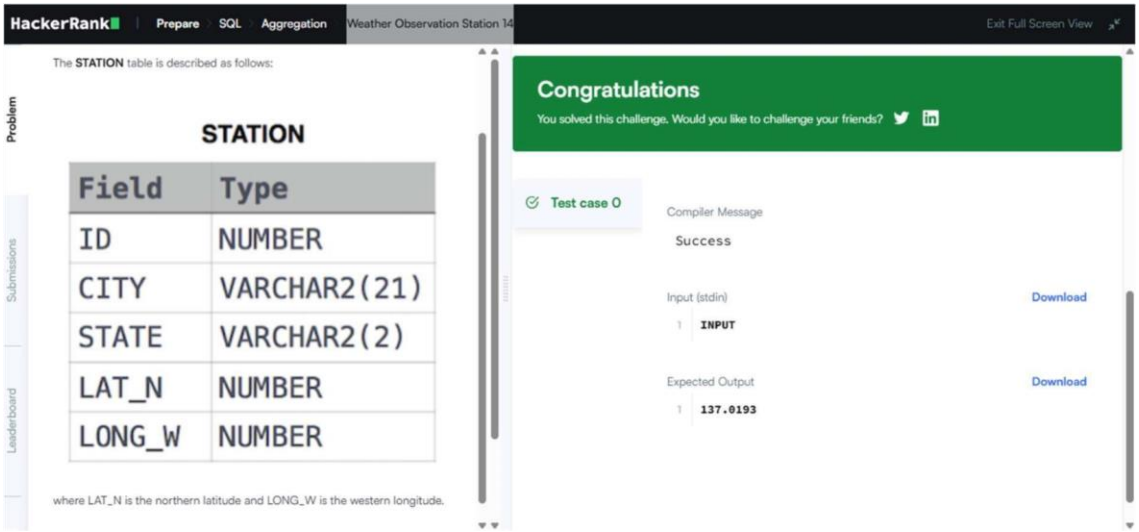
WEATHER OBSERVATION STATION 14

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



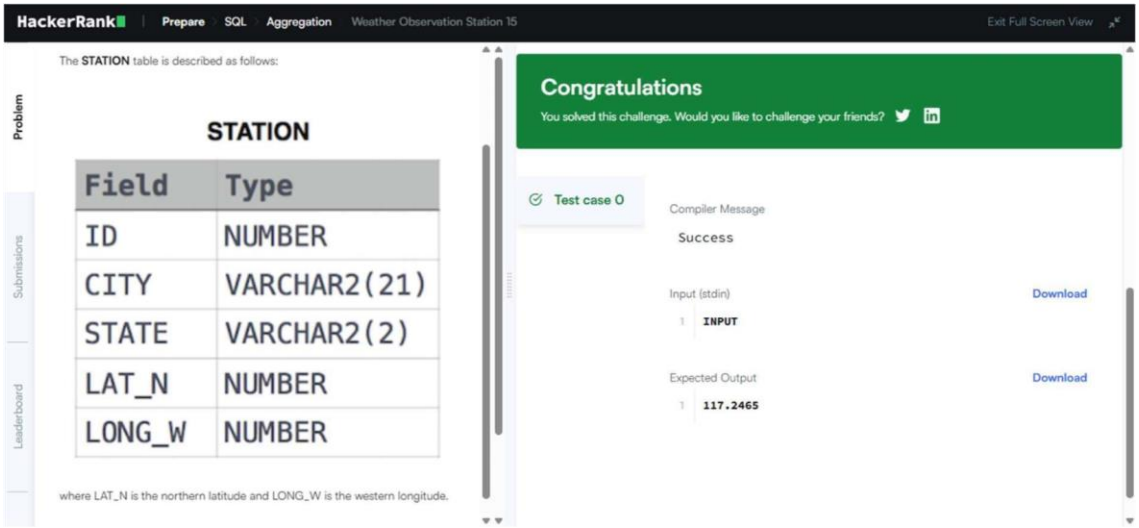
WEATHER OBSERVATION STATION 15

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



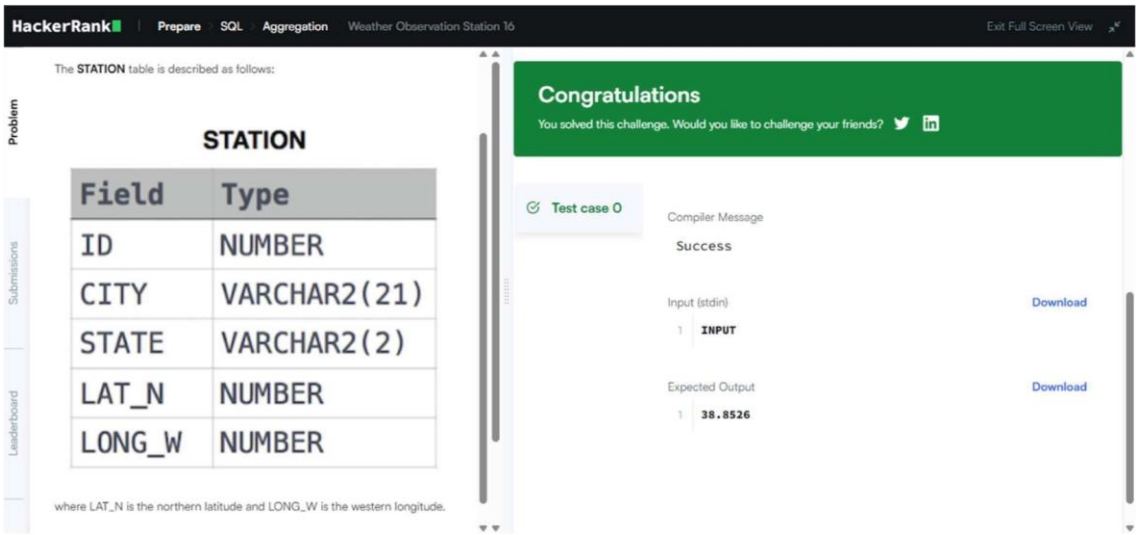
WEATHER OBSERVATION STATION 16

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



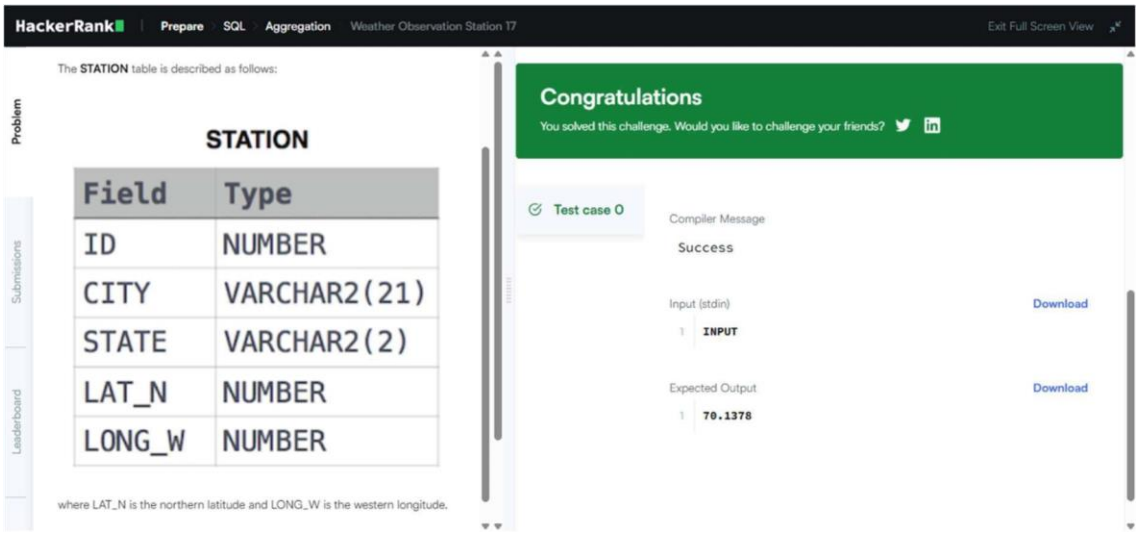
WEATHER OBSERVATION STATION 17

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



POPULATION CENSUS

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the CONTINENT is 'Asia'.

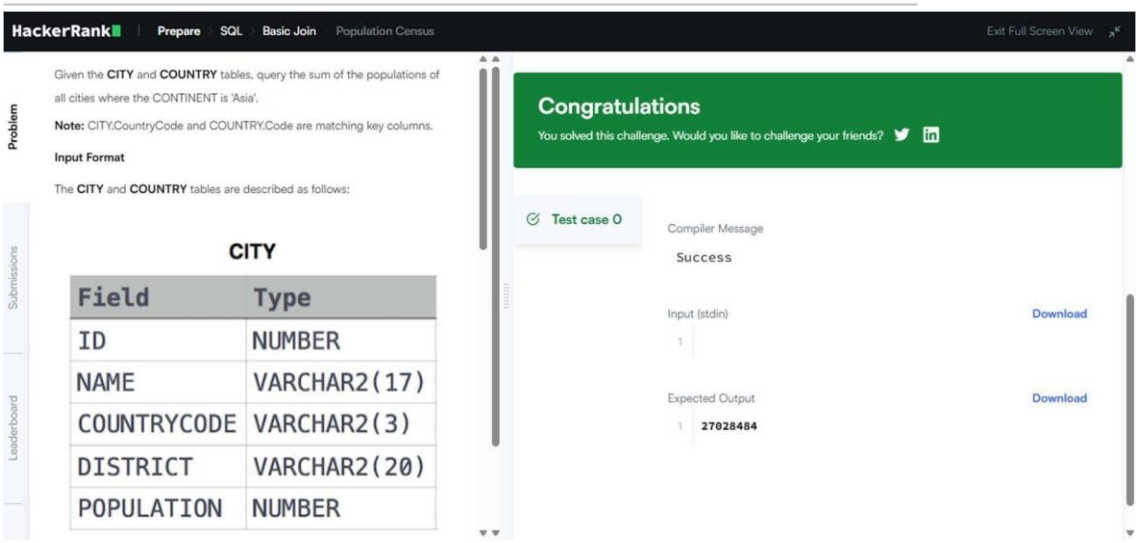
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



AFRICAN CITIES

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the CONTINENT is 'Africa'.

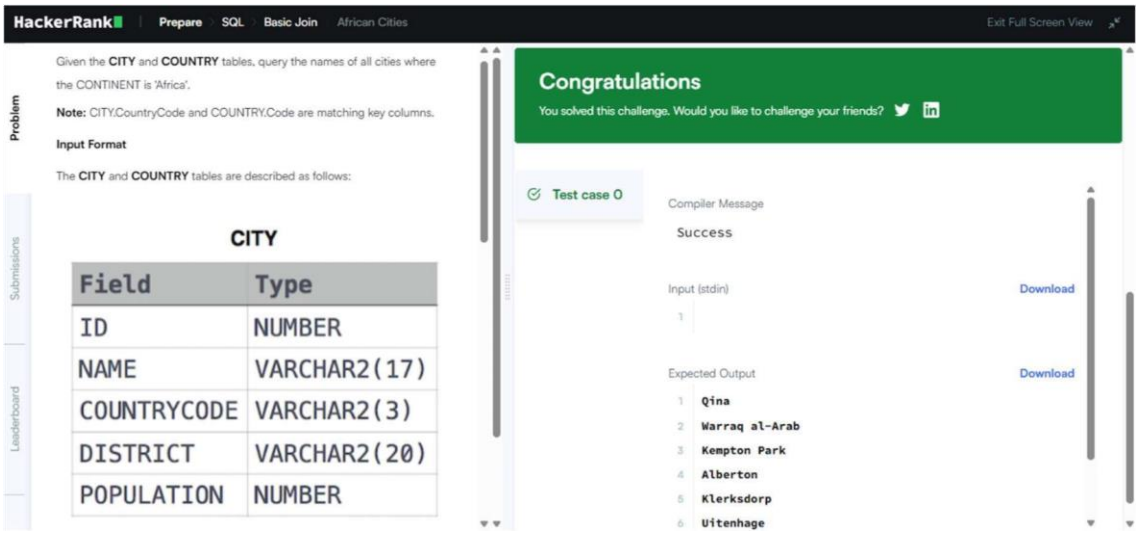
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



AVERAGE POPULATION OF EACH CONTINENT

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (COUNTRY.Continent) and their respective average city populations (CITY.Population) rounded down to the nearest integer.

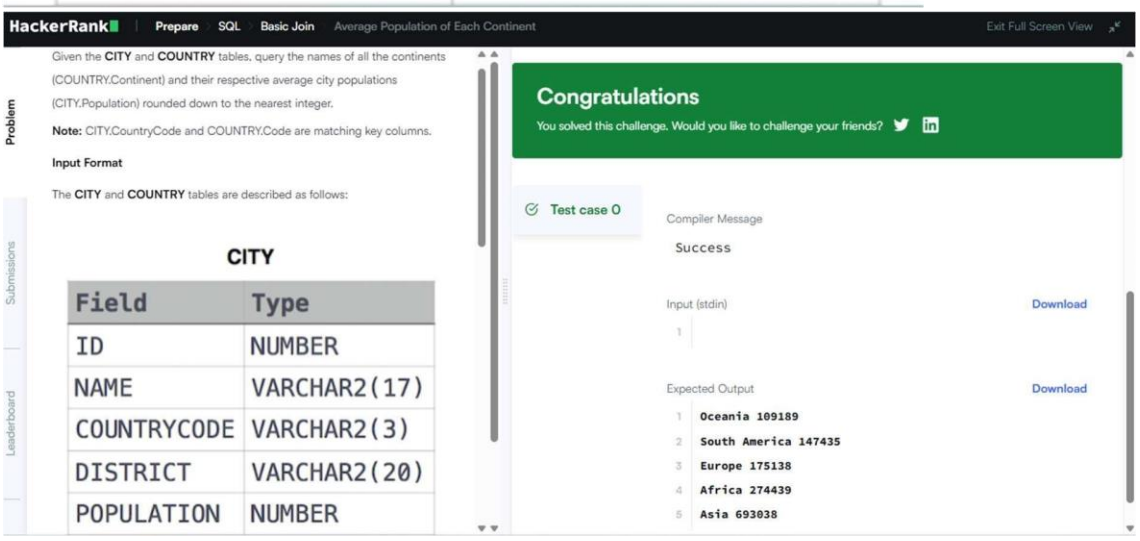
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

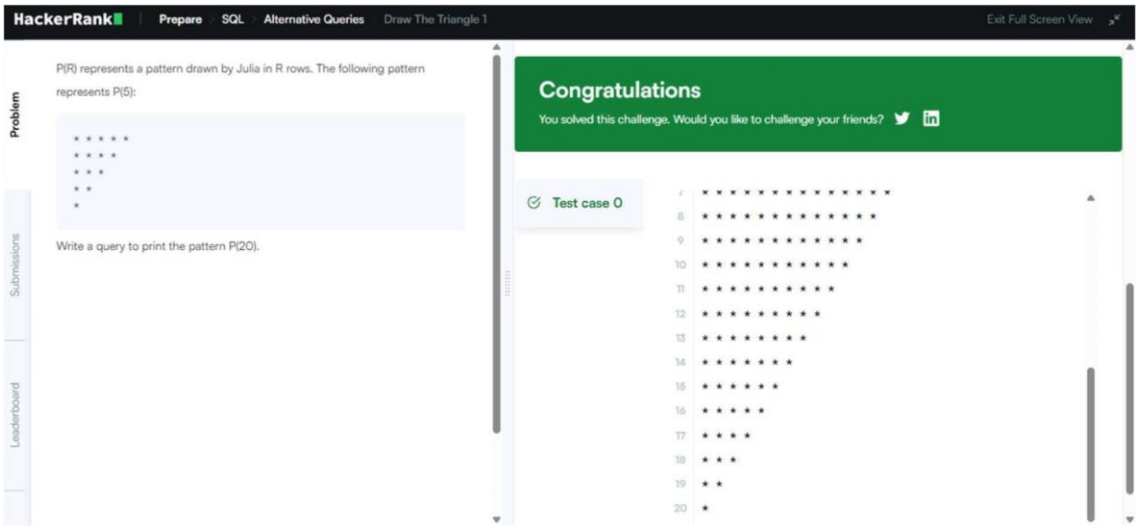


DRAW THE TRIANGLE 1

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern P(20).

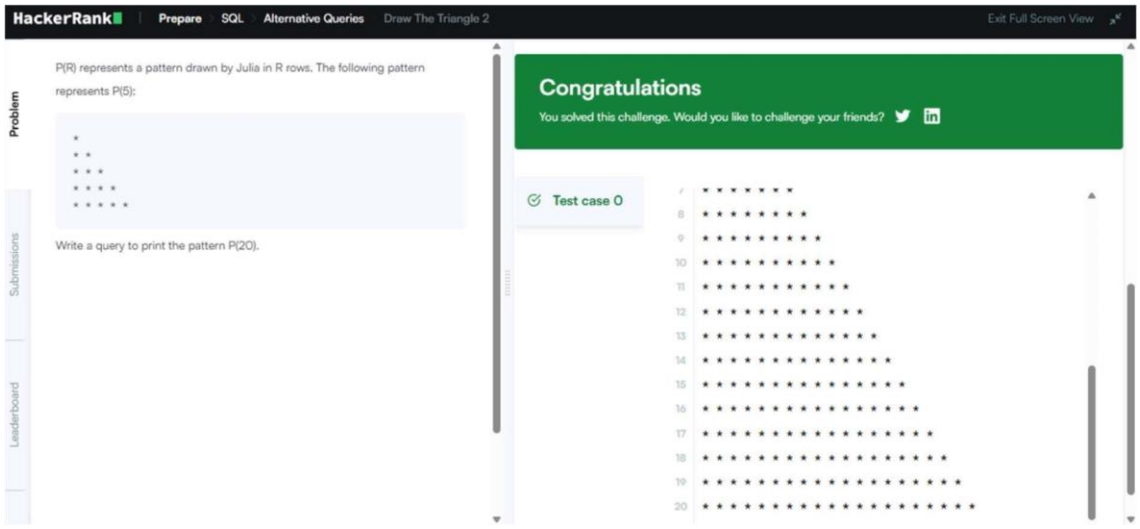


DRAW THE TRIANGLE 2

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern P(20).



THE PADS

Generate the following two result sets:

1. Query an *alphabetically ordered* list of all names in OCCUPATIONS, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).

1.Query the number of ocurrences of each occupation in OCCUPATIONS. Sort the occurrences in *ascending order*, and output them in the following format:where [occupation_count] is the number of occurrences of an occupation in OCCUPATIONS and [occupation] is the *lowercase* occupation name. If more than one *Occupation* has the same [occupation_count], they should be ordered alphabetically.

Input Format

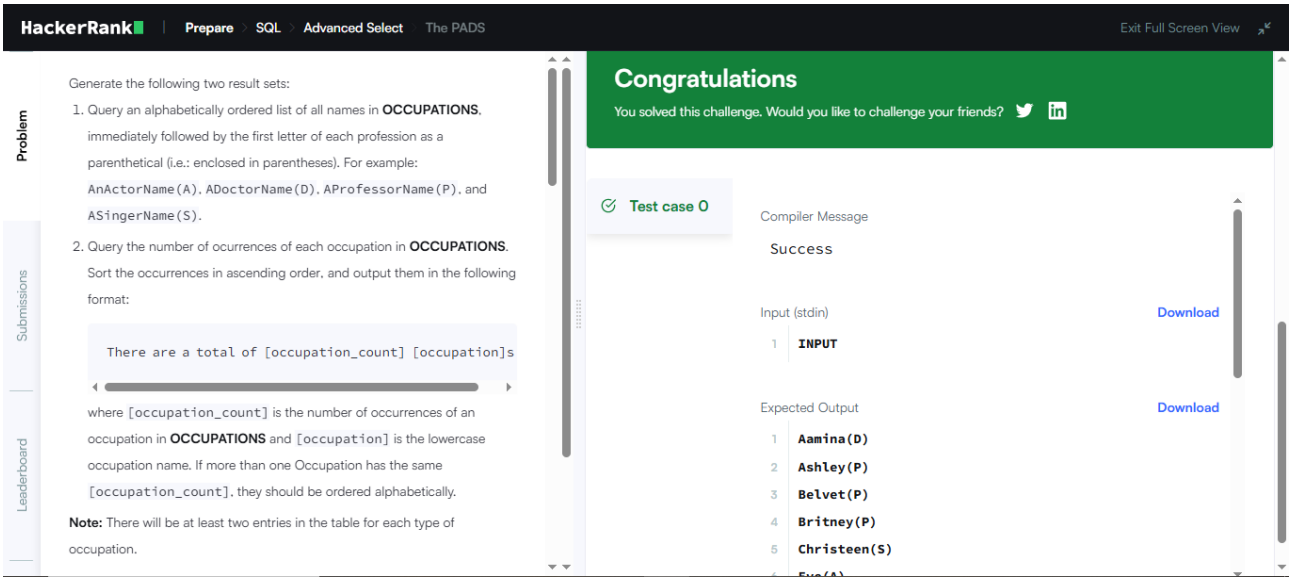
The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name;
SELECT CONCAT('There are a total of ', COUNT(Occupation), ' ', LOWER(Occupation), 's.')
FROM OCCUPATIONS
```

GROUP BY Occupation

ORDER BY COUNT(Occupation), Occupation;



OCCUPATIONS

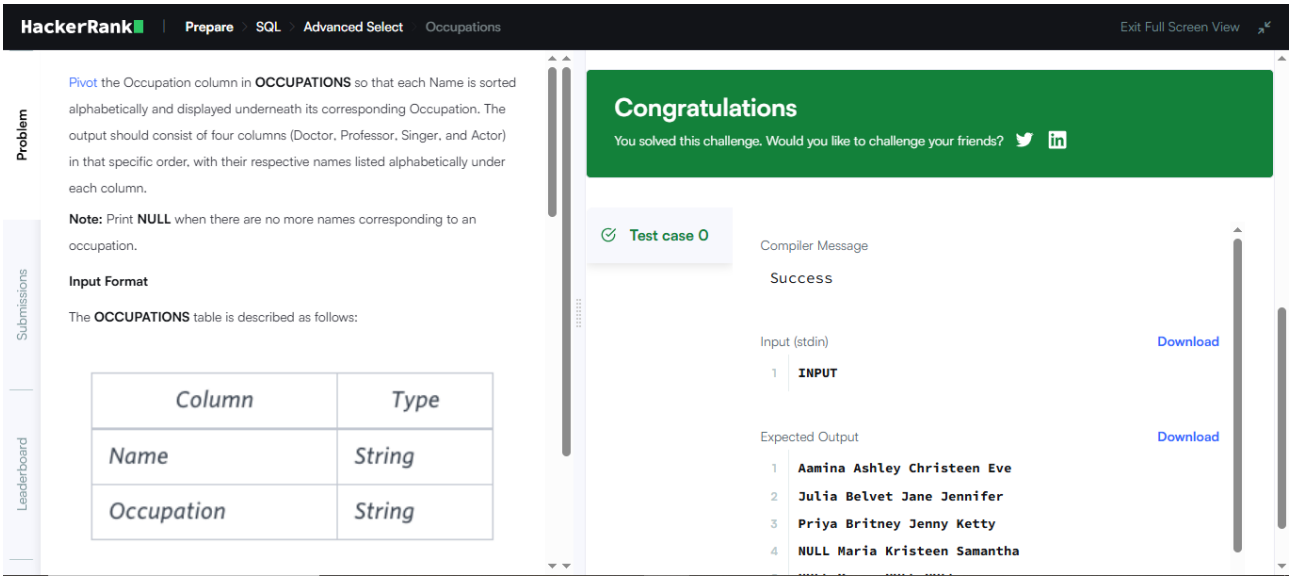
The *Occupation* column in OCCUPATIONS so that each *Name* is sorted alphabetically and displayed underneath its corresponding *Occupation*. The output should consist of four columns (*Doctor*, *Professor*, *Singer*, and *Actor*) in that specific order, with their respective names listed alphabetically under each column.

Input Format

The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

```
SET @r1=0, @r2=0, @r3=0, @r4=0;
SELECT MIN(Doctor), MIN(Professor), MIN(Singer), MIN(Actor)
FROM (
   SELECT
      CASE
         WHEN Occupation = 'Doctor' THEN (@r1 := @r1 + 1)
         WHEN Occupation = 'Professor' THEN (@r2 := @r2 + 1)
         WHEN Occupation = 'Singer' THEN (@r3 := @r3 + 1)
         WHEN Occupation = 'Actor' THEN (@r4 := @r4 + 1)
      END AS RowNum,
      CASE WHEN Occupation = 'Doctor' THEN Name END AS Doctor,
      CASE WHEN Occupation = 'Professor' THEN Name END AS Professor,
      CASE WHEN Occupation = 'Singer' THEN Name END AS Singer,
      CASE WHEN Occupation = 'Actor' THEN Name END AS Actor
   FROM OCCUPATIONS
   ORDER BY Name
) AS Temp
GROUP BY RowNum;
```



BINARY TREE NODES

You are given a table, *BST*, containing two columns: *N* and *P,* where *N* represents the value of a node in *Binary Tree*, and *P* is the parent of *N*

Column	Type
N	Integer
P	Integer

.Write a query to find the node type of *Binary Tree* ordered by the value of the node. Output one of the following for each node:

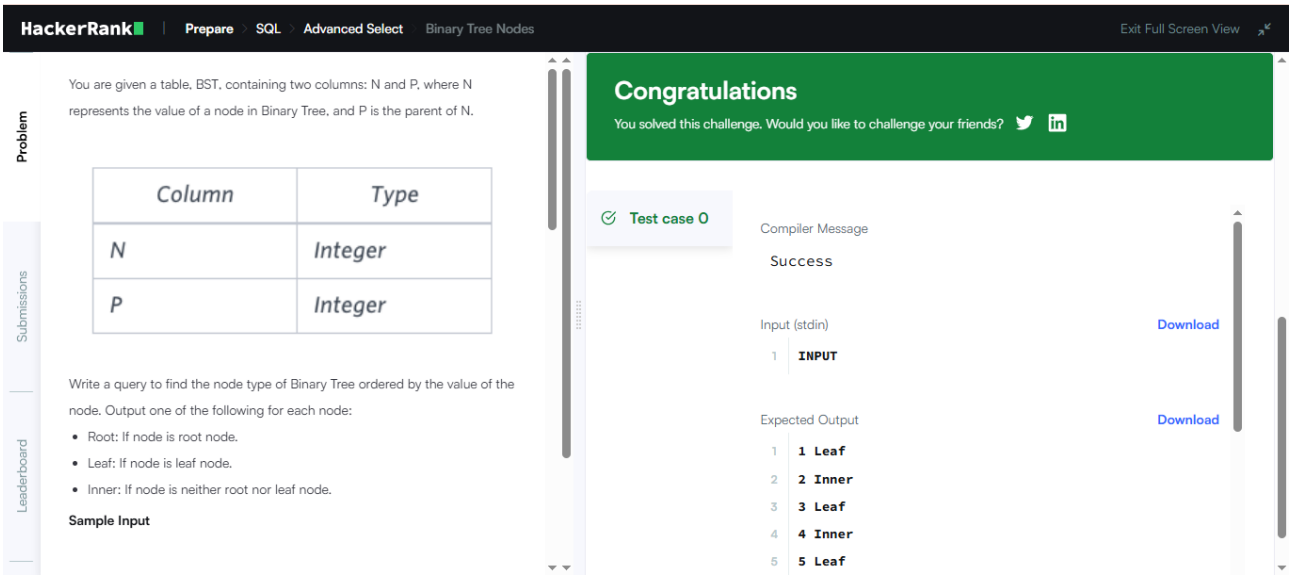
- *Root*: If node is root node.
- *Leaf*: If node is leaf node.
- *Inner*: If node is neither root nor leaf node.

Sample Input

N	P
1	2
3	2
6	8
9	8
2	5
8	5
5	null

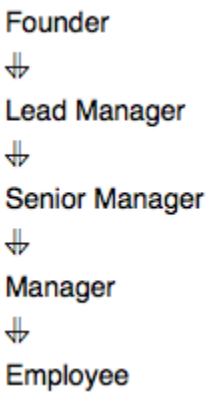
SELECT N, CASE WHEN P IS NULL THEN 'Root' WHEN N IN (SELECT P FROM BST)

THEN 'Inner' ELSE 'Leaf' END FROM BST ORDER BY N;



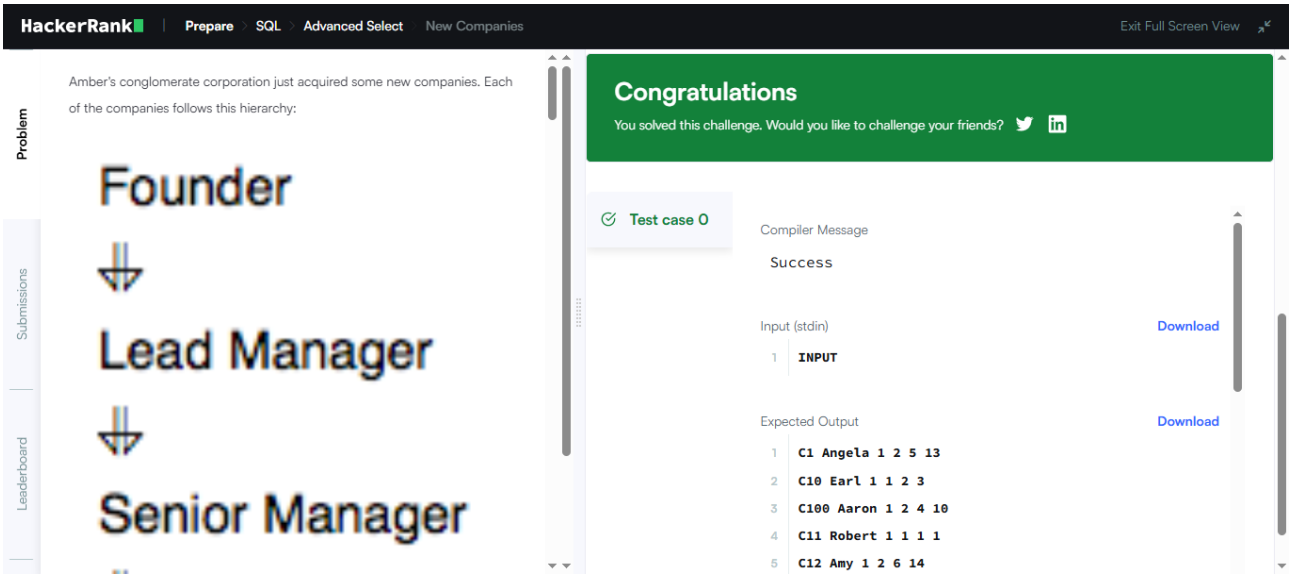
NEW COMPANIES

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:



Given the table schemas below, write a query to print the *company_code*, *founder* name, total number of *lead* managers, total number of *senior* managers, total number of *managers*, and total number of *employees*. Order your output by ascending *company_code*.

```
SELECT c.company_code, c.founder, COUNT(DISTINCT e.lead_manager_code),
COUNT(DISTINCT e.senior_manager_code), COUNT(DISTINCT e.manager_code),
COUNT(DISTINCT e.employee_code) FROM Company c LEFT JOIN Employee e ON
c.company_code = e.company_code GROUP BY c.company_code, c.founder ORDER BY
c.company_code;
```



WEATHER ONSERVATION STATION 18

Consider p1(a,b) and p2(c,d)to be two points on a *2D* plane.

- A happens to equal the minimum value in *Northern Latitude* (*LAT_N* in **STATION**).
- B happens to equal the minimum value in *Western Longitude* (*LONG_W* in **STATION**).
- C happens to equal the maximum value in *Northern Latitude* (*LAT_N* in **STATION**).
- D happens to equal the maximum value in *Western Longitude* (*LONG_W* in **STATION**).

Query the Manhattan Distance between points and and round it to a scale of decimal places.

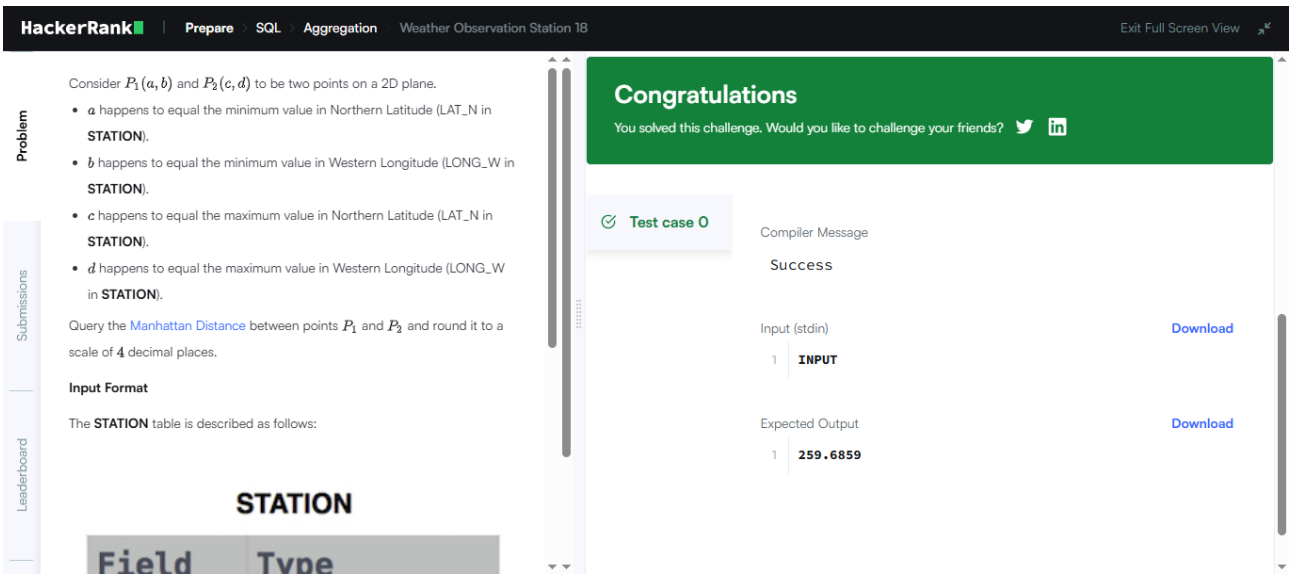
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
SELECT ROUND(ABS(MIN(LAT_N) - MAX(LAT_N)) + ABS(MIN(LONG_W) -
MAX(LONG_W)), 4) FROM STATION;
```



WEATHER ONSERVATION STATION 19

Consider ranto be two points on a 2D plane where Are the respective minimum and maximum values of *Northern Latitude* (*LAT_N*) and are the respective minimum and maximum values of *Western Longitude* (*LONG_W*) in STATION.

Query the between point and *ormat your answer* to display decimal digits.

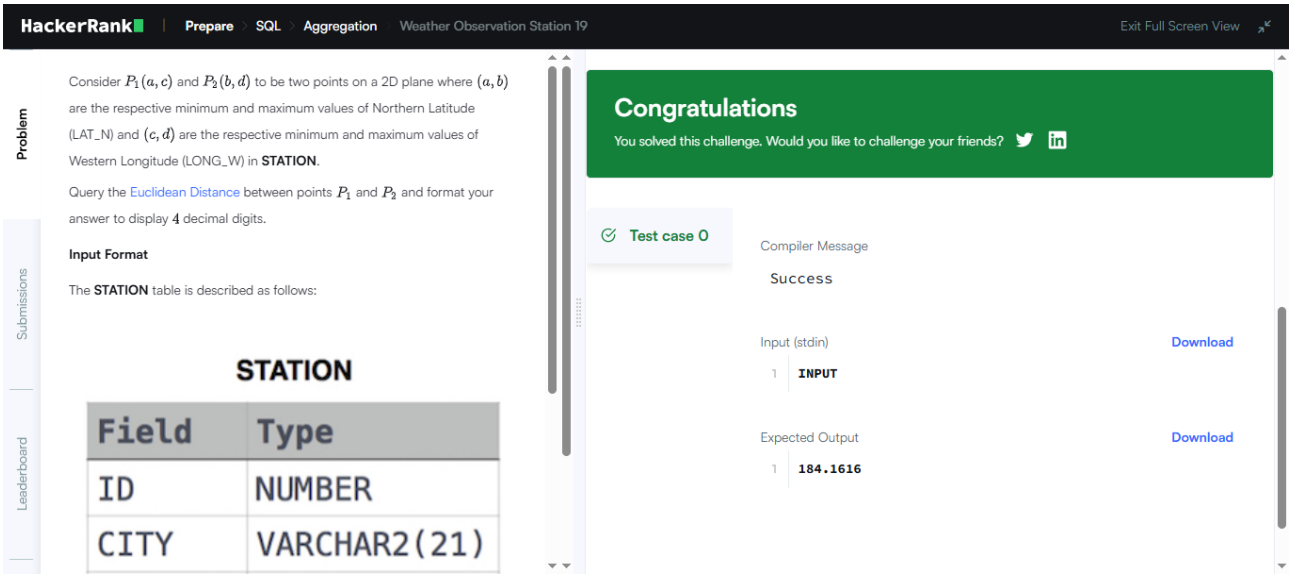
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

SELECT ROUND(SQRT(POW(MIN(LAT_N) - MAX(LAT_N), 2) + POW(MIN(LONG_W) - MAX(LONG_W), 2)), 4) FROM STATION;



WEATHER ONSERVATION STATION 20

A median is defined as a number separating the higher half of a data set from the lower half.

Query the *median* of the *Northern Latitudes* from STATION and round your answer to decimal places.

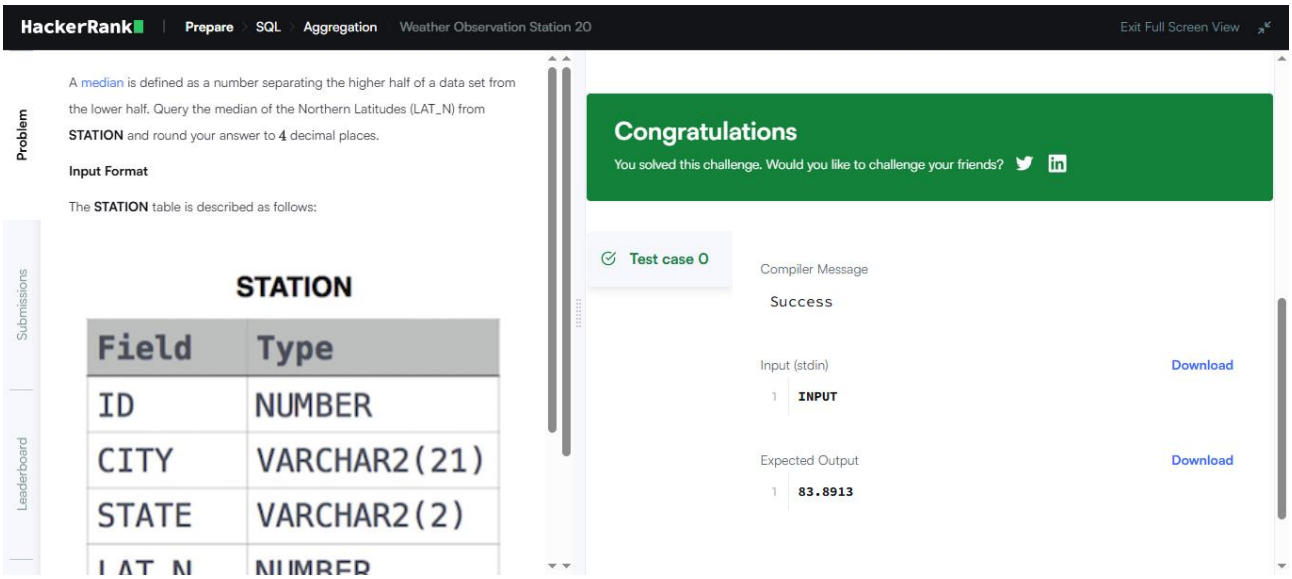
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
SELECT ROUND(AVG(LAT_N), 4)
FROM (
    SELECT LAT_N,
        ROW_NUMBER() OVER (ORDER BY LAT_N) AS RowNum,
        COUNT(*) OVER () AS TotalRows
    FROM STATION
) AS RankedStation
WHERE RowNum IN (FLOOR((TotalRows + 1) / 2), CEIL((TotalRows + 1) / 2));
```



THE REPORT

You are given two tables: *Students* and *Grades*. *Students* contains three columns *ID*, *Name* and *Marks*.

Column	Type
ID	Integer
Name	String
Marks	Integer

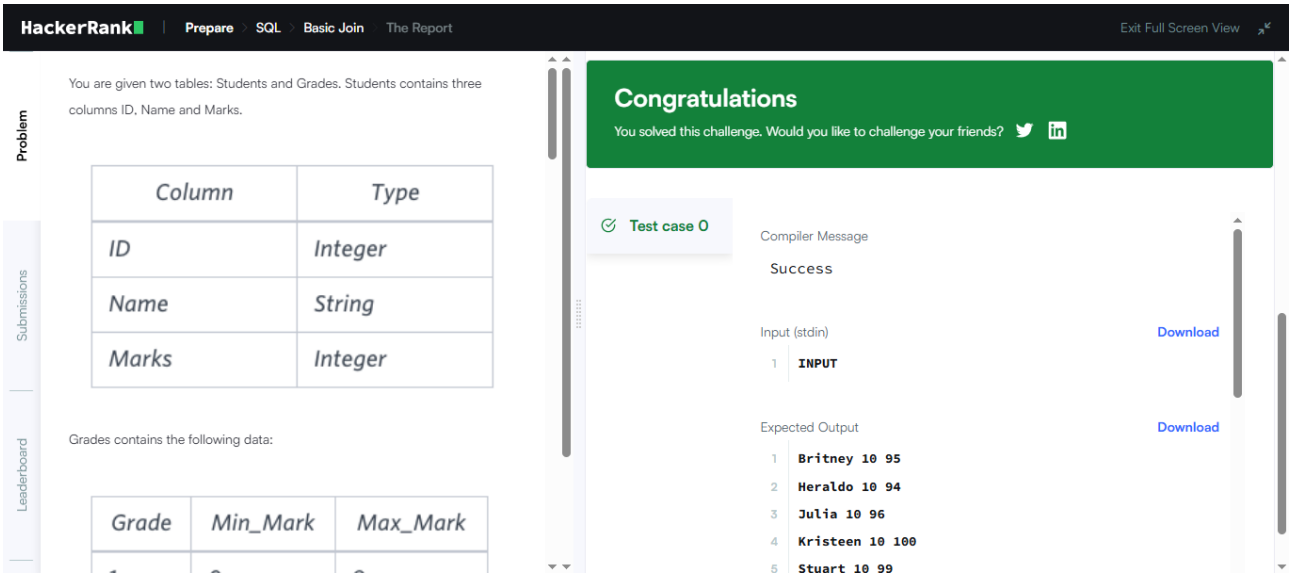
Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives *Eve* a task to generate a report containing three columns: *Name*, *Grade* and *Mark*. *Ketty* doesn't want the NAMES of those students who received a grade lower than *8*. The report must be in descenading order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their

name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

```
SELECT
    CASE WHEN g.Grade >= 8 THEN s.Name ELSE NULL END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
  ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
    CASE WHEN g.Grade >= 8 THEN s.Name END ASC,
    CASE WHEN g.Grade < 8 THEN s.Marks END ASC;
```



TOP COMPETITORS

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective *hacker_id* and *name* of hackers who achieved full scores for *more than one* challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending *hacker_id*.

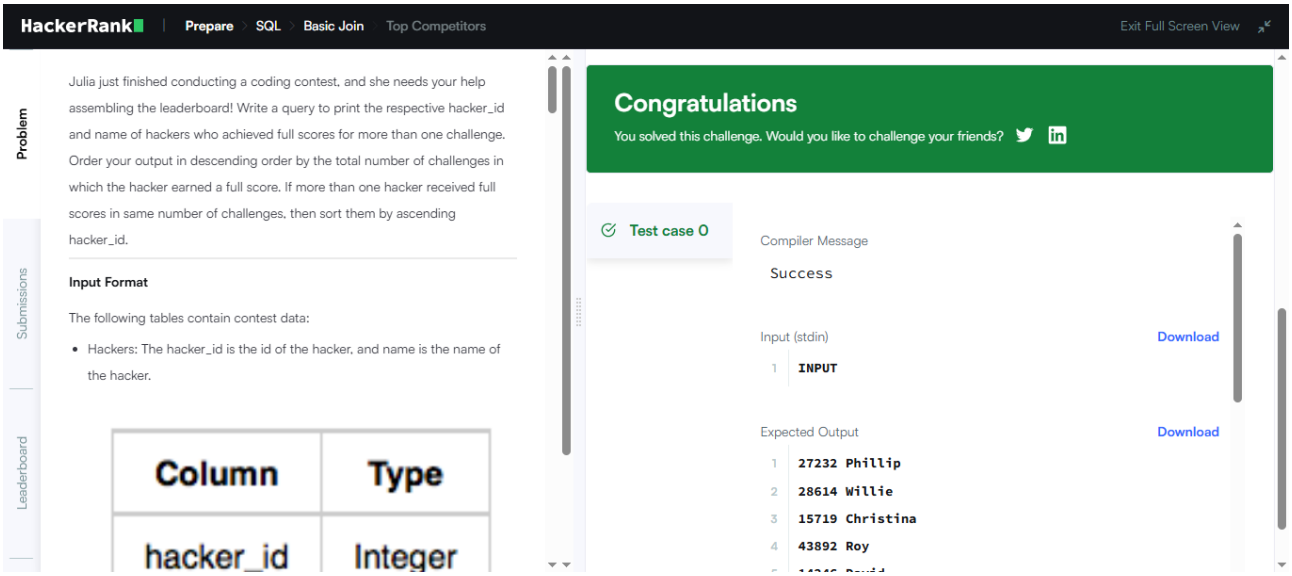
Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
SELECT h.hacker_id, h.name
FROM Submissions s
JOIN Challenges c ON s.challenge_id = c.challenge_id
JOIN Difficulty d ON c.difficulty_level = d.difficulty_level
JOIN Hackers h ON s.hacker_id = h.hacker_id
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```



OLLIVANDER'S INVENTORY

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each *non-evil* wand of high power and age. Write a query to print the *id*, *age*, *coins_needed*, and *power* of

the wands that Ron's interested in, sorted in order of descending *power*. If more than one wand has same power, sort the result in order of descending *age*.

Input Format

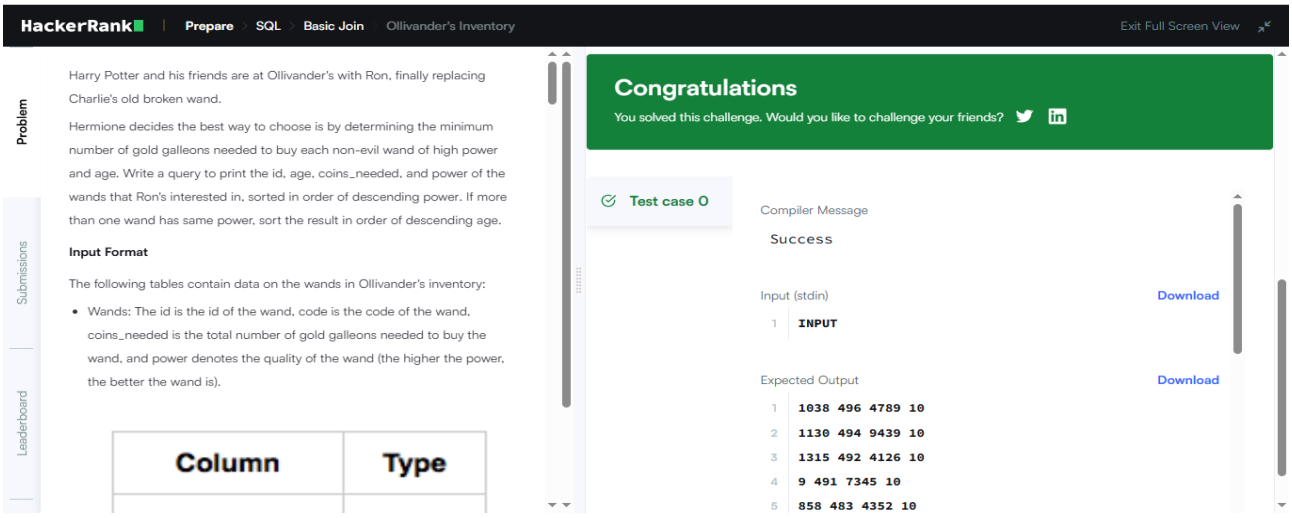
The following tables contain data on the wands in Ollivander's inventory:

- *Wands:* The *id* is the id of the wand, *code* is the code of the wand, *coins_needed* is the total number of gold galleons needed to buy the wand, and *power* denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

```
SELECT w.id, p.age, w.coins_needed, w.power
FROM Wands AS w
JOIN Wands_Property AS p
  ON w.code = p.code
WHERE p.is_evil = 0
  AND w.coins_needed = (
      SELECT MIN(x.coins_needed)
      FROM Wands AS x
      JOIN Wands_Property AS y
        ON x.code = y.code
```

```
    WHERE x.power = w.power
      AND y.age = p.age
  )
ORDER BY w.power DESC, p.age DESC;
```



CHALLENGES

Julia asked her students to create some coding challenges. Write a query to print the *hacker_id*, *name*, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by *hacker_id*. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

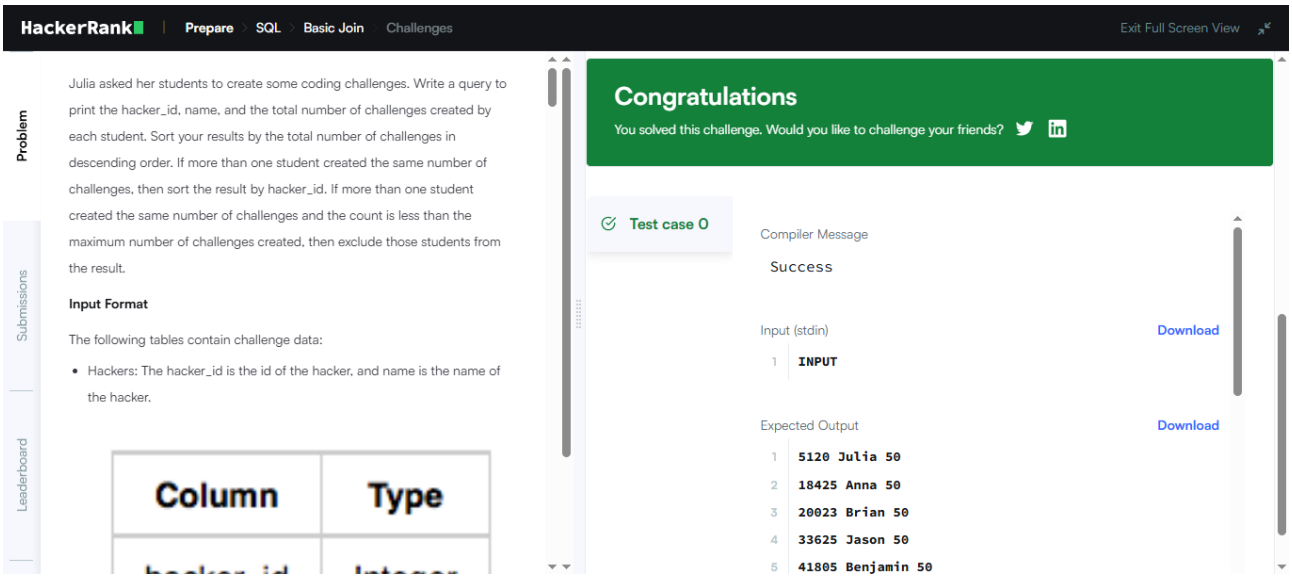
Column	Type
hacker_id	Integer
name	String

```
WITH ChallengeCounts AS (
    SELECT
        h.hacker_id,
        h.name,
        COUNT(c.challenge_id) AS total_challenges
    FROM Hackers h
    JOIN Challenges c ON h.hacker_id = c.hacker_id
    GROUP BY h.hacker_id, h.name
)

SELECT hacker_id, name, total_challenges
FROM ChallengeCounts
WHERE total_challenges = (
    /* Condition 1: Keep it if it matches the maximum challenges created by any hacker */
    SELECT MAX(total_challenges) FROM ChallengeCounts
)
OR total_challenges IN (
    /* Condition 2: Keep it if the challenge count is completely unique */
    SELECT total_challenges
    FROM ChallengeCounts
    GROUP BY total_challenges
    HAVING COUNT(total_challenges) = 1
```

)

ORDER BY total_challenges DESC, hacker_id ASC;



CONTEST LEADERBOARD

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the *hacker_id*, *name*, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending *hacker_id*. Exclude all hackers with a total score of from your result.

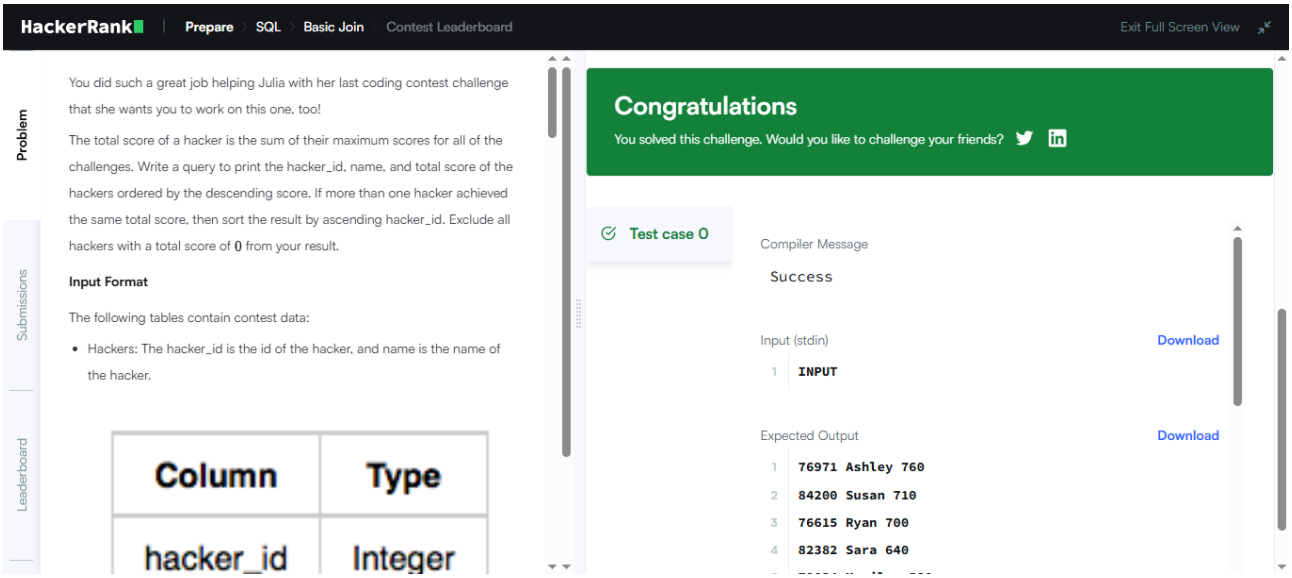
Input Format

The following tables contain contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
SELECT h.hacker_id, h.name, SUM(t.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT hacker_id, challenge_id, MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) t ON h.hacker_id = t.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_score > 0
ORDER BY total_score DESC, h.hacker_id ASC;
```



SQL PROJECT PLANNING

You are given a table, *Projects*, containing three columns: *Task_ID*, *Start_Date* and *End_Date*. It is guaranteed that the difference between the *End_Date* and the *Start_Date* is equal to *1* day for each row in the table.

Column	*Type*
Task_ID	*Integer*
Start_Date	*Date*
End_Date	*Date*

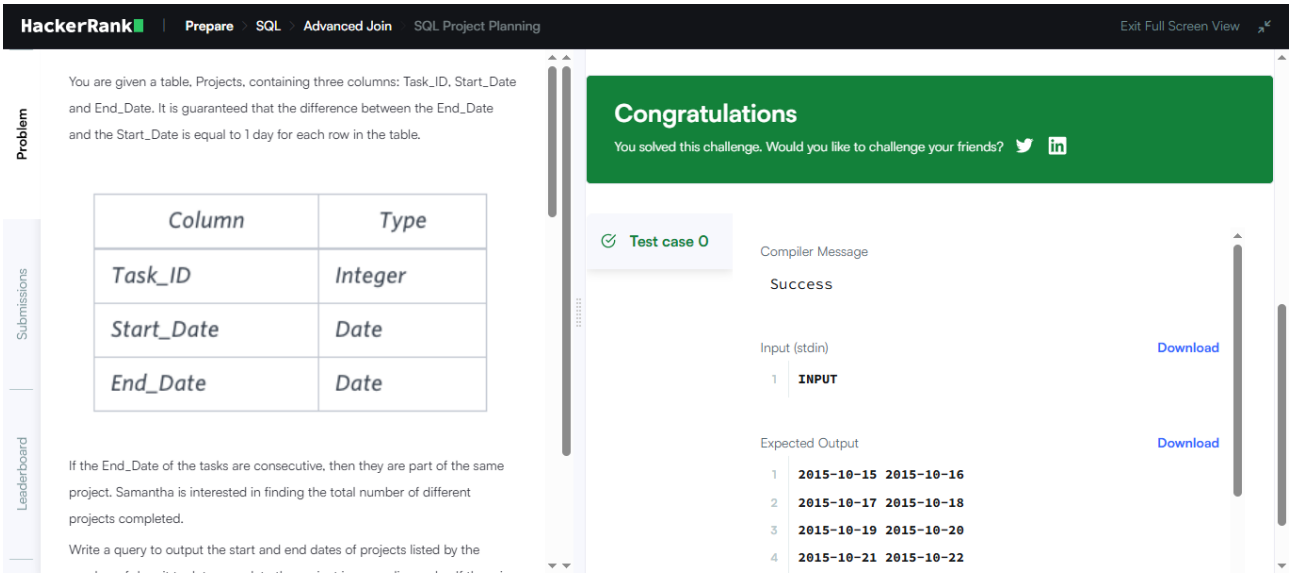
If the *End_Date* of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

```
SELECT A.Start_Date, MIN(B.End_Date)
FROM
    (SELECT Start_Date
    FROM Projects
    WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)) AS A
    INNER JOIN
    (SELECT End_Date
    FROM Projects
    WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)) AS B
WHERE A.Start_Date < B.End_Date
GROUP BY A.Start_Date
ORDER BY MIN(B.End_Date) - A.Start_Date ASC, A.Start_Date;
```



PLACEMENTS

You are given three tables: *Students*, *Friends* and *Packages*. *Students* contains two columns: *ID* and *Name*. *Friends* contains two columns: *ID* and *Friend_ID* (*ID* of the ONLY best friend). *Packages* contains two columns: *ID* and *Salary* (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

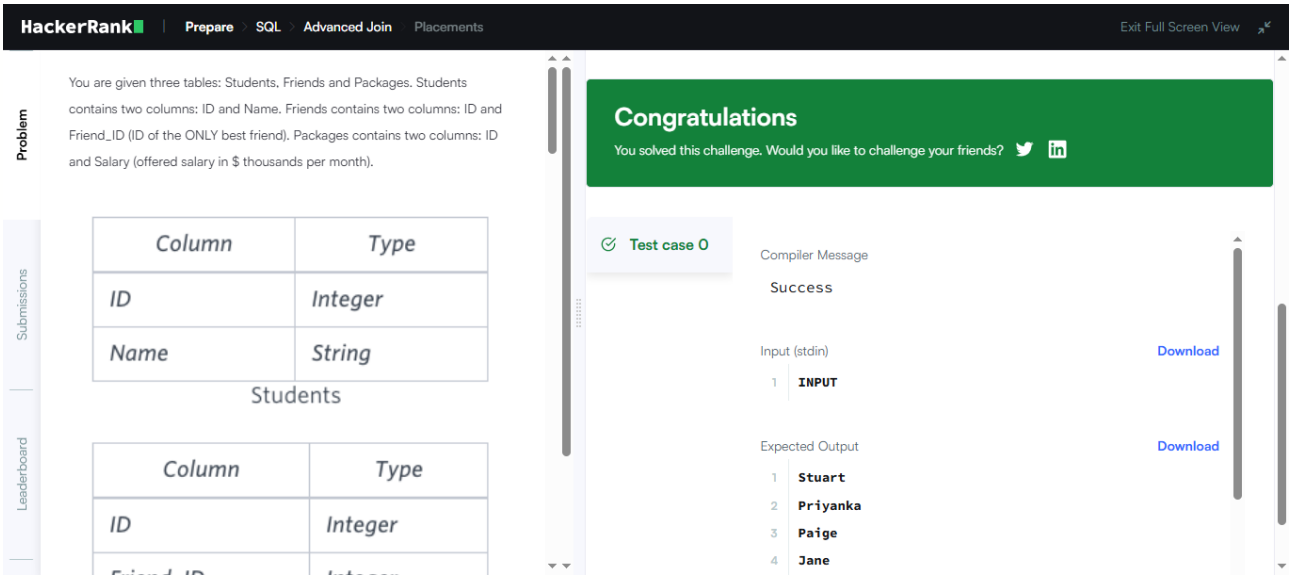
Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

```
SELECT s.Name
FROM Students s
JOIN Friends f ON s.ID = f.ID
JOIN Packages p1 ON s.ID = p1.ID
JOIN Packages p2 ON f.Friend_ID = p2.ID
WHERE p2.Salary > p1.Salary
ORDER BY p2.Salary;
```



SYMMETRIC PAIRS

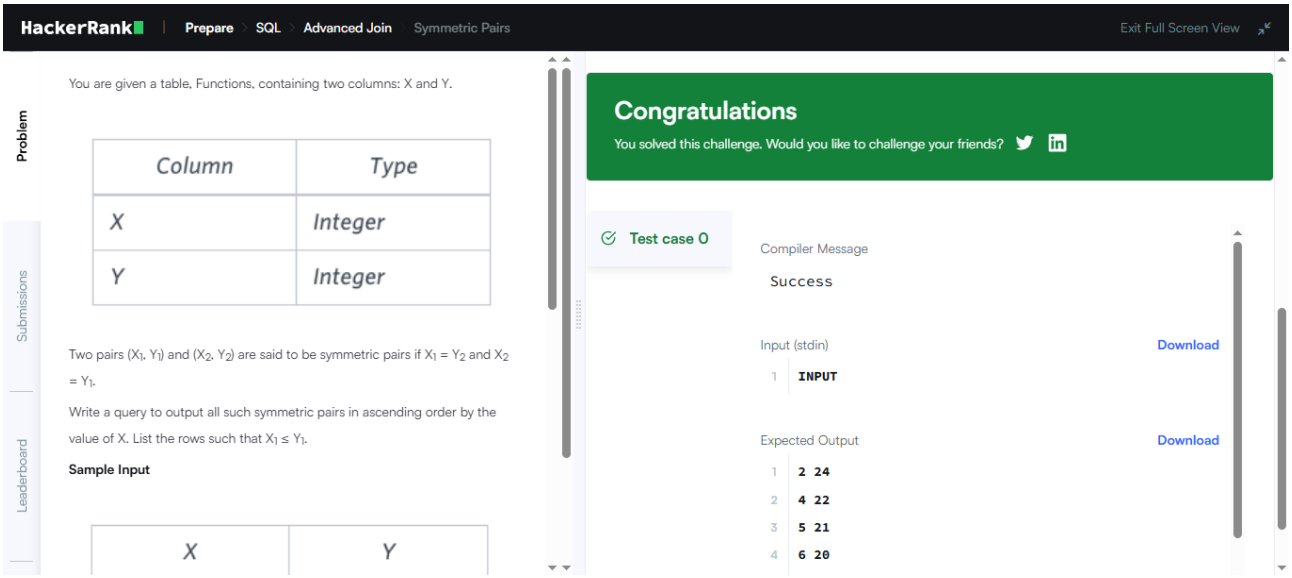
You are given a table, *Functions*, containing two columns: *X* and *Y*.

Column	Type
X	Integer
Y	Integer

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be *symmetric pairs* if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such *symmetric pairs* in ascending order by the value of *X*. List the rows such that $X_1 \leq Y_1$.

```
SELECT f1.X, f1.Y
FROM Functions f1
JOIN Functions f2
    ON f1.X = f2.Y AND f1.Y = f2.X
GROUP BY f1.X, f1.Y
HAVING f1.X < f1.Y OR COUNT(f1.X) > 1
ORDER BY f1.X;
```



PRINT PRIME NUMBERS

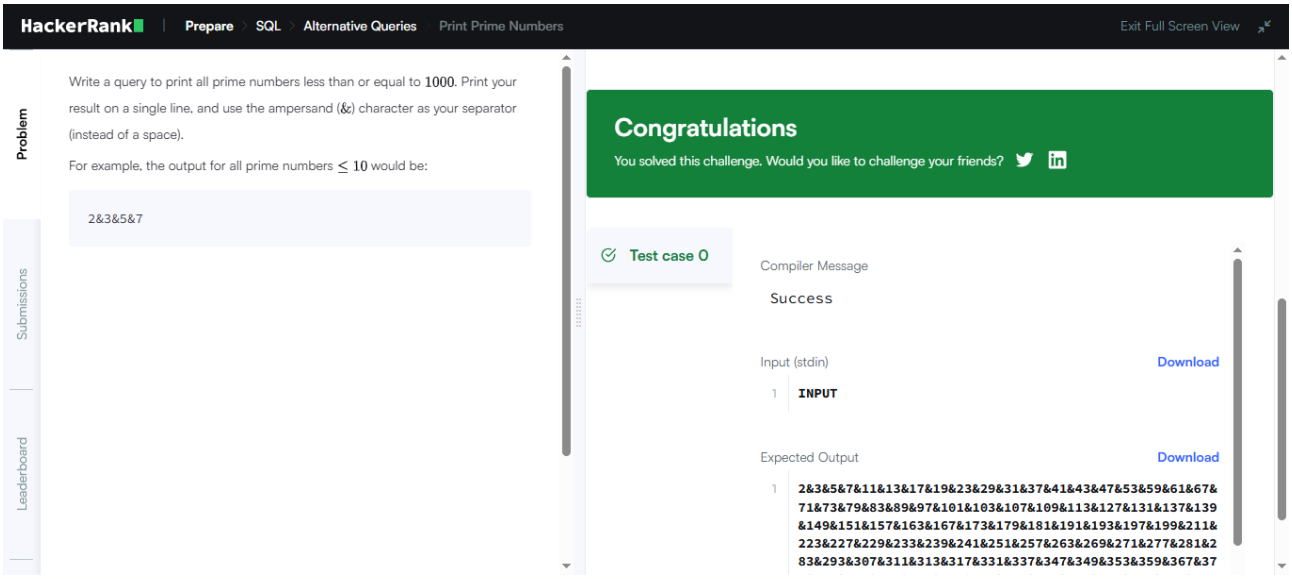
Write a query to print all *prime numbers* less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

For example, the output for all prime numbers would be:

```
2&3&5&7
```

```
WITH RECURSIVE Numbers AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n + 1 FROM Numbers WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
```

```
FROM Numbers n1
WHERE NOT EXISTS (
    SELECT 1
    FROM Numbers n2
    WHERE n2.n < n1.n AND n1.n % n2.n = 0
);
```



INTERVIEWS

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the *contest_id*, *hacker_id*, *name*, and the sums of *total_submissions*, *total_accepted_submissions*, *total_views*, and *total_unique_views* for each contest sorted by *contest_id*. Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- *Contests:* The *contest_id* is the id of the contest, *hacker_id* is the id of the hacker who created the contest, and *name* is the name of the hacker.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- *Colleges:* The *college_id* is the id of the college, and *contest_id* is the id of the contest that Samantha used to screen the candidates.

Column	Type
college_id	Integer
contest_id	Integer

- *Challenges:* The *challenge_id* is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and *college_id* is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

- *View_Stats:* The *challenge_id* is the id of the challenge, *total_views* is the number of times the challenge was viewed by candidates, and *total_unique_views* is the number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- *Submission_Stats:* The *challenge_id* is the id of the challenge, *total_submissions* is the number of submissions for the challenge, and *total_accepted_submission* is the number of submissions that achieved full scores.

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Contests Table:

contest_id	hacker_id	name
66406	17973	Rose
66556	79153	Angela
94828	80275	Frank

Colleges Table:

college_id	contest_id
11219	66406
32473	66556
56685	94828

Challenges Table:

challenge_id	college_id
18765	11219
47127	11219
60292	32473
72974	56685

View_Stats Table:

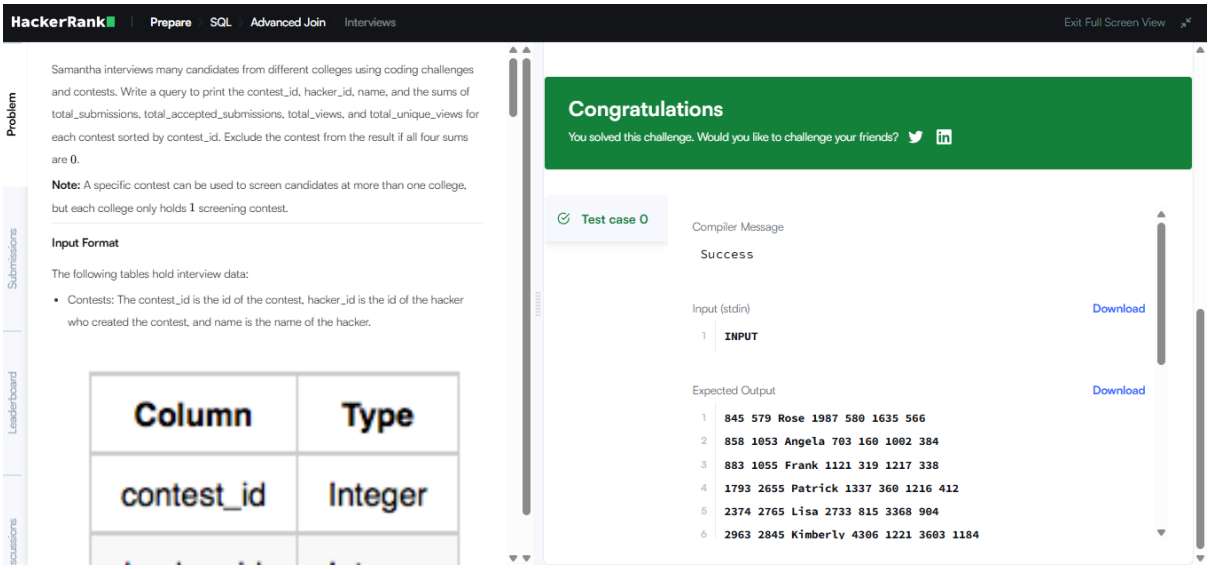
challenge_id	total_views	total_unique_views
47127	26	19
47127	15	14
18765	43	10
18765	72	13
75516	35	17
60292	11	10
72974	41	15
75516	75	11

Submission_Stats Table:

challenge_id	total_submissions	total_accepted_submissions
75516	34	12
47127	27	10
47127	56	18
75516	74	12
75516	83	8
72974	68	24
72974	82	14
47127	28	11

```
SELECT
    c.contest_id,
    c.hacker_id,
    c.name,
    SUM(COALESCE(s.total_submissions, 0)) AS total_submissions,
    SUM(COALESCE(s.total_accepted_submissions, 0)) AS total_accepted_submissions,
    SUM(COALESCE(v.total_views, 0)) AS total_views,
    SUM(COALESCE(v.total_unique_views, 0)) AS total_unique_views
FROM Contests c
JOIN Colleges col
    ON c.contest_id = col.contest_id
JOIN Challenges ch
    ON col.college_id = ch.college_id
LEFT JOIN (
    SELECT
```

```
        challenge_id,
        SUM(total_submissions) AS total_submissions,
        SUM(total_accepted_submissions) AS total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) s
ON ch.challenge_id = s.challenge_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_views) AS total_views,
        SUM(total_unique_views) AS total_unique_views
    FROM View_Stats
    GROUP BY challenge_id
) v
ON ch.challenge_id = v.challenge_id
GROUP BY
    c.contest_id,
    c.hacker_id,
    c.name
HAVING
    SUM(COALESCE(s.total_submissions, 0)) +
    SUM(COALESCE(s.total_accepted_submissions, 0)) +
    SUM(COALESCE(v.total_views, 0)) +
    SUM(COALESCE(v.total_unique_views, 0)) > 0
ORDER BY
    c.contest_id;
```



15 DAYS OF LEARNING SQL

Julia conducted a days of learning SQL contest. The start date of the contest was *March 01, 2016* and the end date was *March 15, 2016*.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the *hacker_id* and *name* of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest *hacker_id*. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- *Hackers:* The *hacker_id* is the id of the hacker, and *name* is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- *Submissions:* The *submission_date* is the date of the submission, *submission_id* is the id of the submission, *hacker_id* is the id of the hacker who made the submission, and *score* is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was *March 06, 2016*.

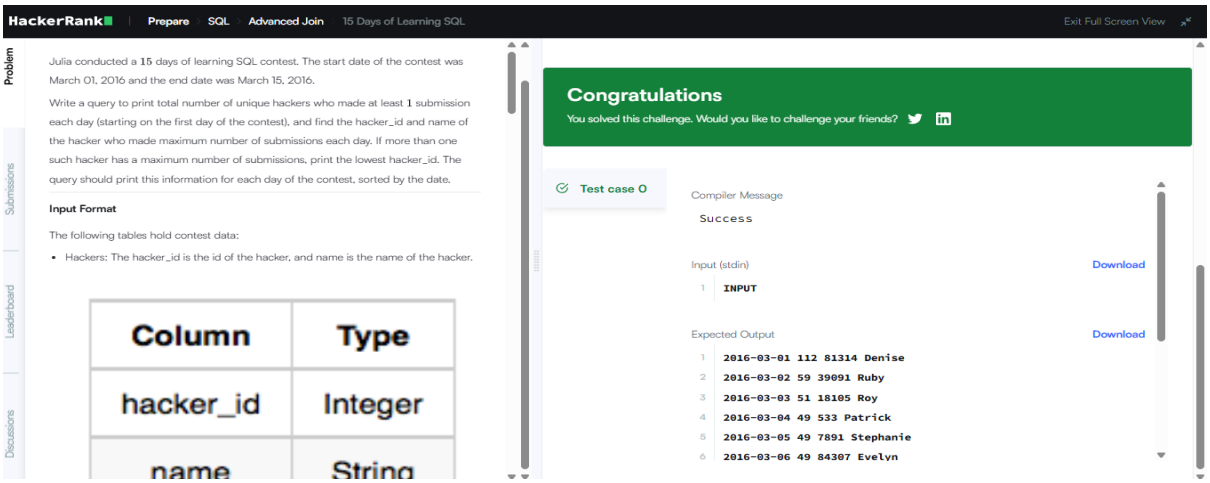
Hackers Table:

hacker_id	name
15758	Rose
20703	Angela
36396	Frank
38289	Patrick
44065	Lisa
53473	Kimberly
62529	Bonnie
79722	Michael

Submissions **Table:**

submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

```
SELECT
    s.submission_date,
    (
        SELECT COUNT(DISTINCT hacker_id)
        FROM Submissions s2
        WHERE s2.submission_date <= s.submission_date
        AND (
            SELECT COUNT(DISTINCT s3.submission_date)
            FROM Submissions s3
            WHERE s3.hacker_id = s2.hacker_id
            AND s3.submission_date <= s.submission_date
        ) = DATEDIFF(s.submission_date, '2016-03-01') + 1
    ) AS total_hackers,
    s.hacker_id,
    h.name
FROM Submissions s
JOIN Hackers h
ON s.hacker_id = h.hacker_id
WHERE s.hacker_id = (
    SELECT s1.hacker_id
    FROM Submissions s1
    WHERE s1.submission_date = s.submission_date
    GROUP BY s1.hacker_id
    ORDER BY COUNT(*) DESC, s1.hacker_id
    LIMIT 1
)
GROUP BY s.submission_date, s.hacker_id, h.name
ORDER BY s.submission_date;
```


HackerRank
Prepare
SQL
Advanced Join
15 Days of Learning SQL
Exit Full Screen View
Problem
Submissions
Leaderboard
Discussions
Julia conducted a 15 days of learning SQL contest. The start date of the contest was March 01, 2016 and the end date was March 15, 2016.
Write a query to print total number of unique hackers who made at least 1 submission each day (starting on the first day of the contest), and find the hacker_id and name of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest hacker_id. The query should print this information for each day of the contest, sorted by the date.
Input Format
The following tables hold contest data:
Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.
Column
Type
hacker_id
Integer
name
String
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 2016-03-01 112 81314 Denise
2 2016-03-02 59 39091 Ruby
3 2016-03-03 51 18105 Roy
4 2016-03-04 49 533 Patrick
5 2016-03-05 49 7891 Stephanie
6 2016-03-06 49 84307 Evelyn

CODECHEF

BASIC COMMANDS

1. ALL PRODUCTS

Task

Write a query to output Products table.

Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT

select*from products;

OUTPUT

Your Output



product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture

✓ Excellent work!

2. HIGH PRICE OF PRODUCTS

Task

Write a query to find all product_name and category that have a price greater than 100.00 from the **Products** table.

Table name: **Products**

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT

SELECT product_name, category FROM Products WHERE price > 100.00;

OUTPUT

Your Output

product_name	category
Laptop	Electronics
Desk Chair	Furniture
Smartphone	Electronics
Standing Desk	Furniture
Tablet	Electronics

✔ Excellent work!

3. AVERAGE SALARY

Task

- Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

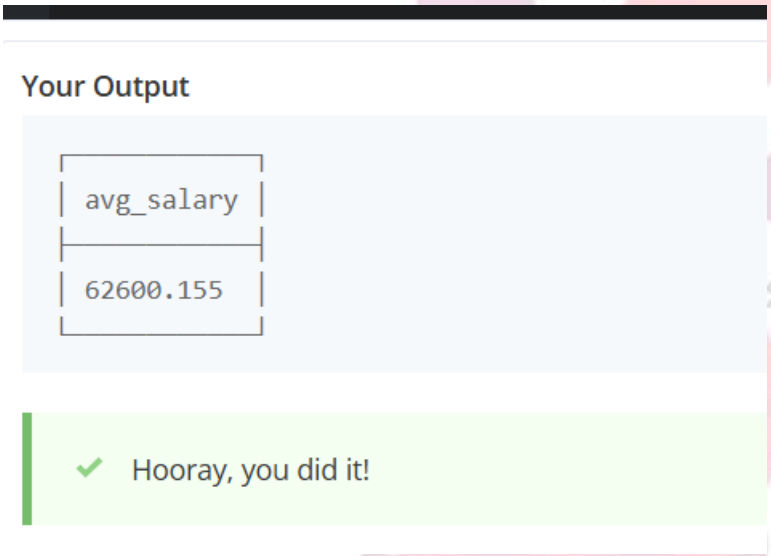
employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00

6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

INPUT

SELECT AVG(salary) AS avg_salary FROM Works;

OUTPUT



4. LOCATE PEOPLE

Task

- Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

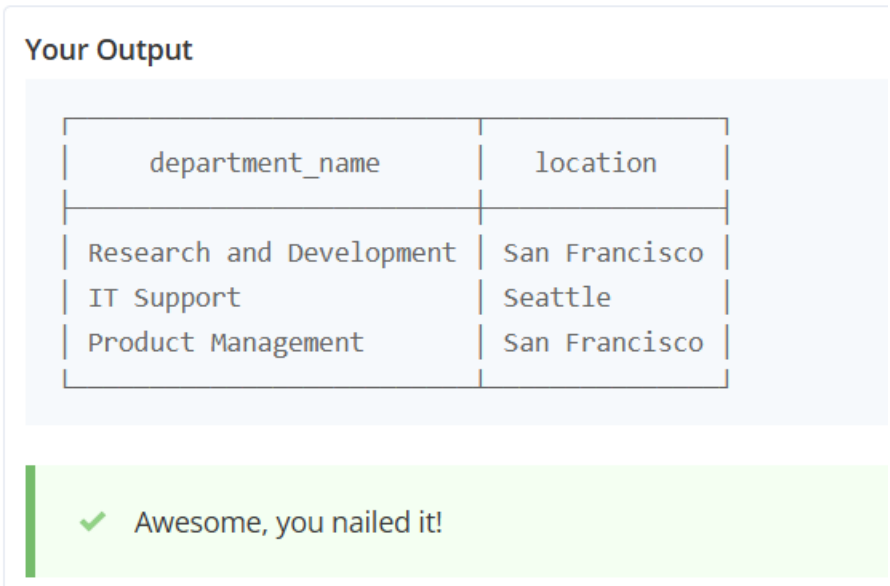
Table name: **departments**

department_id	department_name	location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Boston

INPUT

SELECT department_name, location FROM departments WHERE location LIKE 'S%';

OUTPUT



5. DISTINCT COMPANIES

Task

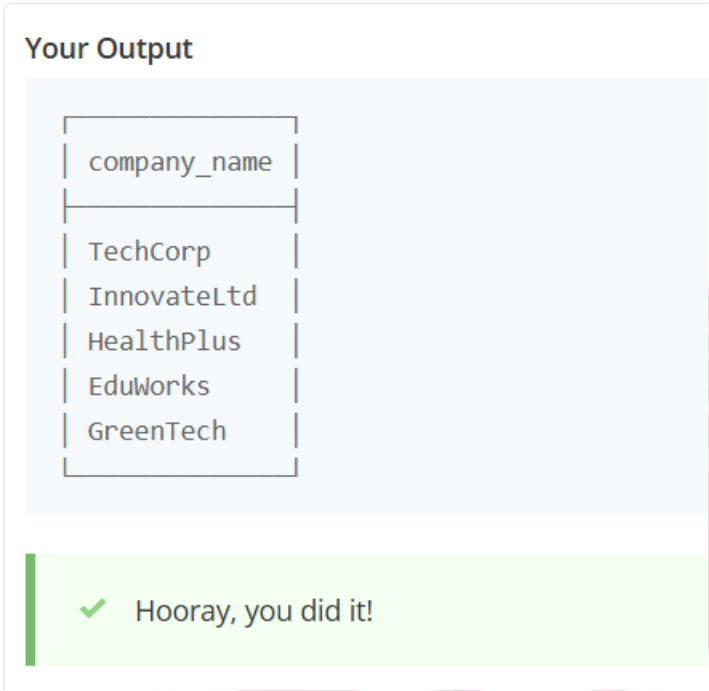
Write a query to select all the distinct companies (company_name) in the Works table.
Table name: **Works**

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

INPUT

```
SELECT DISTINCT company_name FROM Works;
```

OUTPUT



6. FICTION COLLECTION SIZE

Task

Write a query to find the total count of books whose genre is **Fiction**.
Note: Output column name should be **fiction_count**.

Table name: **Books**

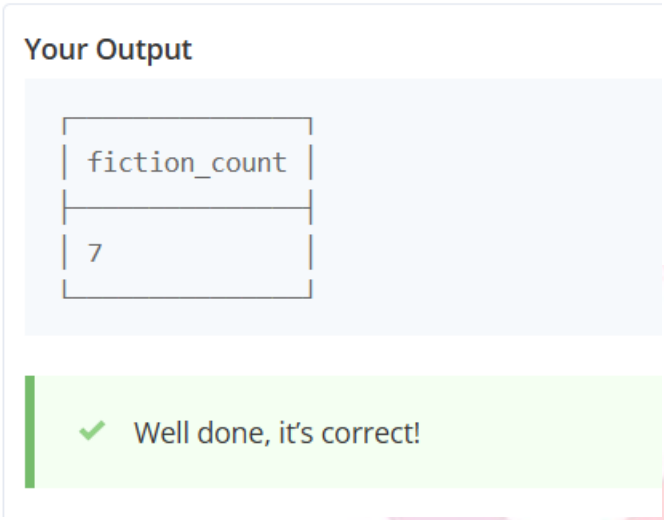
id	title	author	genre	price	published_year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951

5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985
14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

INPUT

SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';

OUTPUT



7. LIST OF MOVIES WITH RATINGS

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

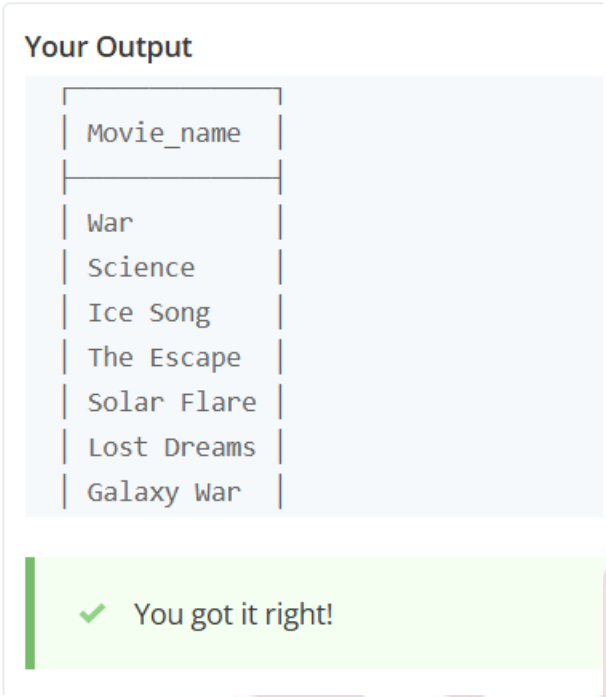
Table: **Cinema**

Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

INPUT

```
SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0;
```

OUTPUT



8. HANDLING NULL VALUES

Task

Write a query to retrieve book_id, title, author and published_year of the books which have **NULL** rating for their books.

Table name: Library

book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

INPUT

select book_id, title, author, published_year from library where rating is null;

OUTPUT

Your Output

book_id	title	author	published_year
2	To Kill a Mockingbird	Harper Lee	1960
4	The Catcher in the Rye	J.D. Salinger	1951

✔ Excellent work!

9. SALARY OF EMPLOYEES

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in **descending** order

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000

6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

INPUT

SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time' ORDER BY salary DESC;

OUTPUT

Your Output

employee_name	company	salary
Bob Brown	FinServ	90000
David Green	TechCorp	85000
John Smith	TechCorp	80000
Grace Grey	TechCorp	70000
Frank Black	HealthPlus	60000

✔ You got it right!

10. DEPARTMENT OF EACH EMPLOYEE

Task

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

Table name: Employees

employee_id	employee_name	company	category	department	salary

1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

INPUT

select department, count(*) as total_employees from employees group by department;

OUTPUT

Your Output

department	total_employees
Engineering	2
Finance	2
HR	2
IT	1
Marketing	2
Sales	1

✔ Perfect answer!

11. ARTICLE VIEWS

Task

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in **ascending order**.

Table name: Views

article_id	author_id	author_name	viewer_id	view_count	publication_name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

INPUT

select distinct author_id, author_name, publication_name from views where view_count = 0 order by author_id asc;

OUTPUT



12. PLAYER PERFORMANCE INSIGHTS

Task

Write a query to find the names of the **top 3 distinct players** by highest score who have **won** matches, including their **scores**.

Table 1: **Players**

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: **Matches**

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

INPUT

select distinct player_name, score from players join matches where winner=player_name

order by score desc limit 3;

OUTPUT

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

Awesome, you nailed it!

13 . PLAYER DETAILS

Task

Write a query to retrieve the details of the **last five matches** played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **Players** and **Matches**.

Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

INPUT

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score

FROM Matches m

JOIN Players p ON m.winner = p.player_name

ORDER BY m.match_date DESC

LIMIT 5;
```

OUTPUT

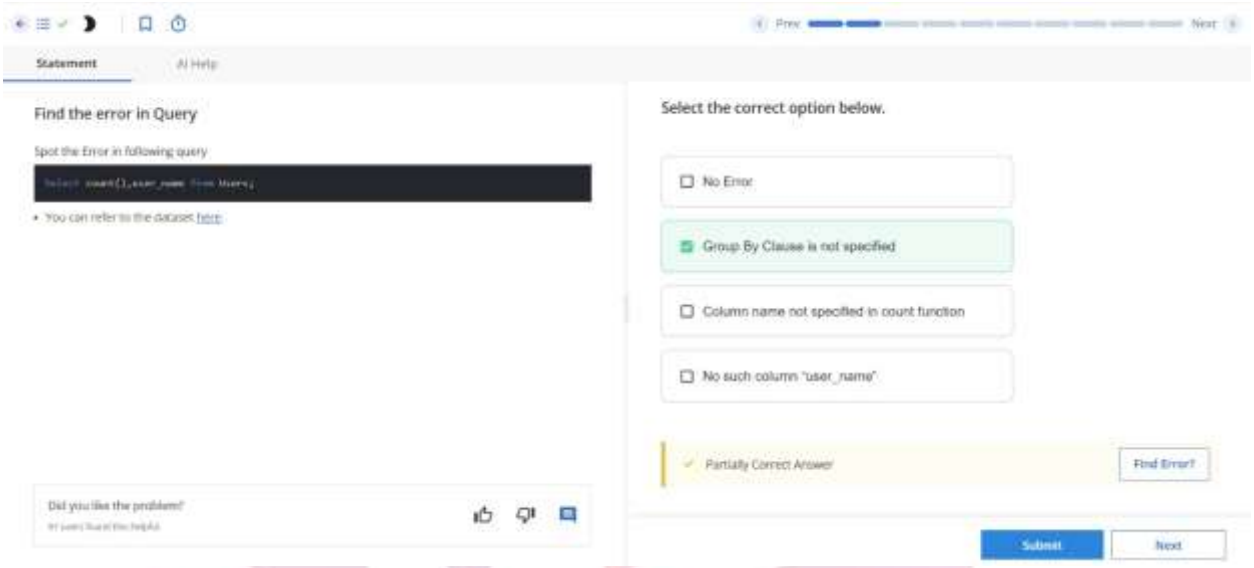
Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

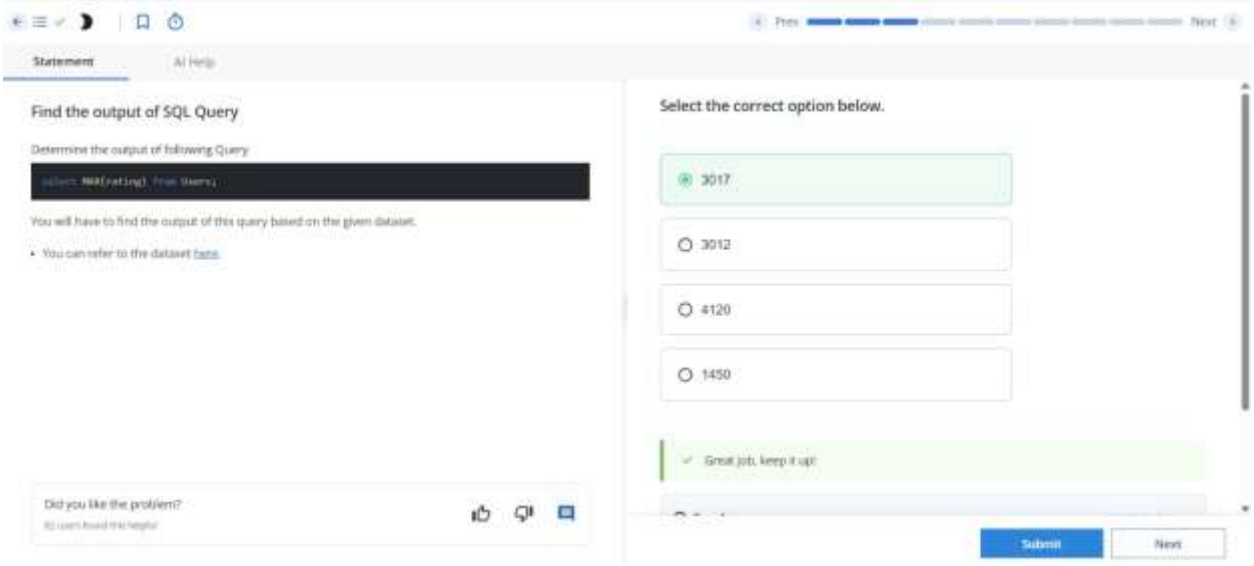
✔ Awesome, you nailed it!

CASE STUDY – CODECHEF DATABASE

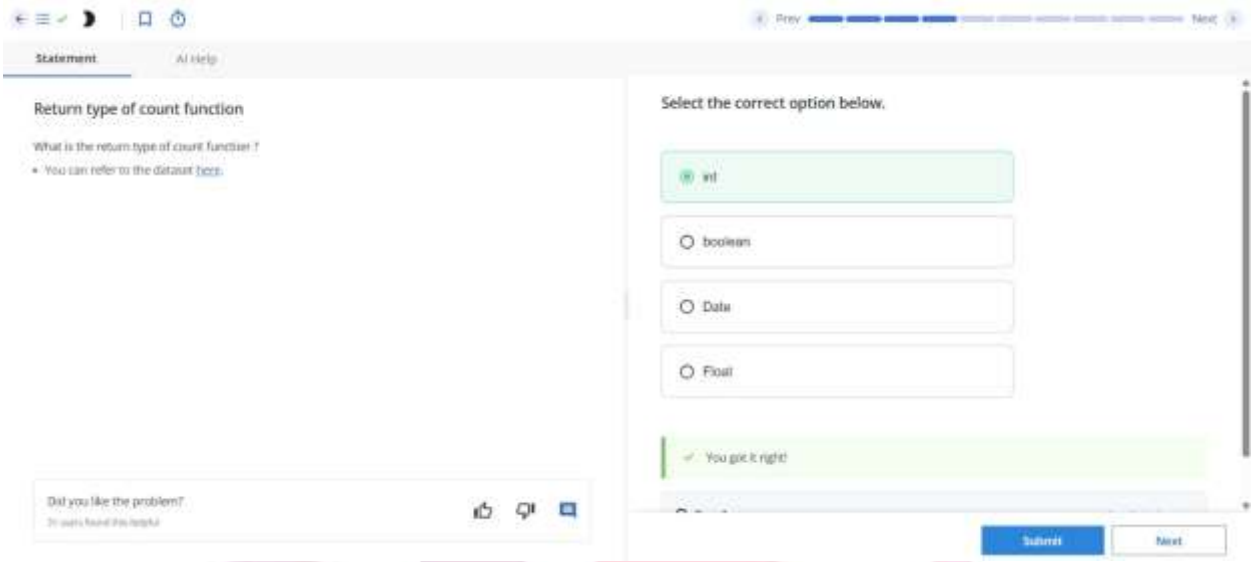
1.FIND THE ERROR IN QUERY



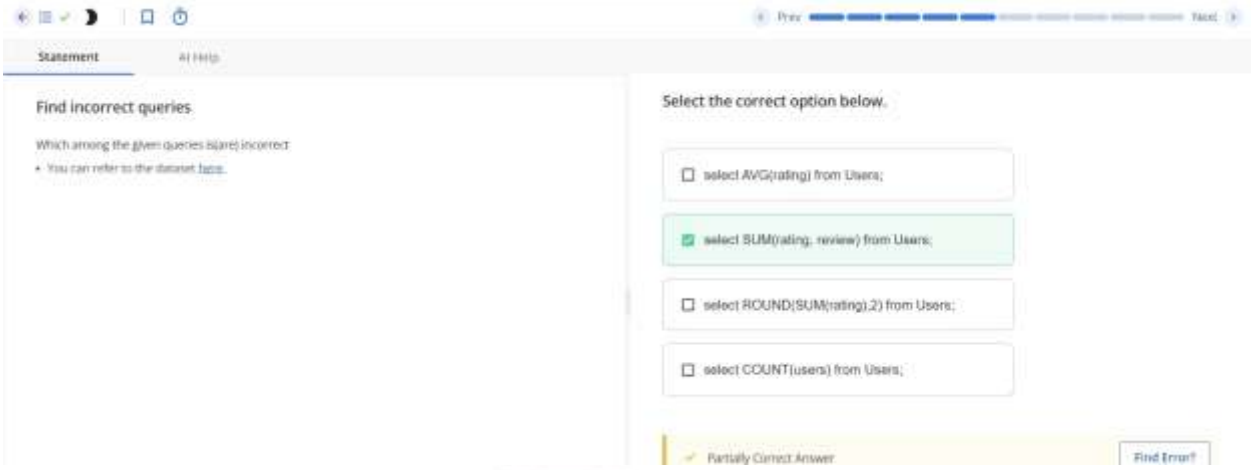
2. FIND THE OUTPUT OF SQL QUERY



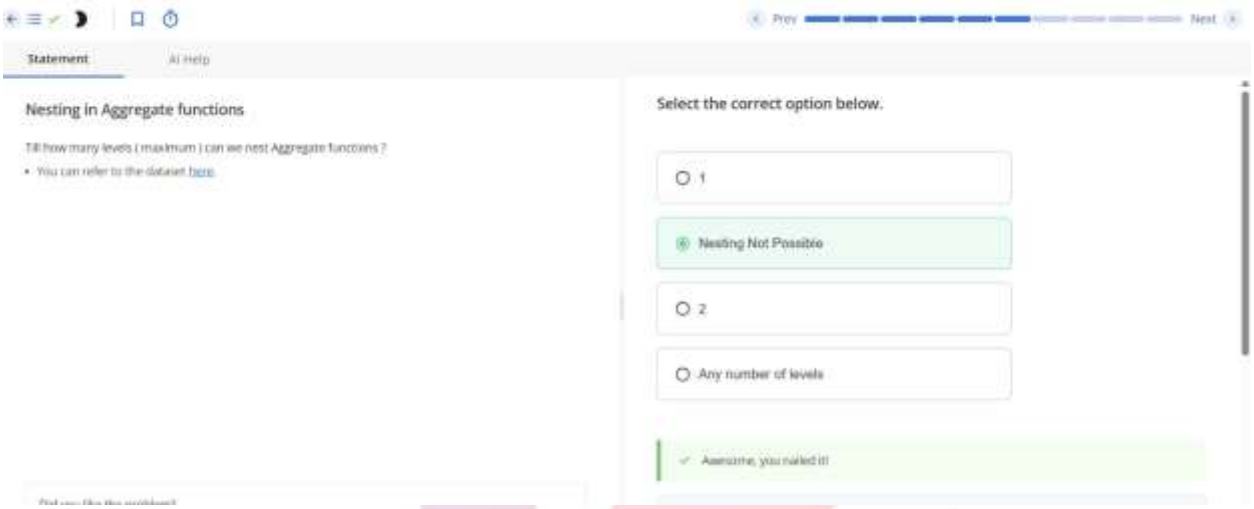
3. RETURN TYPE OF COUNT FUNCTION



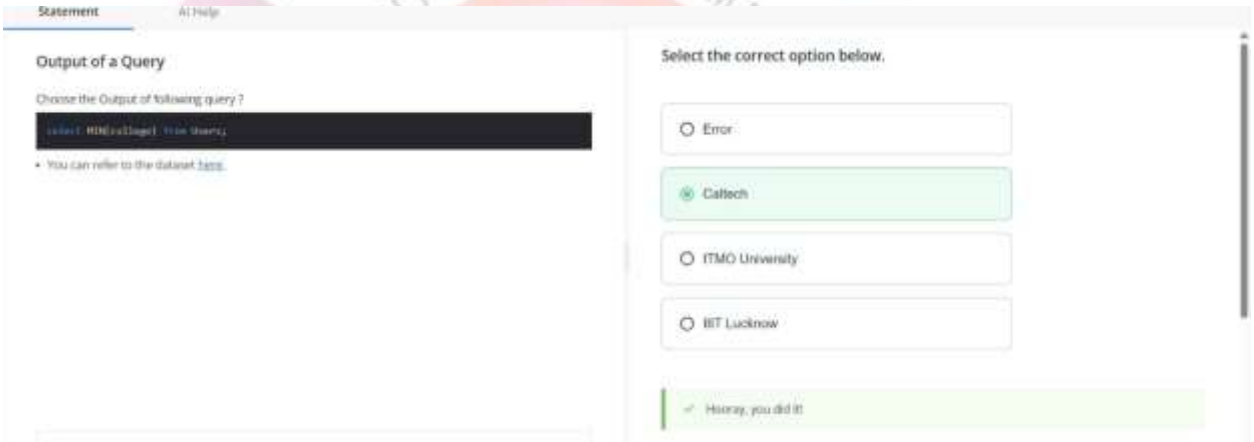
4. FIND INCORRECT QUERIES



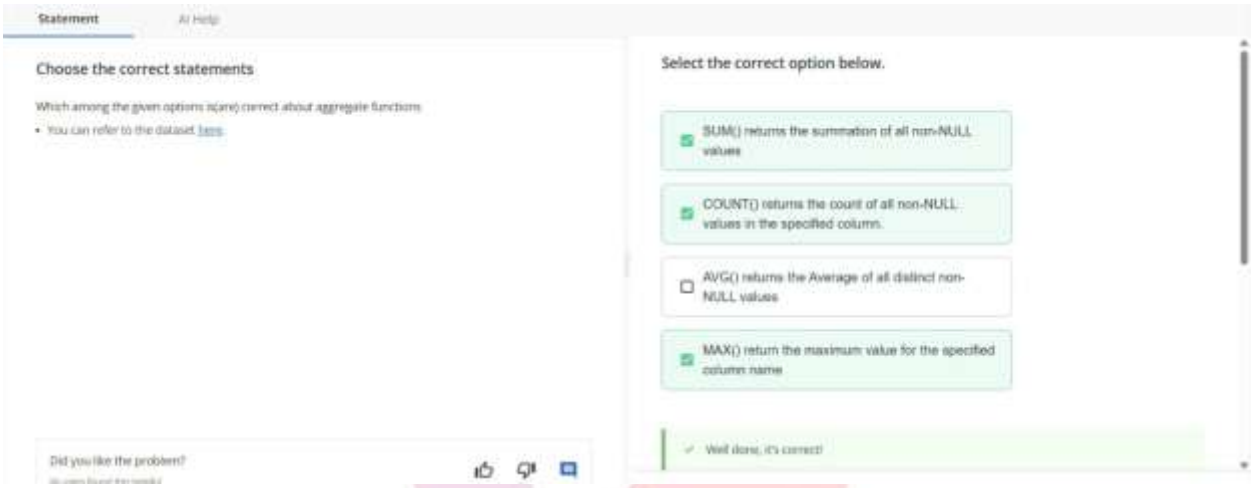
5.NESTED IN AGGREGATE FUNCTIONS



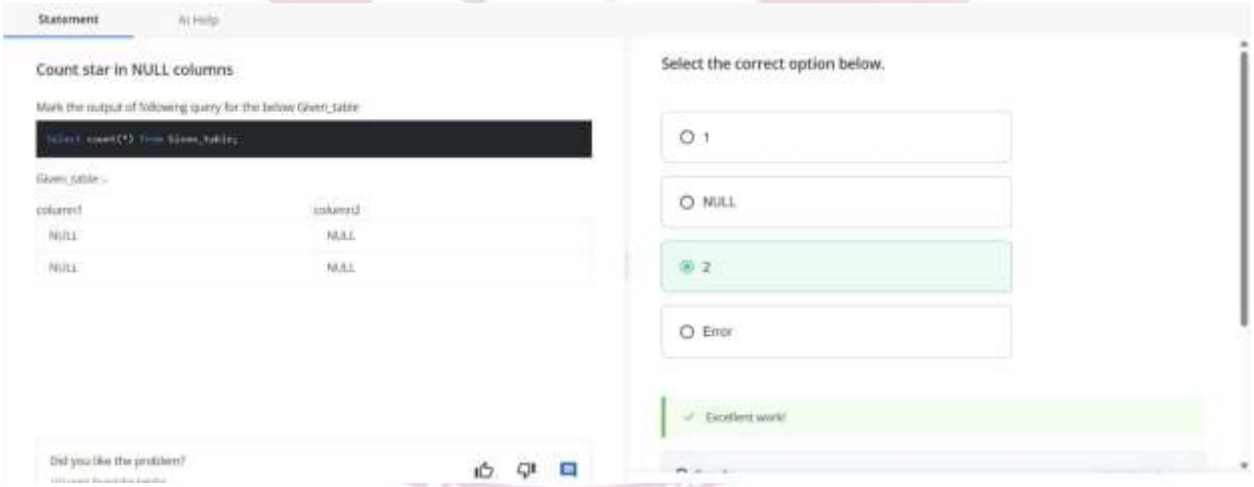
6. OUTPUT OF A QUERY



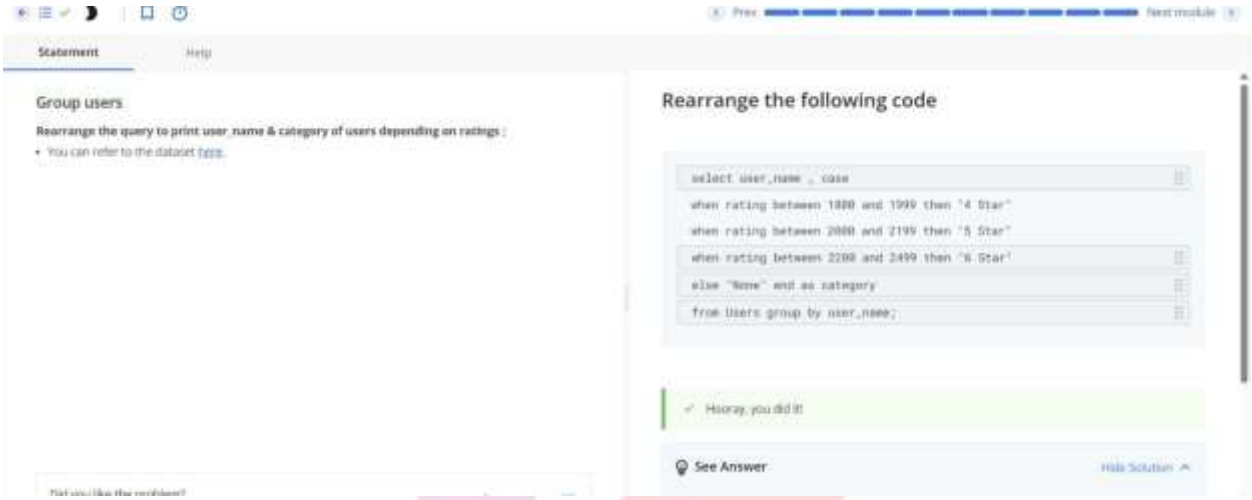
7. CHOOSE THE CORRECT STATEMENTS



8. COUNT STAR IN NULL COLUMNS



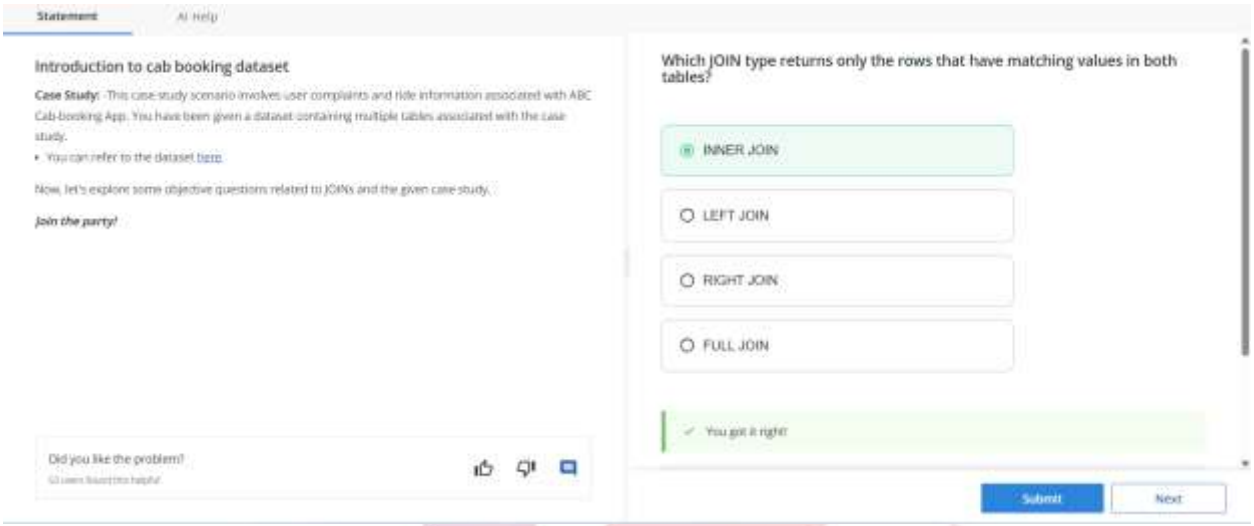
9.GROUP USERS



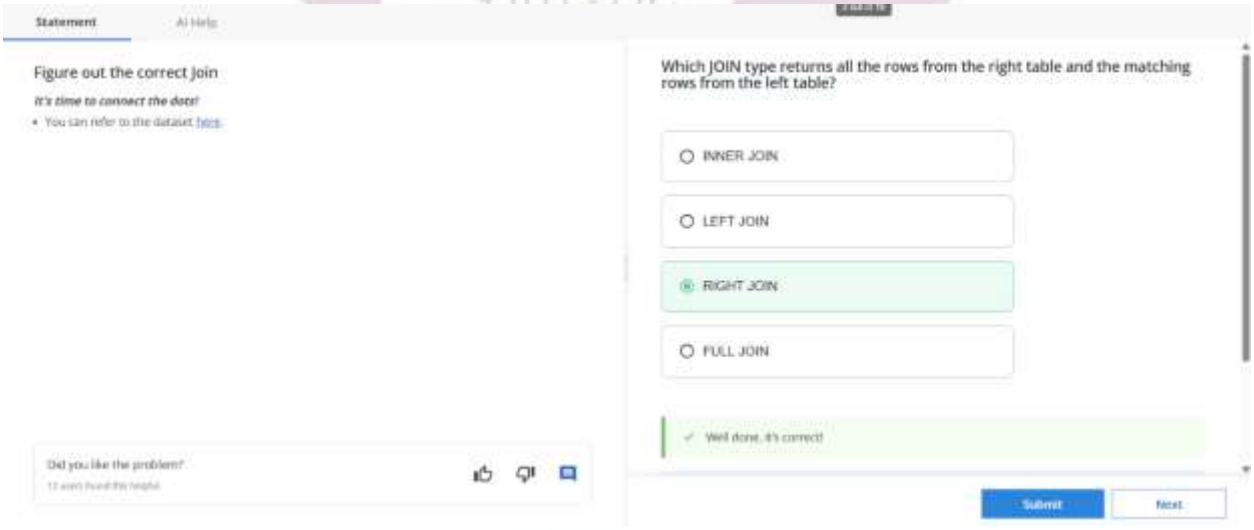
TRUTH-TOIL-TRIUMPH
AUTONOMOUS

CASE STUDY – ABC CAB BOOKIN APP

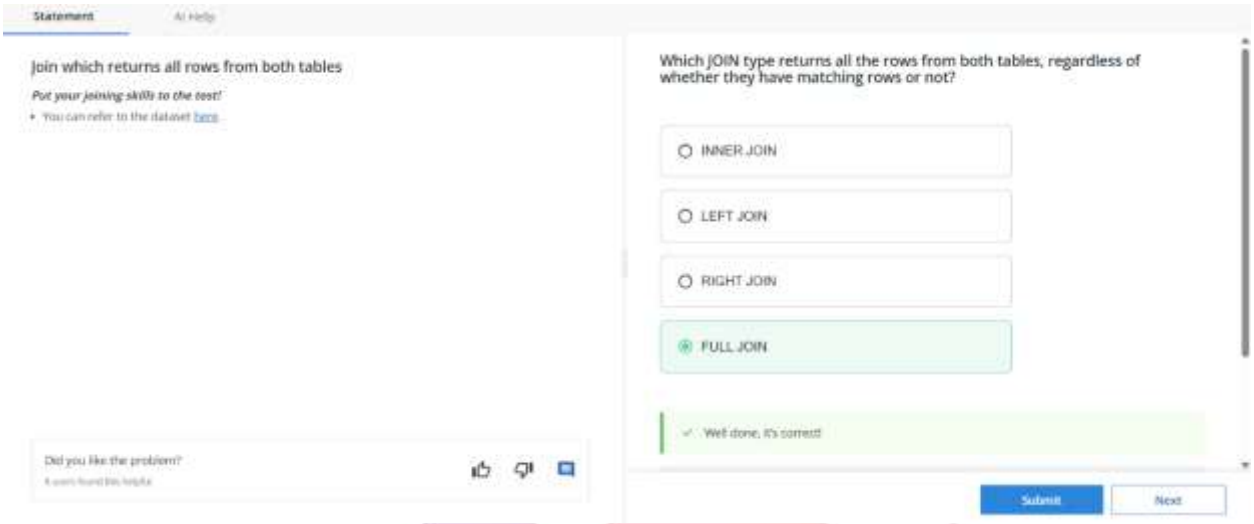
1.INTRODUCTION TO CAB BOOKING DATASET



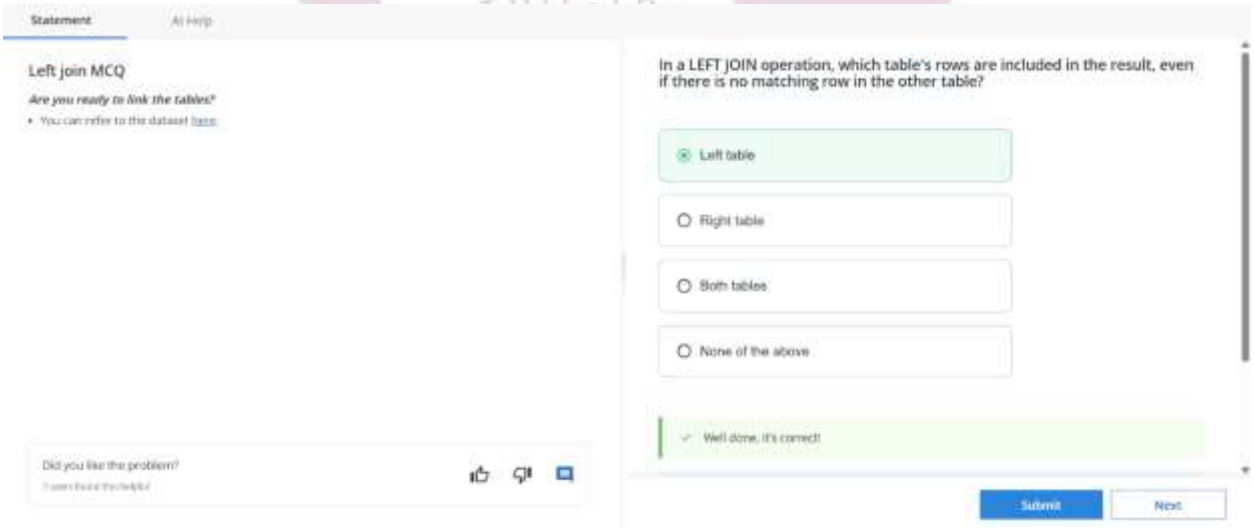
2. FIGURE OUT THE CORRECT JOIN



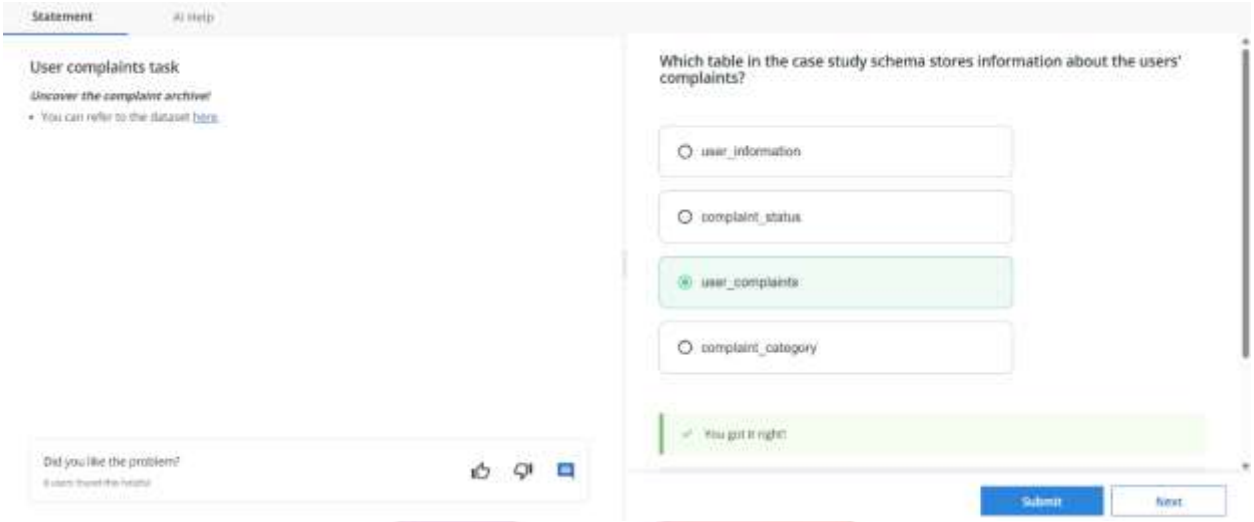
3. JOIN WHICH RETURN ALL ROWS FROM BOTH TABLES



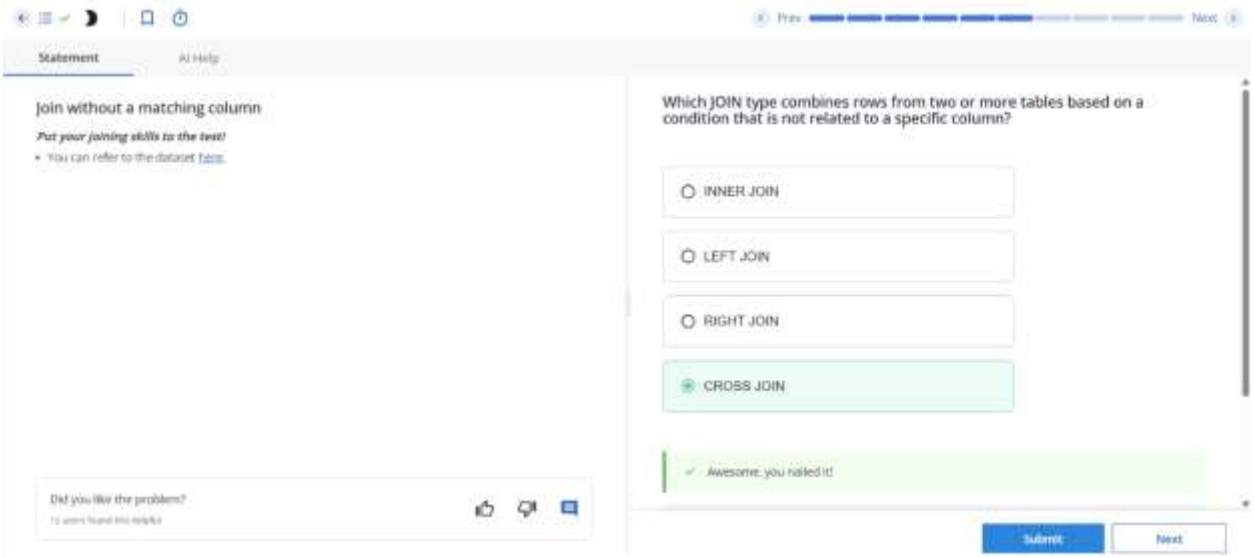
4. LEFT JOIN MCQ



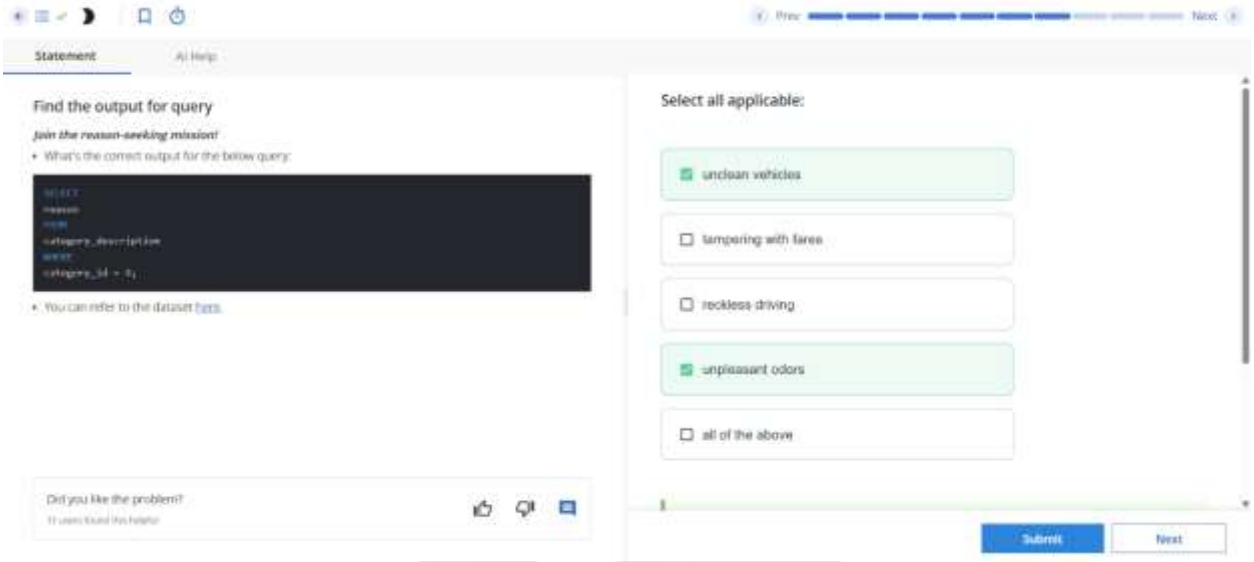
5. USER COMPLAINTS TASK



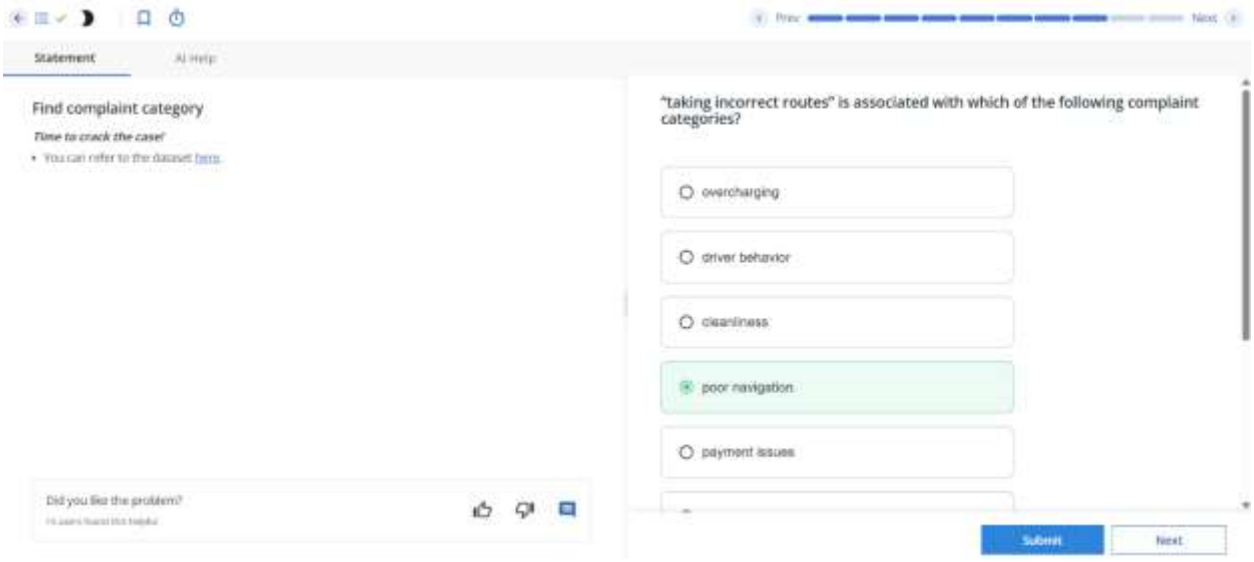
6. JOIN WITHOUT A MATCHING COLUMN



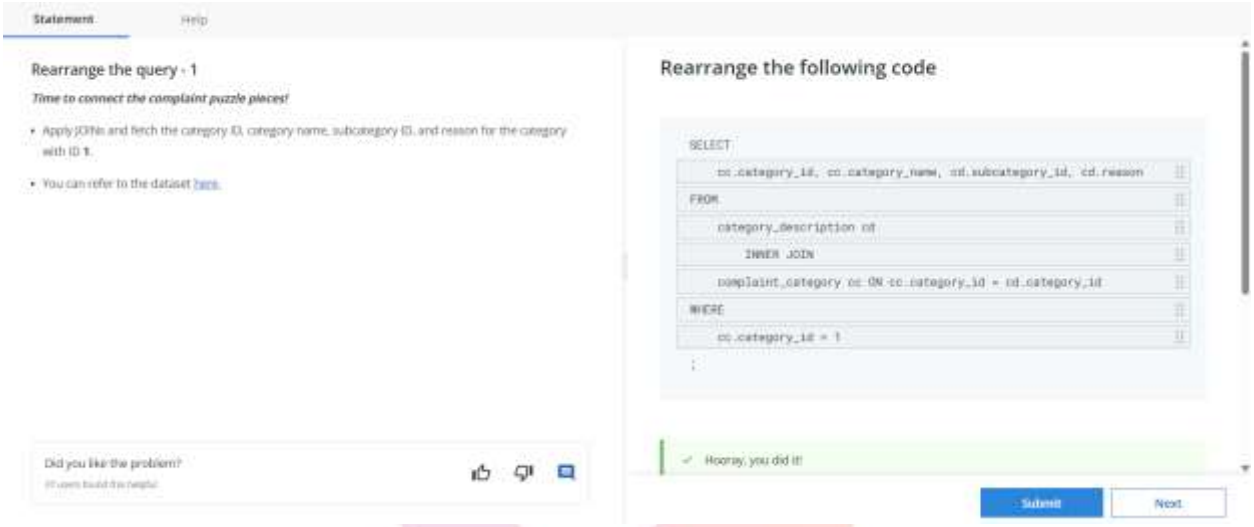
7. FIND THE OUTPUT FOR QUERY



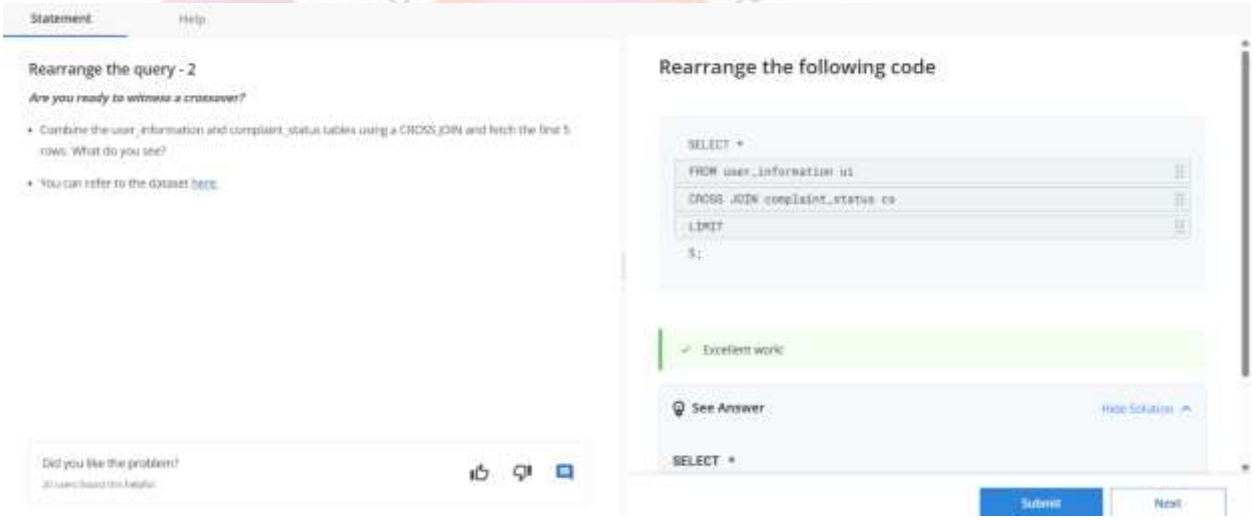
8. FIND COMPLAINT CATEGORY



9. REARRANGE THE QUERY - 1

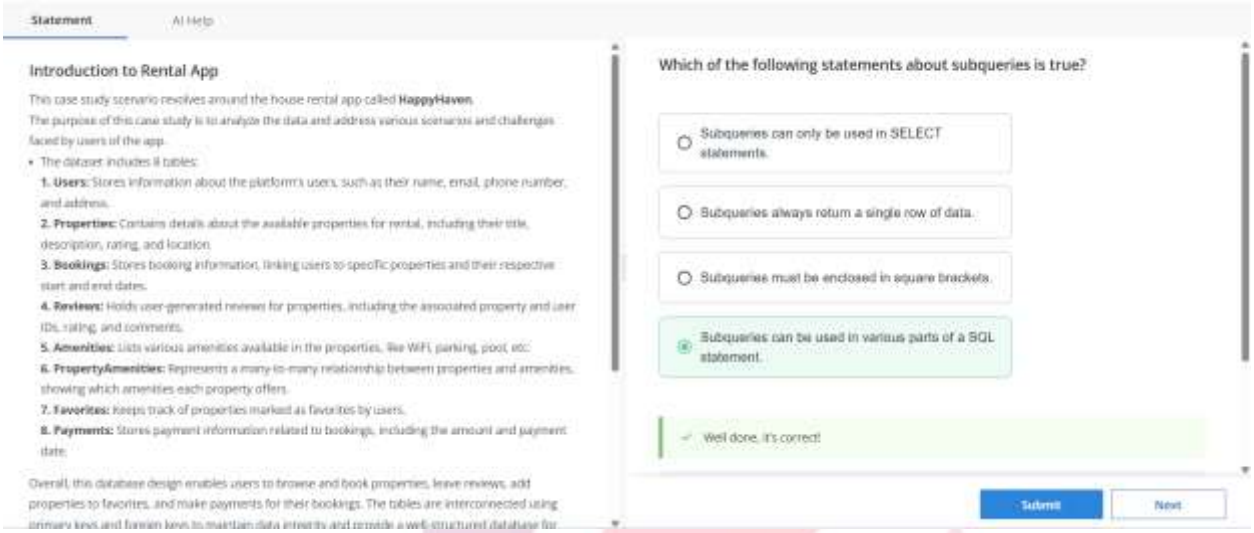


10 . REARRANGE THE QUERY - 2

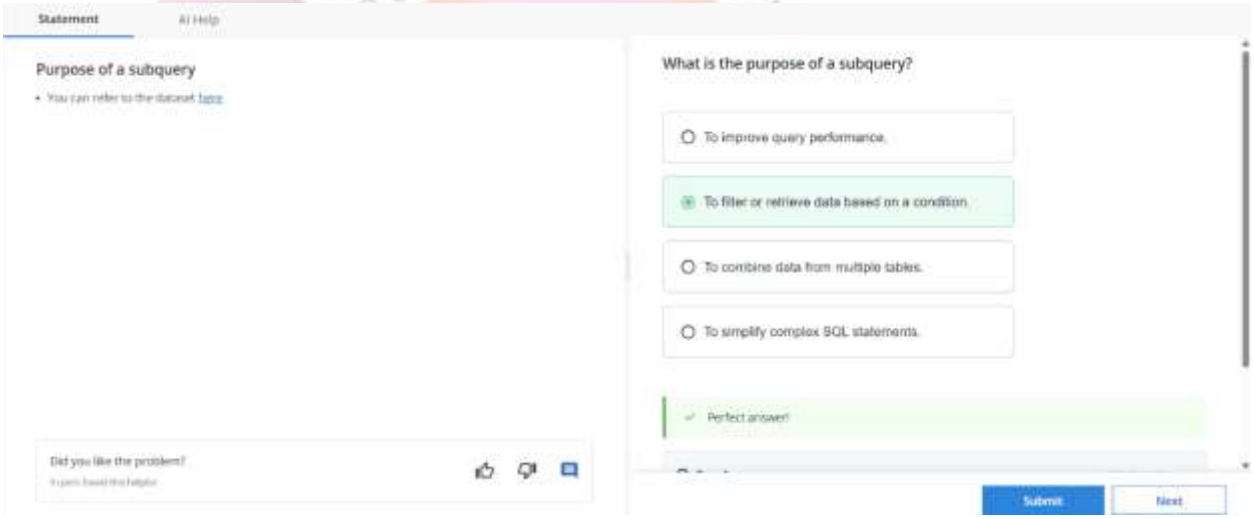


CASE STUDY – HAPPY HAVEN HOUSE RENTAL APP

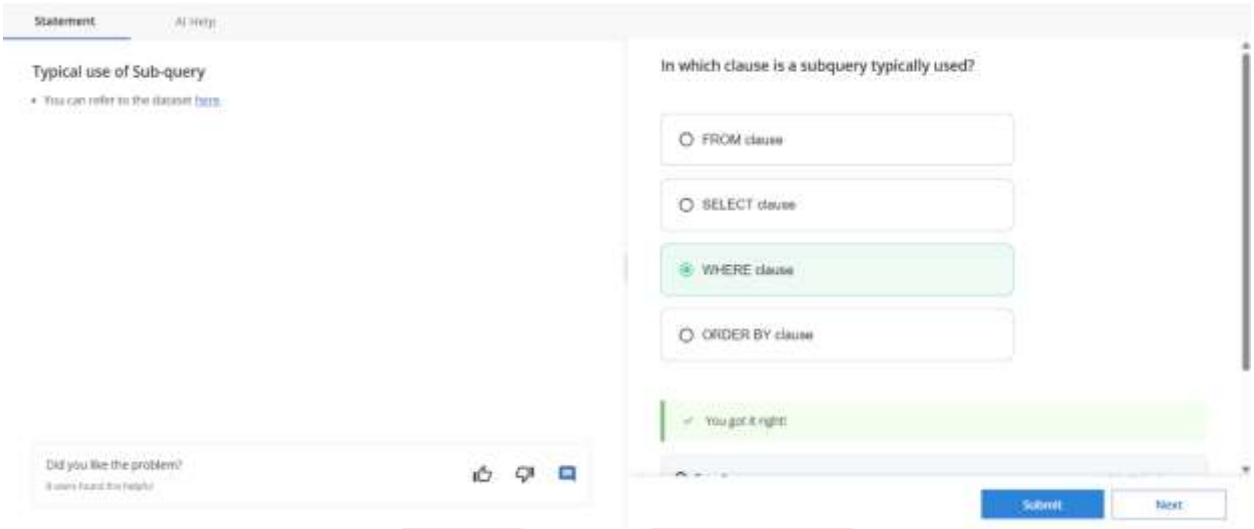
1.INTRODUCTION TO RENTAL APP



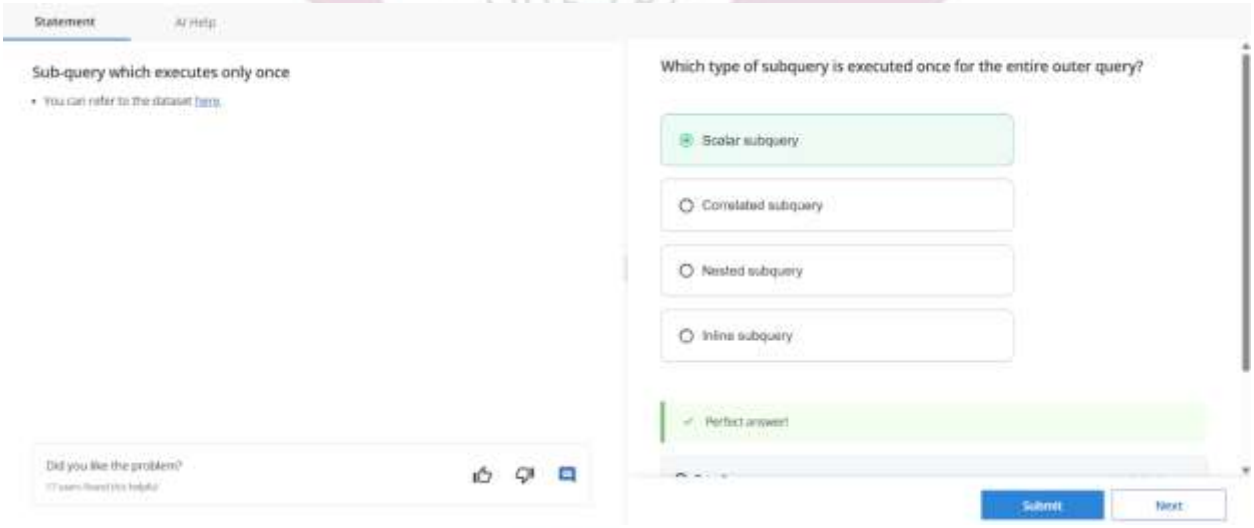
2. PURPOSE OF A SUBQUERY



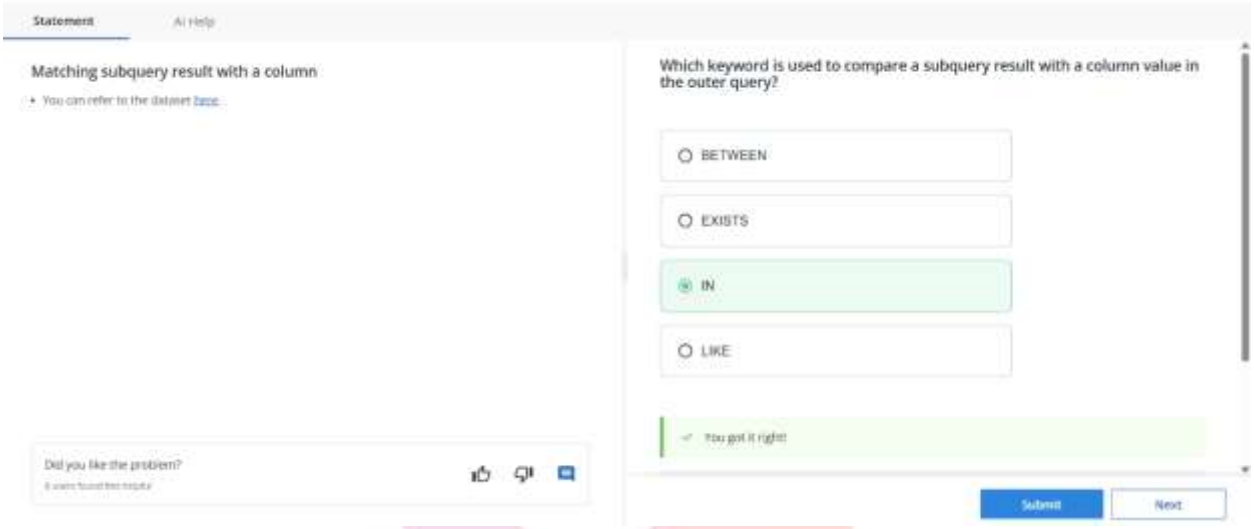
3. TYPICAL USE OF SUB-QUERY



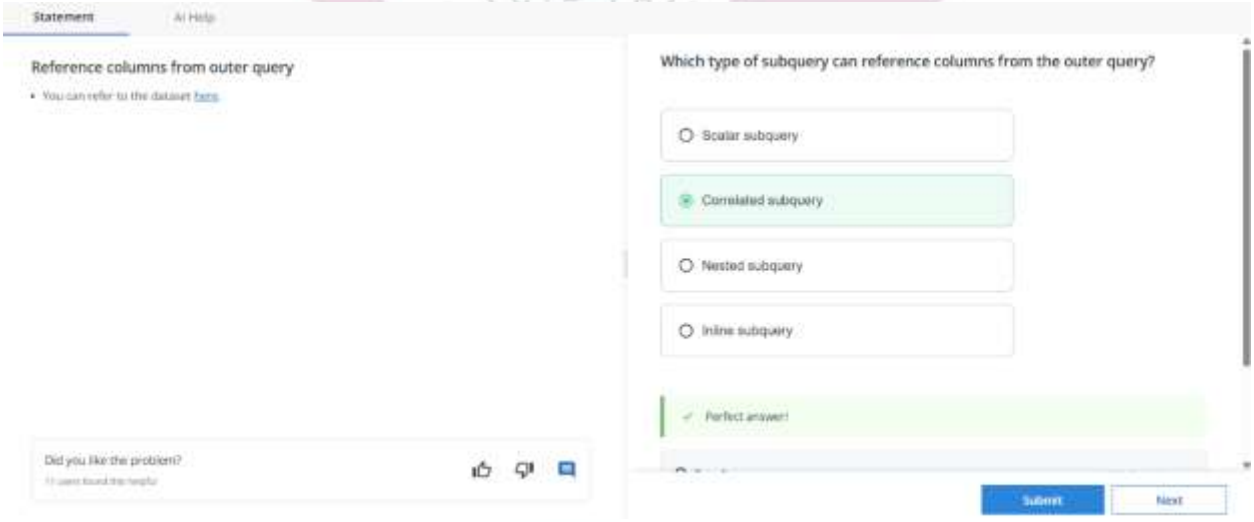
4. SUB-QUERY WHICH EXECUTES ONLY ONCE



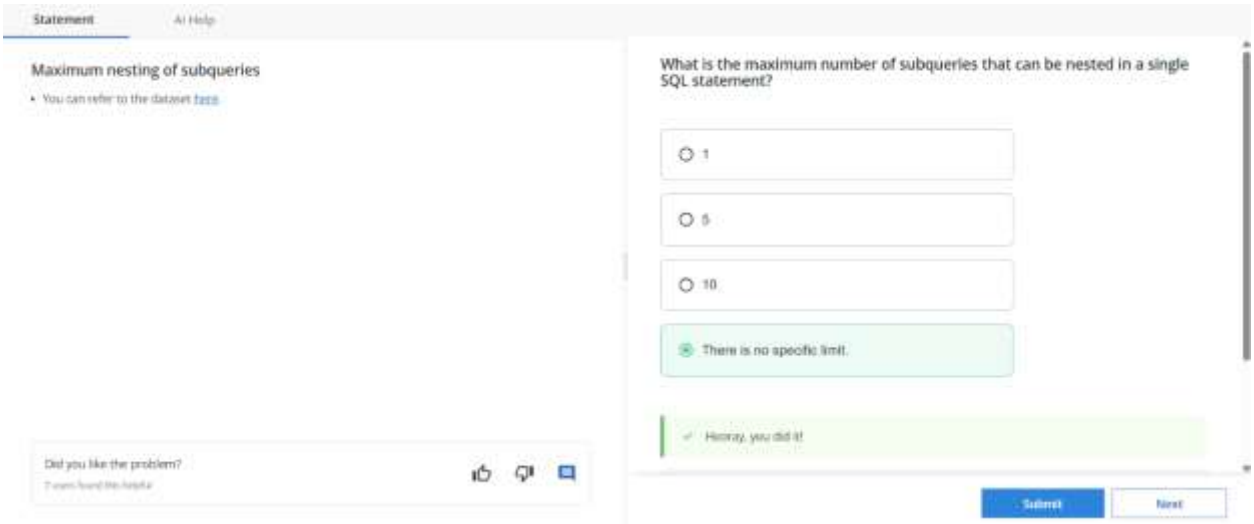
5. MATCHING SUBQUERY RESULT WITH A COLUMN



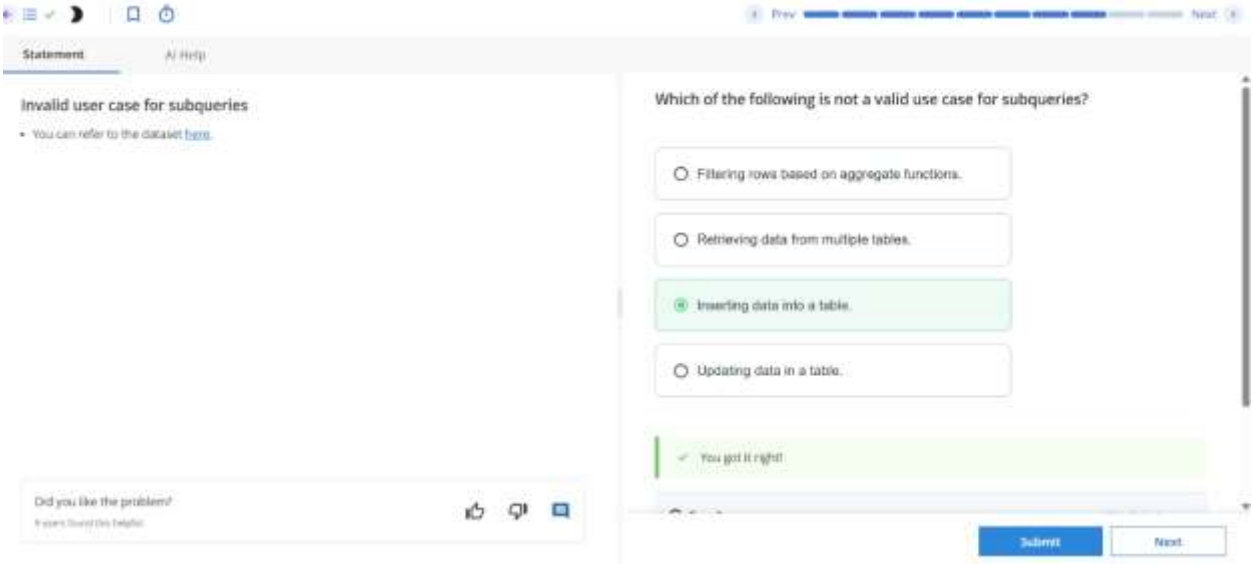
6. REFERENCE COLUMNS FROM OUTER QUERY



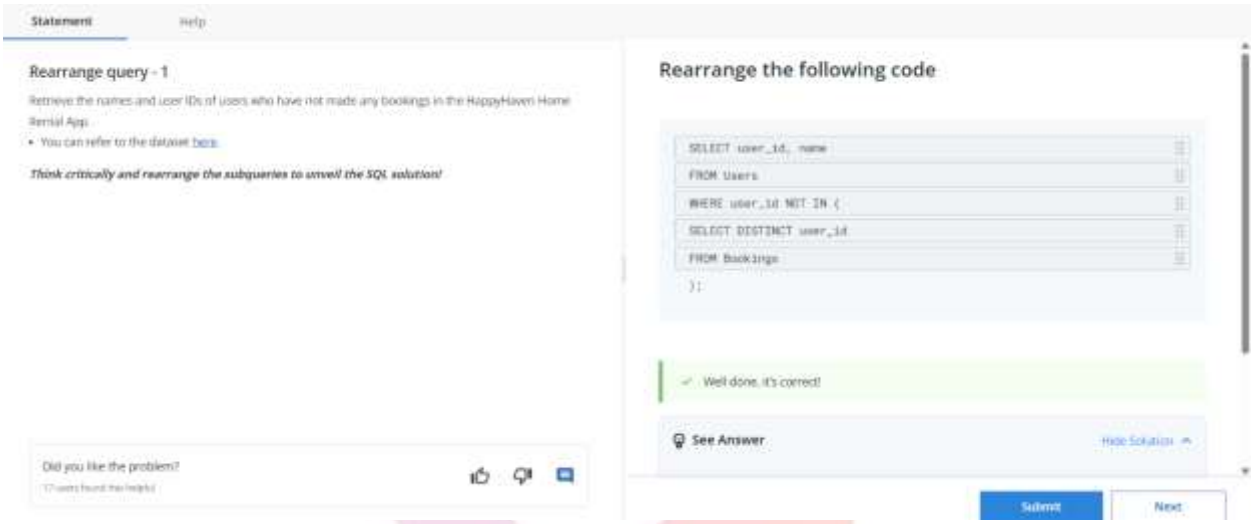
7. MAXIMUM NESTING OF SUBQUERIES



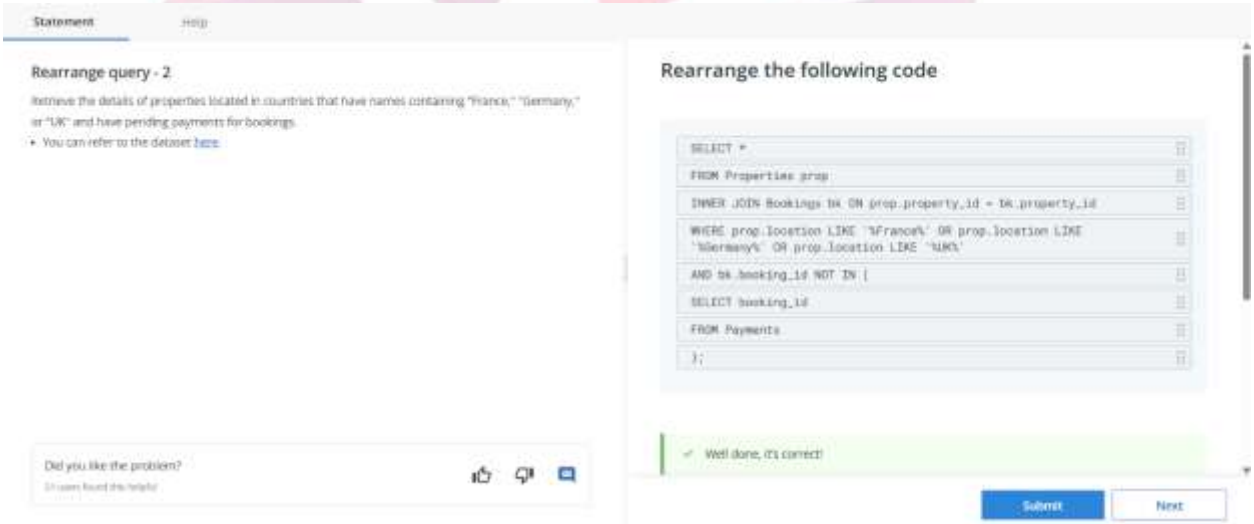
8. INVALID USER CASE FOR SUBQUERIES



9. REARRANGE QUERY-1

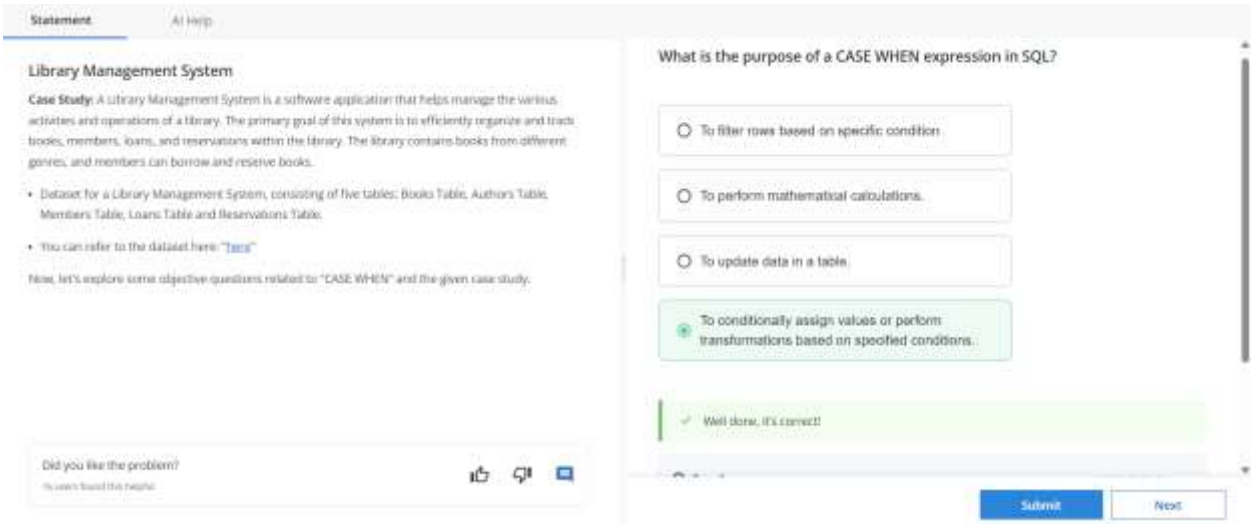


10. REARRANGE QUERY - 2

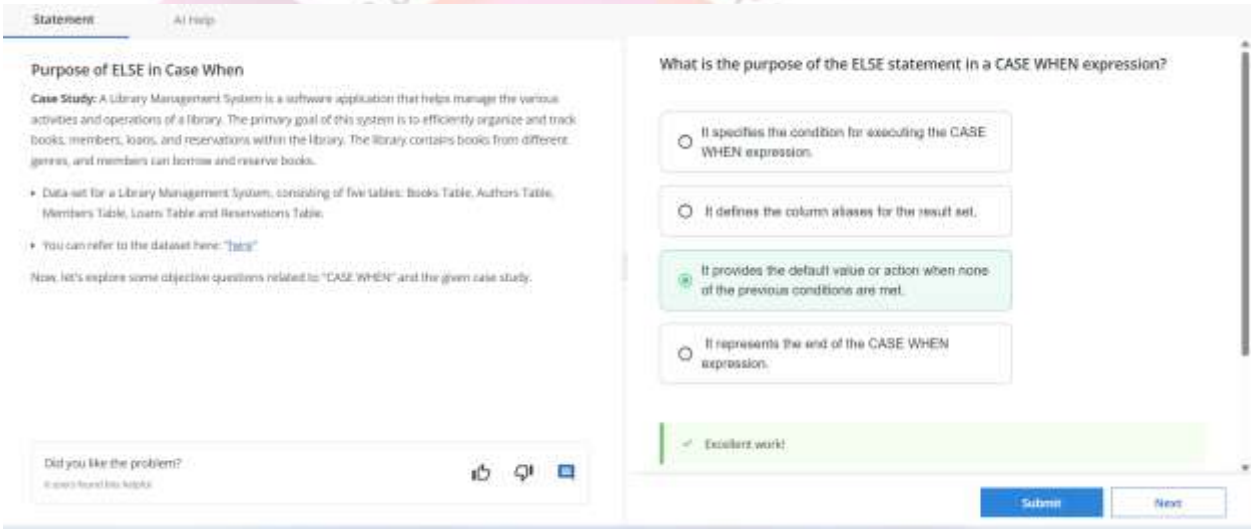


CASE STUDY – LIBRARY MANAGEMENT SYSTEM

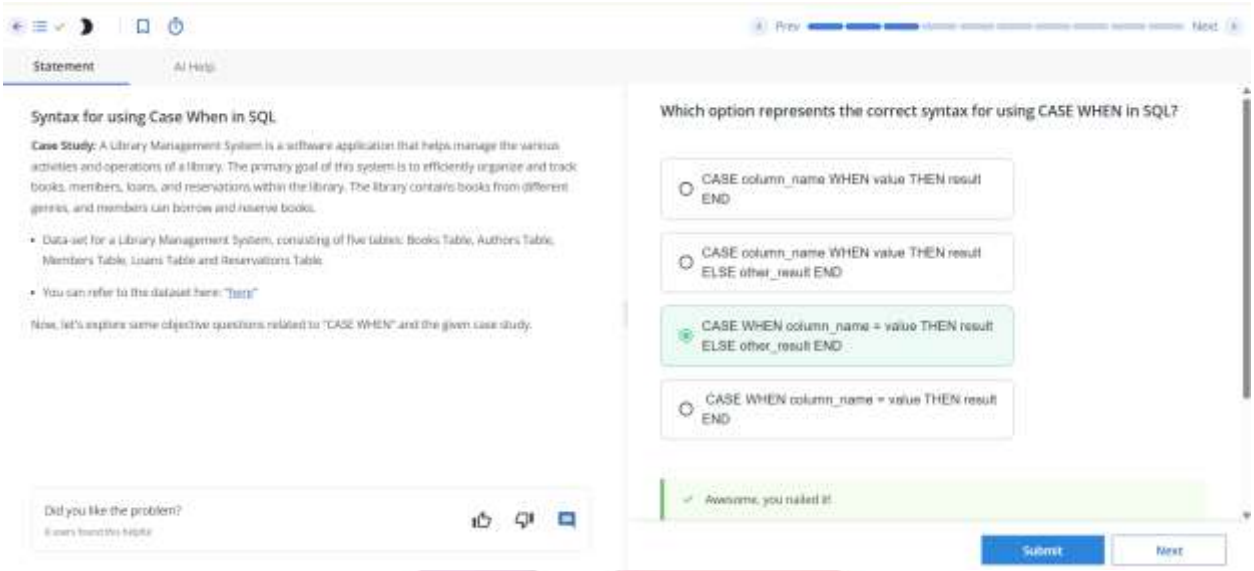
1. LIBRARY MANAGEMENT SYSTEM



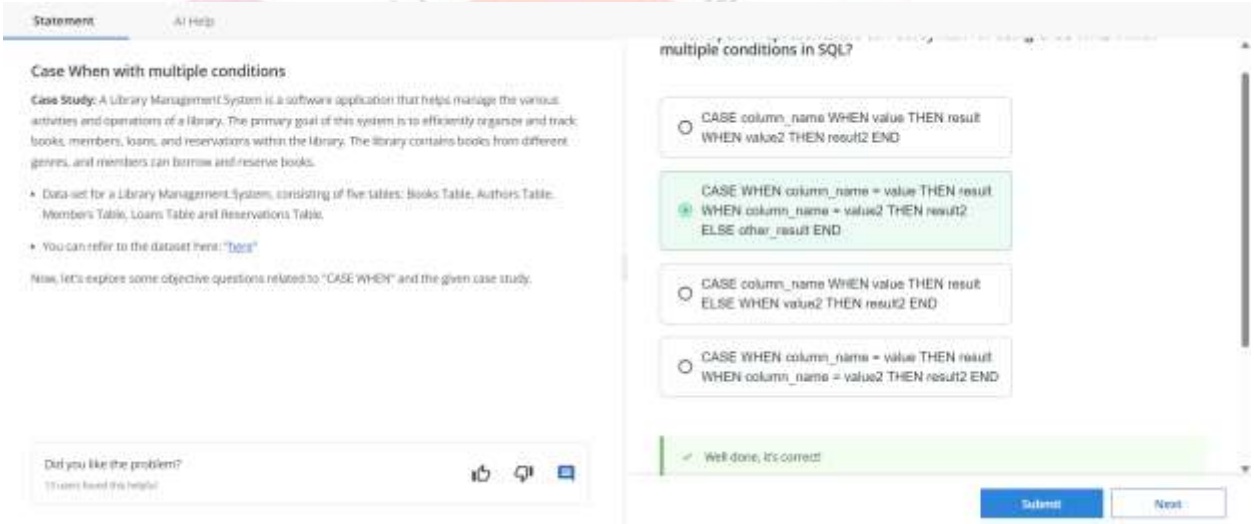
2.PURPOSE OF ELSE IN CASE WHEN



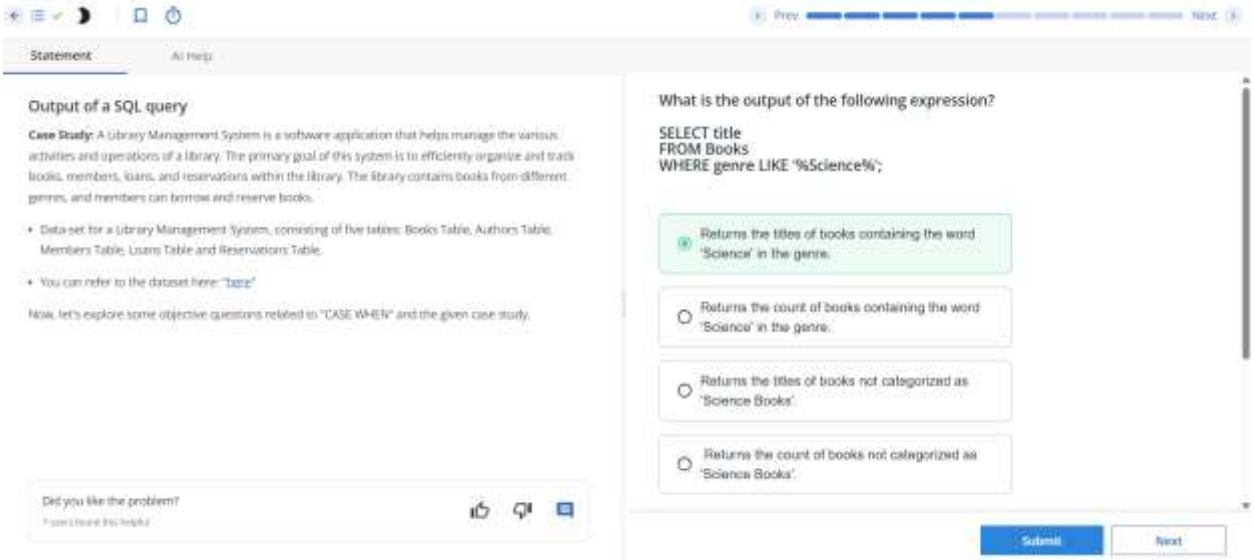
3.SYNTAX FOR USING CASE WHEN IN SQL



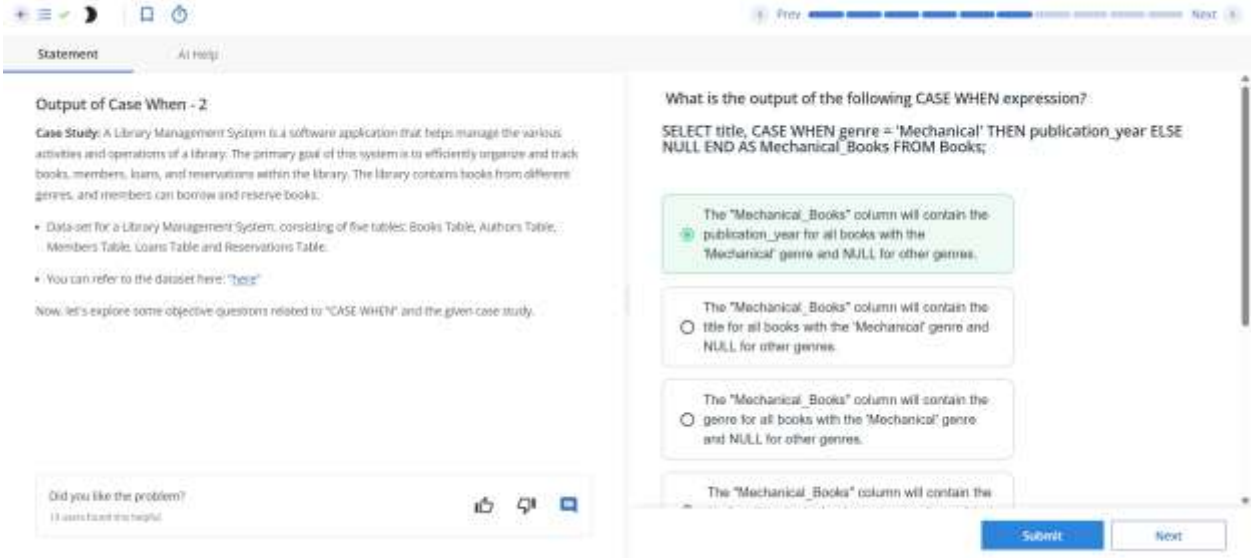
4. CASE WHEN WITH MULTIPLE CONDITIONS



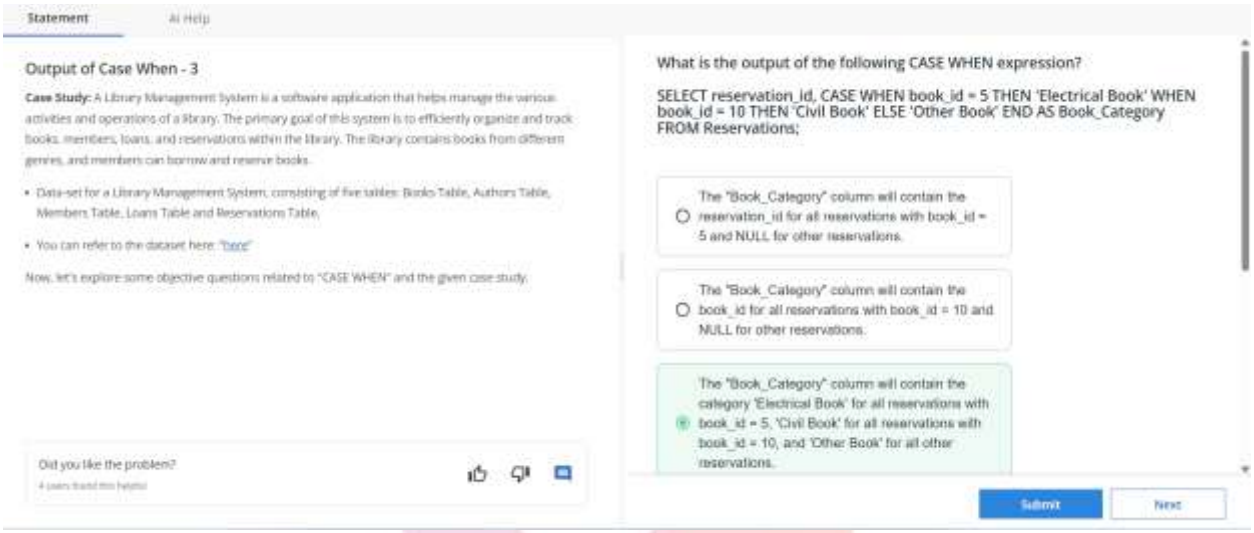
5. OUTPUT OF A SQL QUERY



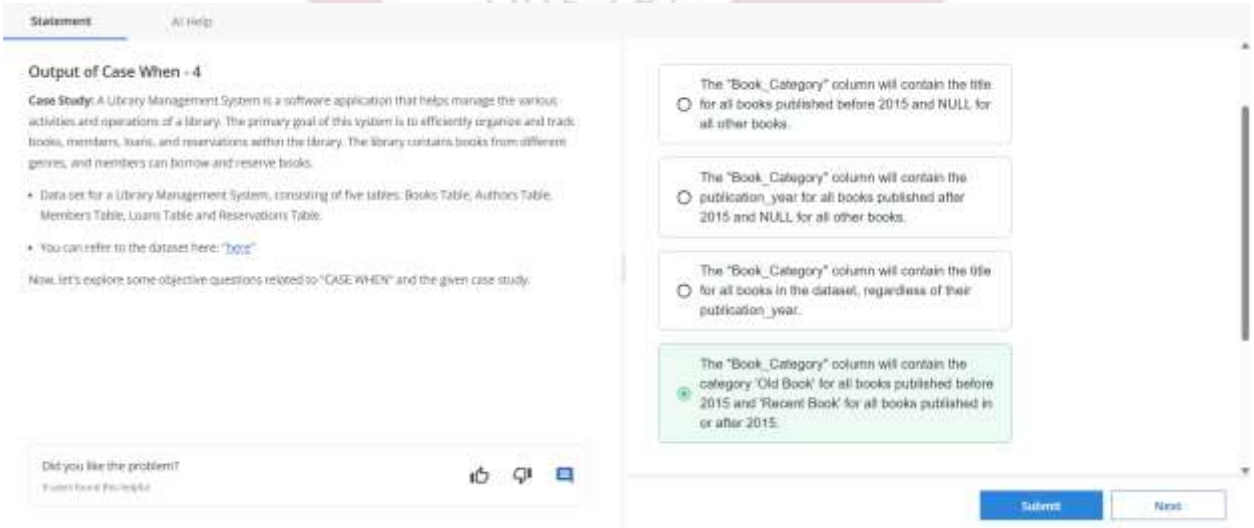
6. OUTPUT OF CASE WHEN - 2



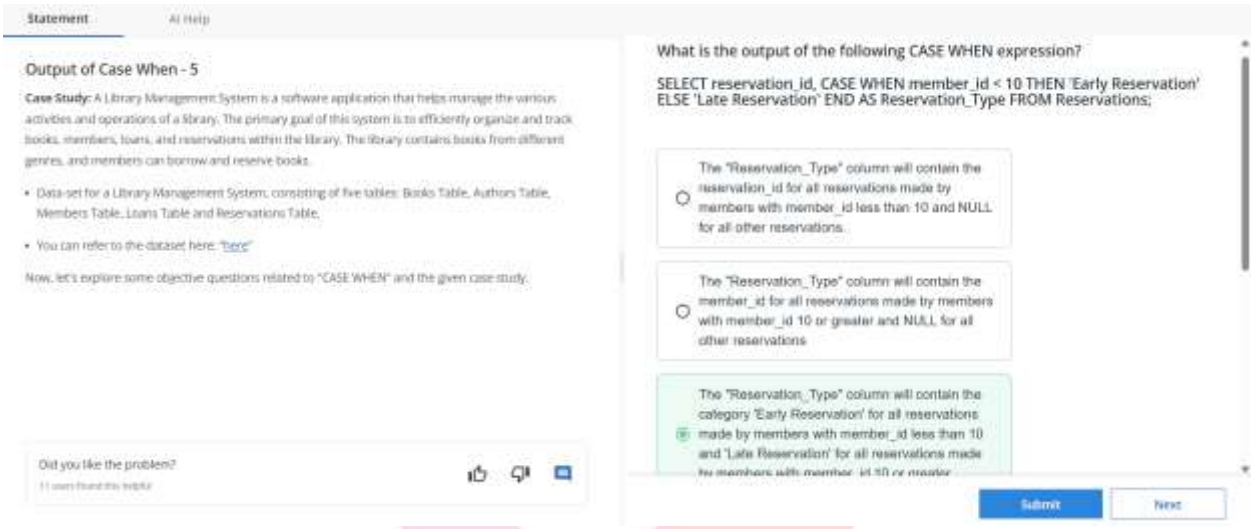
7. OUTPUT OF CASE WHEN - 3



8. OUTPUT OF CASE WHEN - 4



9. OUTPUT OF CASE WHEN -5



10. OUTPUT OF CASE WHEN - 6

